

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐÔNG Á
KHOA CÔNG NGHỆ THÔNG TIN



BÀI TẬP LỚN

HỌC PHẦN: TRÍ TUỆ NHÂN TẠO

**MÃ ĐỀ THI 02: Xây dựng mô phỏng trò chơi tìm
đường trong mê cung bằng BFS, DFS và A***

LỚP TÍN CHỈ : Trí tuệ nhân tạo-1-1-26(N03)

Giảng viên giảng dạy: Trần Xuân Thanh

Danh sách sinh viên thực hiện: Nhóm 10

STT	Mã sinh viên	Sinh viên thực hiện	Lớp hành chính
1	20232139	Nguyễn Thanh Hùng	DCCNTT.14.6
2			
3			

Bắc Ninh, năm 2026

MỤC LỤC

DANH MỤC BẢNG BIỂU	6
DANH MỤC HÌNH ẢNH	7
LỜI CẢM ƠN	9
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI	11
1.1. Đặt vấn đề.....	11
1.2. Lý do lựa chọn.....	13
1.3. Mục tiêu đề tài.....	14
1.4. Phạm vi nghiên cứu	17
1.4.1. Đối tượng nghiên cứu	17
1.4.2. Phạm vi nghiên cứu.....	19
1.5. Phương pháp thực hiện	20
1.6. Ý nghĩa của đề tài.....	22
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	25
2.1. Tổng quan về Trí tuệ nhân tạo.....	25
2.1.1. Khái niệm Trí tuệ nhân tạo	25
2.1.2. Đặc điểm và mục tiêu của Trí tuệ nhân tạo	27
2.1.3. Các lĩnh vực ứng dụng của Trí tuệ nhân tạo	29
2.2. Tổng quan về Học máy	31
2.2.1. Khái niệm Học máy.....	31
2.2.2. Mối quan hệ giữa AI và Học máy	33
2.2.3. Các phương pháp Học máy cơ bản	35
2.3. Logic trong Trí tuệ nhân tạo	37
2.3.1. Khái niệm logic	37
2.3.2. Logic mệnh đề	38

2.3.3.	<i>Logic vị từ</i>	40
2.3.4.	<i>Vai trò của logic trong AI</i>	43
2.4.	<i>Bài toán tìm đường trong mê cung</i>	44
2.5.	<i>Mô hình hóa mê cung dưới dạng đồ thị</i>	48
2.5.1.	<i>Biểu diễn đỉnh của đồ thị</i>	49
2.5.2.	<i>Biểu diễn cạnh giữa các đỉnh</i>	49
2.5.3.	<i>Đồ thị của mê cung</i>	50
2.5.4.	<i>Không gian trạng thái của bài toán</i>	50
2.5.5.	<i>Vai trò của đồ thị đối với BFS, DFS và A*</i>	51
2.5.6.	<i>Ví dụ về quá trình chuyển từ mê cung sang đồ thị</i>	52
2.5.7.	<i>Ý nghĩa của việc mô hình hóa</i>	53
2.6.	<i>Thuật toán BFS</i>	53
2.7.	<i>Thuật toán DFS</i>	58
2.8.	<i>Thuật toán A*</i>	62
2.9.	<i>Hàm heuristic trong A*</i>	65
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG		68
3.1.	<i>Mô tả bài toán</i>	68
3.2.	<i>Yêu cầu chức năng</i>	69
3.3.	<i>Yêu cầu phi chức năng</i>	70
3.4.	<i>Quy tắc hoạt động của mê cung</i>	70
3.5.	<i>Biểu diễn mê cung trong chương trình</i>	71
3.6.	<i>Thiết kế cấu trúc dữ liệu</i>	72
3.7.	<i>Thiết kế chức năng tìm đường</i>	73
3.8.	<i>Thiết kế giao diện mô phỏng</i>	74
3.9.	<i>Luồng hoạt động của chương trình</i>	75
CHƯƠNG 4: XÂY DỰNG CHƯƠNG TRÌNH MÔ PHỎNG		77

4.1.	Môi trường và công nghệ phát triển.....	77
4.2.	Cấu trúc chương trình.....	77
4.3.	Xây dựng mô cung	78
4.4.	Xây dựng điểm bắt đầu và điểm đích.....	80
4.5.	Cài đặt thuật toán BFS	80
4.6.	Cài đặt thuật toán DFS.....	82
4.7.	Cài đặt thuật toán A*.....	83
4.8.	Xây dựng chức năng trực quan hóa quá trình tìm kiếm	85
4.9.	Hiển thị đường đi tìm được	85
4.10.	Các chức năng điều khiển trò chơi	86
4.11.	Giao diện chương trình hoàn thiện	86
CHƯƠNG 5: THỰC NGHIỆM VÀ ĐÁNH GIÁ.....		89
5.1.	Xây dựng các trường hợp thử nghiệm	89
5.2.	Kết quả thực nghiệm thuật toán BFS.....	89
5.3.	Kết quả thực nghiệm thuật toán DFS	90
5.4.	Kết quả thực nghiệm thuật toán A*	90
5.5.	So sánh số ô/trạng thái đã duyệt.....	91
5.6.	So sánh độ dài đường đi	92
5.7.	So sánh thời gian thực thi	94
5.7.	So sánh mức sử dụng bộ nhớ	94
5.8.	Tổng hợp kết quả BFS, DFS và A*.....	95
5.9.	Nhận xét và đánh giá	99
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		101
6.1.	Kết quả đạt được	101
6.2.	Hạn chế của chương trình	101
6.3.	Hướng phát triển trong tương lai	102

TÀI LIỆU THAM KHẢO 103

DANH MỤC BẢNG BIỂU

Bảng 2.1: Bảng ví dụ chân trị	39
Bảng 2.2: Một số thành phần cơ bản trong logic vị từ	42
Bảng 2.3: Bảng kết quả của ví dụ ở hình 2.3	57
Bảng 2.4: Bảng kết quả thuật toán DFS cho hình 2.3	61
Bảng 2.5: Kết quả của thuật toán A* cho hình 2.6	64
Bảng 3.6: So sánh cấu trúc dữ liệu nền tảng của ba thuật toán tìm đường	73
Bảng 5.7: Bảng so sánh dung lượng bộ nhớ đỉnh của tập biên (Nút)	94
Bảng 5.8: Bảng tổng hợp đánh giá ba thuật toán theo 4 tiêu chuẩn Trí tuệ nhân tạo	98

DANH MỤC HÌNH ẢNH

Hình 1.1: Các đối tượng nghiên cứu của đề tài	18
Hình 1.2: Phạm vi di chuyển của tác nhân trong mê cung hai chiều.....	19
Hình 1.3: Quy trình thực hiện đề tài.....	22
Hình 1.4: Một số ứng dụng của bài toán tìm đường trong thực tế	24
Hình 1.5: Một số lĩnh vực và phương pháp cơ bản trong Trí tuệ nhân tạo.....	26
Hình 2.6: Quy trình cơ bản của một hệ thống Trí tuệ nhân tạo	28
Hình 2.7: Một số lĩnh vực ứng dụng của Trí tuệ nhân tạo.....	30
Hình 2.8: Quy trình cơ bản của Học máy.....	32
Hình 2.9: Mối quan hệ giữa Trí tuệ nhân tạo và Học máy	33
Hình 2.10: Các phương pháp Học máy cơ bản	36
Hình 2.11: Vai trò của Logic trong Trí tuệ nhân tạo.....	38
Hình 2.12: Vai trò của Logic trong Trí tuệ nhân tạo.....	44
Hình 2.13: Thuật toán tìm kiếm theo chiều rộng.....	54
Hình 14.2: Ví dụ đồ thị cho bài toán tìm đường đi.....	55
Hình 2.15: Một số cây tìm kiếm sinh ra khi tìm kiếm theo chiều rộng.....	56
Hình 2.16: Thuật toán tìm kiếm theo chiều sâu Tránh các nút lặp:	60
Hình 2.17: Thuật toán A*	63
Hình 2.18: Ví dụ tìm đường đi trên đồ thị từ S tới G	64
Hình 4.19: Giao diện tổng quan hoàn thiện của hệ thống mô phỏng tìm đường trong mê cung.....	86
Hình 4.20: Cận cảnh thanh công cụ điều khiển trung tâm (Control Dock).....	87
Hình 4.21: Mô phỏng cận cảnh thuật toán BFS với các ô duyệt lan tỏa tròn đồng tâm.....	87
Hình 4.22: Mô phỏng cận cảnh thuật toán DFS với nhánh duyệt đâm sâu một hướng.....	87
Hình 4.23: Mô phỏng cận cảnh thuật toán A* với chùm tia định hướng tập trung về đích	88
Hình 5.24: Thực nghiệm trên Bãi vật cản ngẫu nhiên (21×21) - A* vượt trội so với BFS	97

Hình 5.25: Bảng số liệu chi tiết trường hợp Bãi vật cản ngẫu nhiên (A* duyệt 55 ô vs BFS duyệt 290 ô).....	97
Hình 5.26: Thực nghiệm trên Mê cung xoắn ốc (21×21) - Minh chứng hạn chế của DFS khi bị bẫy	97
Hình 5.27: Thực nghiệm trên Bẫy vô nghiệm - Cả ba thuật toán kết luận chính xác không có đường đi.....	98
Hình 5.28: Thực nghiệm trên Bản đồ phòng và hành lang (Rooms & Corridors 21×21)	98

LỜI CẢM ƠN

Trong quá trình học tập và thực hiện bài tập lớn học phần Trí tuệ nhân tạo, nhóm đã nhận được sự quan tâm, hướng dẫn và hỗ trợ từ phía Nhà trường, Khoa Công nghệ Thông tin và giảng viên phụ trách. Đây là những yếu tố quan trọng giúp nhóm có thể vận dụng kiến thức đã học để nghiên cứu và hoàn thành đề tài “*Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A**”.

Trước hết, nhóm xin trân trọng cảm ơn Ban Giám hiệu Trường Đại học Công nghệ Đông Á cùng Khoa Công nghệ Thông tin đã tạo điều kiện về môi trường học tập, chương trình đào tạo và cơ sở vật chất, giúp sinh viên có cơ hội tiếp cận và nghiên cứu các kiến thức chuyên ngành.

Nhóm xin đặc biệt gửi lời cảm ơn sâu sắc đến ThS. Trần Xuân Thanh, giảng viên phụ trách học phần, đã tận tình giảng dạy, hướng dẫn và định hướng cho nhóm trong suốt quá trình thực hiện đề tài. Những kiến thức chuyên môn, nhận xét và góp ý của giảng viên đã giúp nhóm từng bước hoàn thiện cách tiếp cận bài toán, triển khai các thuật toán BFS, DFS và A*, đồng thời nâng cao khả năng phân tích, lập trình và đánh giá kết quả.

Thông qua quá trình thực hiện đề tài, nhóm có điều kiện củng cố những kiến thức về trí tuệ nhân tạo, thuật toán tìm kiếm, biểu diễn không gian trạng thái và lý thuyết đồ thị, đồng thời rèn luyện kỹ năng làm việc nhóm, giải quyết vấn đề và vận dụng kiến thức lý thuyết vào một bài toán thực tế.

Mặc dù nhóm đã nghiêm túc và cố gắng trong quá trình thực hiện, do thời gian nghiên cứu và kiến thức thực tế còn hạn chế, báo cáo không thể tránh khỏi những thiếu sót. Nhóm kính mong nhận được những ý kiến nhận xét và góp ý của giảng viên để có thể tiếp tục bổ sung, hoàn thiện kiến thức và nâng cao chất lượng các sản phẩm nghiên cứu trong tương lai.

Một lần nữa, nhóm xin trân trọng cảm ơn Trường Đại học Công nghệ Đông Á, Khoa Công nghệ Thông tin và ThS. Trần Xuân Thanh đã tận tình hướng dẫn và tạo điều kiện để nhóm hoàn thành bài tập lớn này.

Hà Nội, năm 2026

LỜI MỞ ĐẦU

Trong những năm gần đây, trí tuệ nhân tạo ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực của đời sống và công nghệ. Một trong những nội dung cơ bản của trí tuệ nhân tạo là bài toán tìm kiếm, trong đó máy tính cần xác định một hướng đi phù hợp từ trạng thái ban đầu đến trạng thái đích trong một không gian có nhiều khả năng lựa chọn.

Bài toán tìm đường trong mê cung là một ví dụ trực quan để nghiên cứu vấn đề này. Trong mê cung, từ một vị trí ban đầu, người chơi phải tìm cách di chuyển đến vị trí đích mà không đi qua các vị trí bị chặn. Khi kích thước mê cung tăng lên hoặc xuất hiện nhiều nhánh và ngõ cụt, việc tìm đường trở nên phức tạp hơn. Đây cũng là điều kiện phù hợp để áp dụng các thuật toán tìm kiếm và so sánh cách thức hoạt động của chúng.

Trong bài tập lớn này, nhóm thực hiện đề tài “*Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A**”. Đề tài tập trung xây dựng một chương trình mô phỏng mê cung và áp dụng ba thuật toán Breadth-First Search (BFS), Depth-First Search (DFS) và A* để tìm đường từ điểm bắt đầu đến điểm đích. Thông qua chương trình, quá trình tìm kiếm của từng thuật toán được thể hiện trực quan, đồng thời kết quả của các thuật toán được so sánh dựa trên những tiêu chí như đường đi tìm được, số lượng ô đã duyệt và thời gian thực hiện.

Việc lựa chọn BFS, DFS và A* giúp nhóm có thể thấy rõ sự khác nhau giữa các phương pháp tìm kiếm. BFS tìm kiếm theo từng mức, DFS ưu tiên đi sâu theo từng nhánh, trong khi A* sử dụng thêm thông tin ước lượng để hướng quá trình tìm kiếm về phía đích. Qua đó, đề tài không chỉ giúp nhóm hiểu rõ hơn về nguyên lý của các thuật toán mà còn cho thấy cách chúng được áp dụng vào một bài toán cụ thể.

Báo cáo trình bày quá trình từ tìm hiểu cơ sở lý thuyết, phân tích bài toán, thiết kế và xây dựng chương trình đến thực nghiệm, so sánh và đánh giá kết quả. Qua quá trình thực hiện, nhóm có thêm kinh nghiệm trong việc vận dụng kiến thức môn học vào xây dựng một chương trình hoàn chỉnh, đồng thời hiểu rõ hơn về ưu điểm và hạn chế của từng thuật toán trong bài toán tìm đường.

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Đặt vấn đề

Trong quá trình phát triển của công nghệ thông tin, trí tuệ nhân tạo ngày càng được ứng dụng rộng rãi trong các bài toán yêu cầu máy tính tự đưa ra quyết định và tìm ra phương án xử lý phù hợp. Một trong những vấn đề nền tảng của trí tuệ nhân tạo là bài toán tìm kiếm, trong đó hệ thống phải lựa chọn và đánh giá nhiều trạng thái khác nhau để tìm được lời giải từ trạng thái ban đầu đến trạng thái đích. Mặc dù đây là một bài toán cơ bản, phương pháp tìm kiếm được lựa chọn có ảnh hưởng trực tiếp đến thời gian xử lý, lượng tài nguyên sử dụng và chất lượng của lời giải.

Bài toán tìm đường trong mê cung là một ví dụ điển hình giúp minh họa rõ ràng cho vấn đề trên. Xét một mê cung được tạo thành từ các ô trên một mặt phẳng, trong đó có những ô có thể đi qua và những ô bị chặn. Người chơi bắt đầu tại một vị trí xác định và cần di chuyển đến vị trí đích thông qua các ô hợp lệ. Với một mê cung đơn giản, con người có thể dễ dàng quan sát và lựa chọn đường đi. Tuy nhiên, khi số lượng ô tăng lên, mê cung xuất hiện nhiều nhánh rẽ, ngõ cụt và các đường đi khác nhau, việc xác định đường đi bằng phương pháp thử và sai sẽ trở nên kém hiệu quả. Đối với máy tính, vấn đề còn rõ ràng hơn khi số lượng trạng thái cần xem xét có thể tăng nhanh theo kích thước của không gian tìm kiếm.

Bài toán không chỉ đặt ra yêu cầu tìm được một đường đi, mà còn đặt ra câu hỏi về cách tìm kiếm như thế nào để đạt hiệu quả tốt hơn. Trong cùng một mê cung, các thuật toán khác nhau có thể đưa ra những kết quả khác nhau. Có thuật toán ưu tiên khám phá toàn bộ các vị trí ở cùng một khoảng cách trước khi đi xa hơn; có thuật toán lại đi sâu vào một hướng và chỉ quay lại khi không thể tiếp tục; trong khi một số thuật toán sử dụng thêm thông tin ước lượng để ưu tiên những vị trí có khả năng dẫn đến đích nhanh hơn. Vì vậy, việc nghiên cứu và so sánh các chiến lược tìm kiếm là cần thiết để hiểu được mối quan hệ giữa cách thức tìm kiếm và hiệu quả của thuật toán.

Trong phạm vi đề tài, nhóm lựa chọn ba thuật toán Breadth-First Search (BFS), Depth-First Search (DFS) và A*. Đây là ba phương pháp có cách tiếp cận tương đối khác nhau nhưng đều có thể áp dụng cho bài toán tìm đường trong mê cung. BFS tổ chức quá trình tìm kiếm

theo từng lớp và có ưu điểm là tìm được đường đi ngắn nhất trong trường hợp chi phí di chuyển giữa các bước là như nhau. DFS tập trung khám phá theo chiều sâu, có cách triển khai tương đối đơn giản nhưng không đảm bảo đường đi tìm được là ngắn nhất. Trong khi đó, A* kết hợp chi phí đã đi với một giá trị ước lượng khoảng cách đến đích, từ đó định hướng quá trình tìm kiếm và có khả năng giảm số lượng trạng thái cần khám phá trong những trường hợp phù hợp.

Tuy nhiên, nếu chỉ nghiên cứu các thuật toán dưới dạng lý thuyết thì rất khó quan sát đầy đủ sự khác biệt giữa chúng. Vì vậy, việc xây dựng một chương trình mô phỏng là cần thiết. Thông qua mô phỏng, quá trình khám phá từng ô trong mê cung có thể được thể hiện trực quan, giúp người sử dụng quan sát cách BFS, DFS và A* lựa chọn trạng thái tiếp theo, cách các thuật toán xử lý những nhánh rẽ và ngõ cụt, cũng như cách chúng xây dựng đường đi từ điểm bắt đầu đến điểm đích. Đồng thời, chương trình có thể cung cấp các thông tin thực nghiệm như số lượng trạng thái đã duyệt, độ dài đường đi và thời gian tìm kiếm để làm cơ sở đánh giá.

Xuất phát từ những yêu cầu trên, nhóm lựa chọn đề tài **“Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A*”**. Đề tài hướng đến việc xây dựng một môi trường mê cung dạng lưới, xác định điểm bắt đầu và điểm đích, sau đó áp dụng lần lượt ba thuật toán để giải bài toán tìm đường. Kết quả của từng thuật toán được ghi nhận và so sánh trên cùng một điều kiện thử nghiệm. Qua đó, đề tài không chỉ minh họa cách hoạt động của các thuật toán tìm kiếm mà còn cho phép đánh giá một cách trực quan và thực nghiệm sự khác biệt về hiệu quả giữa BFS, DFS và A*.

Việc thực hiện đề tài có ý nghĩa cả về mặt học tập và thực hành. Về mặt học tập, đề tài giúp củng cố kiến thức về không gian trạng thái, tìm kiếm trong trí tuệ nhân tạo, cấu trúc dữ liệu và tư duy giải quyết bài toán. Về mặt thực hành, việc chuyển từ thuật toán trên lý thuyết thành một chương trình mô phỏng hoàn chỉnh giúp nhóm hiểu rõ hơn những vấn đề phát sinh trong quá trình cài đặt, chẳng hạn như biểu diễn mê cung, kiểm tra trạng thái đã thăm, lưu lại trạng thái cha và truy vết đường đi. Đây cũng là cơ sở để tiếp tục nghiên cứu những bài toán tìm kiếm có quy mô và mức độ phức tạp cao hơn.

1.2. Lý do lựa chọn

Trí tuệ nhân tạo ngày nay không còn là một lĩnh vực xa lạ hay chỉ tồn tại trong các phòng nghiên cứu. Từ trợ lý ảo trên điện thoại, hệ thống gợi ý của các nền tảng mạng xã hội, cho đến xe tự lái hay robot trong nhà máy sản xuất, dấu ấn của trí tuệ nhân tạo hiện diện trong hầu hết mọi lĩnh vực của đời sống hiện đại. Trong bối cảnh đó, việc trang bị kiến thức nền tảng về AI, đặc biệt là các phương pháp tìm kiếm và giải quyết bài toán, trở thành yêu cầu thiết yếu đối với sinh viên ngành Công nghệ thông tin.

Trong quá trình học tập môn Trí tuệ nhân tạo, nhóm nhận thấy rằng các thuật toán tìm kiếm như BFS, DFS và A* thường được giới thiệu thông qua lý thuyết và mã giả, nhưng ít có cơ hội để quan sát trực tiếp cách chúng hoạt động trên một bài toán thực tế. Điều này khiến người học khó nhận ra sự khác biệt thực sự giữa các thuật toán: thuật toán nào duyệt nhiều ô hơn, thuật toán nào cho đường đi ngắn hơn, và trong hoàn cảnh nào thì mỗi thuật toán phát huy được ưu điểm của mình. Đây chính là xuất phát điểm để nhóm nghĩ đến việc xây dựng một chương trình mô phỏng trực quan, nơi người dùng có thể tự mình quan sát và so sánh ba thuật toán trên cùng một môi trường.

Lý do nhóm chọn bài toán tìm đường trong mê cung là vì đây là một bài toán vừa trực quan, vừa đủ phức tạp để thể hiện rõ bản chất của từng thuật toán. Một mê cung có thể chứa nhiều nhánh rẽ, nhiều ngõ cụt, nhiều đường đi khác nhau dẫn đến đích. Khi kích thước mê cung tăng lên, sự khác biệt giữa cách BFS mở rộng từng lớp, cách DFS lao thẳng vào một nhánh, và cách A* dùng thông tin ước lượng để định hướng tìm kiếm trở nên rõ ràng hơn bao giờ hết. Đây là môi trường lý tưởng để không chỉ kiểm chứng lý thuyết mà còn đánh giá thuật toán dựa trên kết quả thực nghiệm cụ thể.

Việc lựa chọn đồng thời cả ba thuật toán BFS, DFS và A* cũng có chủ đích rõ ràng. Ba thuật toán này đại diện cho ba triết lý tìm kiếm khác nhau: BFS đảm bảo tìm được đường đi ngắn nhất nhưng tốn bộ nhớ; DFS đơn giản và tiết kiệm bộ nhớ nhưng không đảm bảo chất lượng đường đi; A* cân bằng giữa hai hướng đó bằng cách sử dụng hàm heuristic để ưu tiên các trạng thái có triển vọng hơn. Khi đặt cả ba vào cùng một mê cung và đo lường các chỉ số như số ô đã duyệt, độ dài đường đi, thời gian thực hiện, người học có thể rút ra nhận xét dựa trên bằng chứng thực tế thay vì chỉ dựa vào những đặc điểm lý thuyết đã biết trước.

Ngoài giá trị học tập, đề tài còn có ý nghĩa thực tiễn đáng kể. Bài toán tìm đường là nền tảng của rất nhiều ứng dụng trong thực tế: hệ thống điều hướng GPS, robot tự hành trong kho hàng, nhân vật NPC trong trò chơi điện tử, hay thậm chí các thuật toán lập lịch và phân công nhiệm vụ trong hệ thống phân tán. Hiểu rõ cách BFS, DFS và A* hoạt động trên một mô hình đơn giản như mê cung là bước đầu tiên để có thể tiếp cận và giải quyết những bài toán phức tạp hơn trong tương lai.

Chính vì những lý do trên, nhóm quyết định lựa chọn đề tài **"Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A*"**. Đề tài không chỉ là cơ hội để nhóm củng cố kiến thức lý thuyết đã học mà còn là bước thực hành quan trọng trong việc chuyển từ thuật toán trên giấy thành một chương trình hoạt động thực sự, có thể quan sát, đo lường và đánh giá một cách khách quan.

1.3. Mục tiêu đề tài

Trong học phần Trí tuệ nhân tạo, bài toán tìm kiếm là một trong những nội dung nền tảng, giúp người học hiểu được cách máy tính có thể khảo sát một tập hợp các trạng thái khác nhau để tìm ra lời giải phù hợp. Từ cơ sở đó, đề tài **"Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A*"** được thực hiện với mục tiêu vận dụng các thuật toán tìm kiếm đã học vào một bài toán có tính thực quan, dễ quan sát nhưng vẫn thể hiện rõ những đặc trưng của quá trình tìm kiếm trong không gian trạng thái.

Mục tiêu trước hết của đề tài là tìm hiểu và củng cố cơ sở lý thuyết liên quan đến bài toán tìm kiếm trong Trí tuệ nhân tạo. Thông qua quá trình nghiên cứu, nhóm tập trung làm rõ các khái niệm như trạng thái, trạng thái ban đầu, trạng thái mục tiêu, không gian trạng thái, phép chuyển trạng thái và đường đi từ trạng thái ban đầu đến trạng thái đích. Trong bài toán mê cung, mỗi vị trí mà người chơi có thể đứng được xem như một trạng thái, còn quá trình di chuyển từ một ô sang một ô lân cận hợp lệ được xem như một phép chuyển trạng thái. Cách tiếp cận này giúp biến một trò chơi tìm đường đơn giản thành một bài toán tìm kiếm có cấu trúc rõ ràng và có thể xử lý bằng các thuật toán trong Trí tuệ nhân tạo.

Trên nền tảng đó, đề tài hướng tới việc nghiên cứu sâu hơn nguyên lý hoạt động của ba thuật toán tìm kiếm được lựa chọn là Breadth-First Search, Depth-First Search và A*. Đối với

BFS, mục tiêu là làm rõ cách thuật toán mở rộng các trạng thái theo từng mức khoảng cách tính từ điểm bắt đầu và sử dụng cấu trúc hàng đợi để quản lý các trạng thái chờ xử lý. Đối với DFS, đề tài tìm hiểu cách thuật toán ưu tiên khám phá sâu theo một nhánh trước khi quay lui để tiếp tục xét những nhánh khác. Trong khi đó, đối với A^* , trọng tâm là nghiên cứu cách thuật toán kết hợp chi phí thực tế đã đi và giá trị ước lượng đến đích thông qua hàm đánh giá, từ đó định hướng quá trình tìm kiếm về phía trạng thái mục tiêu. Qua việc nghiên cứu đồng thời ba thuật toán, đề tài mong muốn làm rõ sự khác nhau giữa tìm kiếm không sử dụng thông tin định hướng và tìm kiếm có sử dụng heuristic

Một mục tiêu quan trọng khác là xây dựng được mô hình bài toán mê cung phù hợp để có thể áp dụng thống nhất cả ba thuật toán. Mê cung trong đề tài được biểu diễn dưới dạng một lưới hai chiều gồm các ô có thể di chuyển và các ô bị chặn. Trên lưới này cần xác định rõ vị trí bắt đầu, vị trí đích, các chướng ngại vật và mối quan hệ giữa một vị trí với những vị trí lân cận. Việc thiết kế mô hình theo cách này không chỉ giúp đơn giản hóa quá trình cài đặt mà còn tạo điều kiện để chuyển mê cung thành một dạng đồ thị, trong đó các ô hợp lệ tương ứng với các đỉnh và khả năng di chuyển giữa hai ô lân cận tương ứng với các cạnh của đồ thị.

Thông qua mô hình đó, đề tài đặt ra mục tiêu xây dựng một chương trình mô phỏng có khả năng thực hiện đầy đủ quá trình tìm đường. Chương trình không chỉ dừng lại ở việc đưa ra kết quả cuối cùng là một đường đi từ điểm bắt đầu đến điểm đích mà còn phải thể hiện được quá trình thuật toán khám phá mê cung. Người sử dụng có thể lựa chọn thuật toán cần chạy, sau đó quan sát các ô được duyệt theo từng bước cho đến khi thuật toán tìm thấy đích hoặc xác định rằng không tồn tại đường đi. Việc mô phỏng từng bước có ý nghĩa quan trọng vì nó giúp thể hiện trực quan cách thức hoạt động của các thuật toán thay vì chỉ quan sát kết quả cuối cùng.

Bên cạnh quá trình tìm kiếm, chương trình còn cần thực hiện chức năng truy vết đường đi sau khi đã tìm được trạng thái đích. Trong quá trình duyệt, thông tin về trạng thái trước đó của mỗi vị trí cần được lưu lại để sau khi đạt tới đích, hệ thống có thể lần ngược từ điểm đích trở về điểm bắt đầu và khôi phục lại toàn bộ đường đi. Đường đi sau đó được hiển thị trên giao diện mô phỏng để người dùng có thể phân biệt rõ giữa những vị trí mà thuật toán đã duyệt và những vị trí thực sự nằm trên đường đi cuối cùng. Trường hợp không tồn tại đường đi từ

điểm bắt đầu đến điểm đích cũng cần được xử lý chính xác để bảo đảm tính đầy đủ của chương trình. Nội dung này cũng là một phần quan trọng trong mô hình tìm kiếm mà báo cáo hiện tại đã đặt ra.

Không chỉ hướng đến việc xây dựng một chương trình có thể hoạt động, đề tài còn đặt mục tiêu sử dụng chương trình như một công cụ để tiến hành thực nghiệm và so sánh các thuật toán. BFS, DFS và A* được chạy trên cùng một tập mê cung và trong những điều kiện đầu vào giống nhau nhằm hạn chế sự khác biệt do dữ liệu thử nghiệm gây ra. Các trường hợp thực nghiệm có thể bao gồm mê cung đơn giản, mê cung có nhiều nhánh rẽ, nhiều ngõ cụt, mê cung có kích thước lớn hoặc trường hợp không tồn tại đường đi. Việc thử nghiệm trên nhiều dạng mê cung giúp kết quả đánh giá phản ánh rõ hơn đặc điểm của từng chiến lược tìm kiếm.

Trong quá trình thực nghiệm, đề tài hướng tới việc thu thập một số thông tin cụ thể như độ dài của đường đi tìm được, số lượng trạng thái hoặc số ô đã được thuật toán duyệt, thời gian thực hiện và khả năng xác định chính xác trường hợp có hoặc không có đường đi. Những số liệu này được sử dụng làm cơ sở để quan sát sự khác biệt giữa ba thuật toán. Chẳng hạn, một thuật toán có thể tìm được đường đi với ít trạng thái được duyệt hơn, trong khi một thuật toán khác có thể khám phá nhiều vị trí hơn trước khi tìm thấy đích. Việc ghi nhận kết quả thực nghiệm giúp quá trình đánh giá không chỉ dựa trên lý thuyết mà còn có cơ sở từ kết quả chạy chương trình thực tế.

Đối với BFS, đề tài hướng đến việc kiểm chứng khả năng tìm được đường đi ngắn nhất trong trường hợp mỗi bước di chuyển có cùng chi phí. Với DFS, chương trình giúp quan sát ảnh hưởng của thứ tự lựa chọn nhánh đến quá trình tìm kiếm, đồng thời cho thấy thuật toán có thể đi sâu vào một nhánh trước khi quay lại các nhánh khác. Đối với A*, mục tiêu là làm rõ vai trò của heuristic trong việc định hướng tìm kiếm về phía đích, từ đó quan sát xem việc sử dụng thông tin ước lượng ảnh hưởng như thế nào đến số lượng trạng thái cần khám phá. Các kết quả này là cơ sở để hiểu rõ hơn bản chất của từng thuật toán trong cùng một bài toán.

Ngoài những mục tiêu về nội dung chuyên môn, đề tài còn hướng đến việc rèn luyện khả năng phân tích và giải quyết một bài toán theo quy trình có hệ thống. Từ một yêu cầu ban đầu là tìm đường trong mê cung, nhóm cần xác định cách biểu diễn dữ liệu, lựa chọn cấu trúc dữ liệu phù hợp, xây dựng thuật toán, chuyển mã giả thành chương trình, kiểm tra các trường

hợp đặc biệt và đánh giá kết quả sau khi chạy thử. Quá trình này giúp hình thành một chuỗi công việc tương đối hoàn chỉnh gồm phân tích bài toán, xây dựng mô hình, thiết kế thuật toán, cài đặt, kiểm thử, thực nghiệm, so sánh và đánh giá.

Thông qua việc thực hiện đề tài, nhóm mong muốn không chỉ hiểu được BFS, DFS và A* ở mức lý thuyết mà còn có thể giải thích được cách các thuật toán hoạt động trong một chương trình cụ thể. Việc trực tiếp quan sát thứ tự duyệt các trạng thái, quá trình xử lý các nhánh rẽ và ngõ cụt, cách truy vết đường đi và sự khác biệt về kết quả giữa các thuật toán sẽ giúp kiến thức về tìm kiếm trong Trí tuệ nhân tạo trở nên trực quan và dễ tiếp cận hơn. Qua đó, đề tài tạo ra sự liên kết giữa kiến thức lý thuyết, quá trình cài đặt và kết quả thực nghiệm.

Cuối cùng, mục tiêu tổng quát của đề tài là xây dựng được một chương trình mô phỏng tìm đường trong mê cung có tính trực quan, hoạt động đúng với nguyên lý của BFS, DFS và A*, đồng thời có khả năng cung cấp các thông tin phục vụ cho việc quan sát và đánh giá quá trình tìm kiếm. Kết quả đạt được từ đề tài sẽ là cơ sở để nhóm hiểu sâu hơn về các phương pháp tìm kiếm trong Trí tuệ nhân tạo và tạo nền tảng cho việc tiếp cận những bài toán tìm đường, lập kế hoạch và tìm kiếm có mức độ phức tạp cao hơn trong tương lai.

1.4. Phạm vi nghiên cứu

1.4.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu chính của đề tài là **bài toán tìm đường trong mê cung** và các phương pháp tìm kiếm được sử dụng để giải quyết bài toán này trong lĩnh vực Trí tuệ nhân tạo. Trong đó, đề tài tập trung vào ba thuật toán gồm **Breadth-First Search (BFS)**, **Depth-First Search (DFS)** và **A***. Đây là ba thuật toán có cách tiếp cận khác nhau đối với quá trình tìm kiếm, qua đó giúp làm rõ sự khác biệt giữa tìm kiếm theo chiều rộng, tìm kiếm theo chiều sâu và tìm kiếm có sử dụng thông tin heuristic.

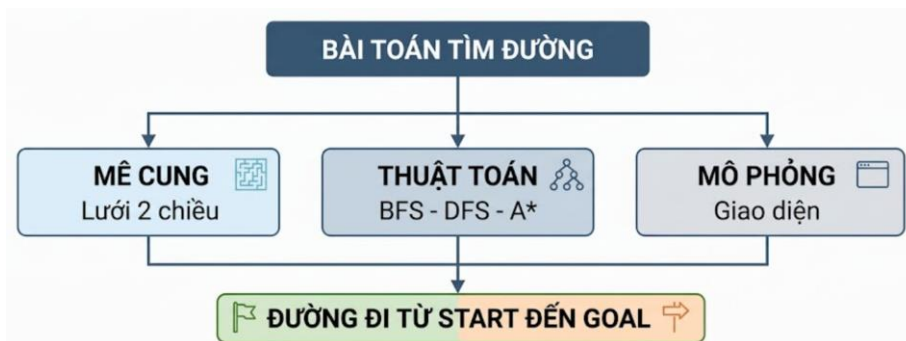
Bên cạnh việc nghiên cứu nguyên lý của các thuật toán, đề tài còn xem xét cách **biểu diễn mê cung dưới dạng lưới hai chiều**, trong đó mỗi ô có thể được coi là một trạng thái của bài toán. Các ô có thể đi qua tạo thành không gian tìm kiếm, còn các ô bị chặn được xem là những trạng thái không hợp lệ. Từ một vị trí bắt đầu xác định,

thuật toán phải lần lượt lựa chọn các trạng thái kế tiếp phù hợp để tìm được đường đi đến vị trí đích.

Một nội dung nghiên cứu quan trọng khác là cách tổ chức quá trình tìm kiếm trong chương trình. Đề tài quan tâm đến việc xác định các trạng thái lân cận, kiểm tra trạng thái hợp lệ, lưu trữ những vị trí đã được khám phá, xác định điều kiện kết thúc và truy vết lại đường đi sau khi tìm thấy đích. Đây là những thành phần cần thiết để chuyển thuật toán từ dạng lý thuyết sang một chương trình mô phỏng có thể hoạt động trong thực tế.

Đối với từng thuật toán, đề tài tập trung nghiên cứu những đặc trưng riêng. BFS được xem xét dưới góc độ tìm kiếm theo chiều rộng và sử dụng hàng đợi để mở rộng các trạng thái theo từng mức. DFS tập trung vào cách tìm kiếm theo chiều sâu, trong đó thuật toán ưu tiên đi sâu vào một nhánh trước khi quay lui. Đối với A*, đề tài nghiên cứu việc kết hợp chi phí thực tế đã đi với giá trị ước lượng đến đích thông qua hàm đánh giá. Đồng thời, **khoảng cách Manhattan** được lựa chọn làm heuristic phù hợp với mê cung chỉ cho phép di chuyển theo bốn hướng.

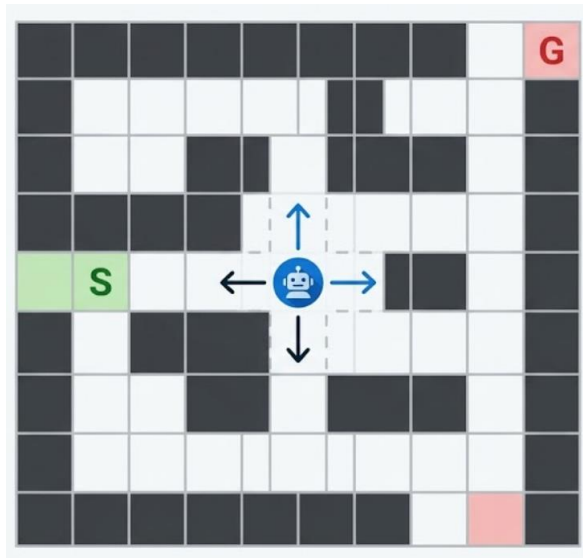
Ngoài phần thuật toán, chương trình mô phỏng cũng là một đối tượng nghiên cứu của đề tài. Thông qua chương trình, quá trình tìm kiếm của BFS, DFS và A* được thực hiện trên cùng một mô hình mê cung, giúp kiểm tra tính đúng đắn của từng thuật toán và tạo điều kiện quan sát trực quan sự khác biệt trong cách các thuật toán khám phá không gian tìm kiếm.



Hình 1.1: Các đối tượng nghiên cứu của đề tài

1.4.2. Phạm vi nghiên cứu

Trong phạm vi của đề tài, bài toán được giới hạn ở **mê cung dạng lưới hai chiều**, gồm các ô có thể di chuyển và các ô bị chặn. Mỗi mê cung có một điểm bắt đầu và một điểm đích xác định. Người chơi hoặc tác nhân chỉ được phép di chuyển giữa các ô liền kề theo bốn hướng cơ bản là **lên, xuống, trái và phải**. Mỗi bước di chuyển được giả định có cùng chi phí, nhờ đó việc áp dụng và so sánh các thuật toán được thực hiện trên cùng một điều kiện.



Hình 1.2: Phạm vi di chuyển của tác nhân trong mê cung hai chiều

Về thuật toán, đề tài chỉ tập trung vào ba phương pháp BFS, DFS và A*. BFS được sử dụng để nghiên cứu đặc điểm của tìm kiếm không có thông tin định hướng và khả năng tìm đường ngắn nhất trong trường hợp chi phí các bước di chuyển là như nhau. DFS được sử dụng để quan sát chiến lược đi sâu vào từng nhánh và ảnh hưởng của thứ tự duyệt đến kết quả tìm kiếm. Trong khi đó, A* được sử dụng để nghiên cứu phương pháp tìm kiếm có thông tin, trong đó heuristic đóng vai trò định hướng thuật toán về phía điểm đích.

Về mặt thực nghiệm, các thuật toán được đánh giá trên cùng một tập mê cung nhằm tạo điều kiện so sánh tương đối thống nhất. Những tiêu chí được quan tâm chủ yếu gồm **khả năng tìm thấy đường đi, độ dài đường đi, số lượng trạng thái đã**

duyet và thời gian thực hiện. Các kết quả thu được được sử dụng để quan sát sự khác biệt giữa ba phương pháp trong những điều kiện mê cung khác nhau.

Để bảo đảm đề tài có phạm vi phù hợp với thời gian và nội dung của bài tập lớn, một số bài toán mở rộng chưa được xem xét sâu. Cụ thể, đề tài không nghiên cứu mê cung ba chiều, môi trường thay đổi theo thời gian thực, nhiều tác nhân cùng tìm đường, nhiều điểm bắt đầu hoặc nhiều điểm đích, cũng như trường hợp mỗi ô có chi phí di chuyển khác nhau. Những nội dung này có thể được xem như hướng mở rộng và phát triển trong tương lai.

1.5. Phương pháp thực hiện

Để thực hiện đề tài “**Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A***”, nhóm tiến hành theo hướng kết hợp giữa nghiên cứu lý thuyết, phân tích bài toán, xây dựng mô hình, cài đặt chương trình và thực nghiệm đánh giá. Cách tiếp cận này giúp đề tài không chỉ dừng lại ở việc trình bày nguyên lý hoạt động của các thuật toán mà còn kiểm chứng được khả năng áp dụng của chúng trên một bài toán tìm đường cụ thể.

Trước tiên, nhóm tiến hành tìm hiểu các tài liệu liên quan đến bài toán tìm kiếm trong Trí tuệ nhân tạo, không gian trạng thái, cách biểu diễn bài toán dưới dạng đồ thị và nguyên lý hoạt động của các thuật toán BFS, DFS và A*. Đối với thuật toán A*, nhóm nghiên cứu thêm vai trò của hàm đánh giá và hàm heuristic trong việc định hướng quá trình tìm kiếm. Việc nghiên cứu lý thuyết được thực hiện trước nhằm tạo cơ sở cho quá trình xây dựng mô hình và lựa chọn phương pháp cài đặt phù hợp.

Sau khi xác định được cơ sở lý thuyết, bài toán tìm đường trong mê cung được phân tích và mô hình hóa dưới dạng một lưới hai chiều. Mỗi ô trong lưới được xem như một trạng thái, trong đó các ô có thể đi qua là những trạng thái hợp lệ, còn các ô tường là những vị trí không thể di chuyển. Từ điểm bắt đầu, chương trình kiểm tra các ô lân cận theo bốn hướng lên, xuống, trái và phải để xác định những trạng thái tiếp theo có thể được mở rộng. Cách biểu diễn này giúp chuyển bài toán mê cung thành một bài toán tìm kiếm trên đồ thị, phù hợp với cả ba thuật toán được lựa chọn.

Trên cơ sở mô hình đã xây dựng, nhóm tiến hành thiết kế và cài đặt lần lượt các thuật toán BFS, DFS và A*. Với BFS, quá trình tìm kiếm được thực hiện theo chiều rộng và mở rộng các trạng thái theo từng mức. DFS ưu tiên khám phá sâu theo một nhánh trước khi quay lại những nhánh chưa được xét. Trong khi đó, A* sử dụng hàm đánh giá kết hợp giữa chi phí đã đi và khoảng cách ước lượng đến đích để lựa chọn trạng thái có mức độ ưu tiên cao hơn. Trong quá trình cài đặt, chương trình đồng thời lưu lại các trạng thái đã được khám phá và thông tin về trạng thái trước đó để phục vụ cho việc truy vết đường đi sau khi tìm thấy điểm đích.

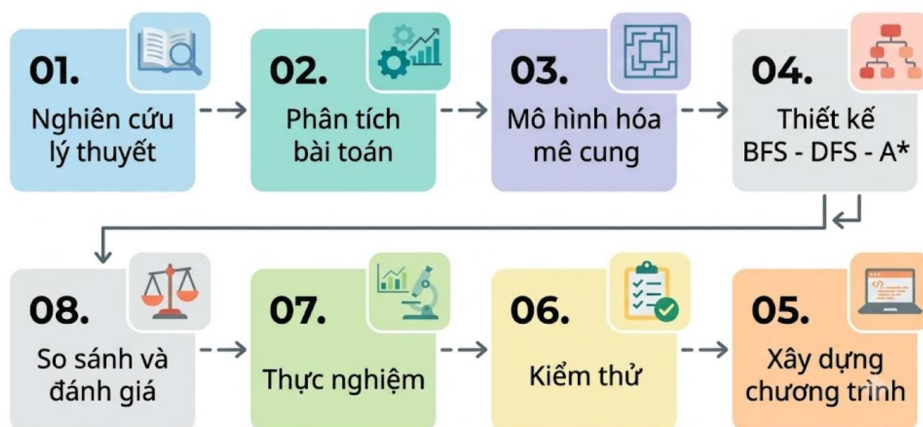
Sau khi các thuật toán được cài đặt, nhóm tiến hành xây dựng phần mô phỏng nhằm trực quan hóa quá trình tìm kiếm. Giao diện chương trình cần thể hiện được cấu trúc mê cung, vị trí bắt đầu, vị trí đích, các ô bị chặn, các trạng thái đã được thuật toán duyệt và đường đi cuối cùng. Việc mô phỏng từng bước giúp người sử dụng dễ dàng quan sát sự khác nhau trong cách BFS, DFS và A* khám phá không gian tìm kiếm.

Tiếp theo, chương trình được kiểm thử với nhiều dạng mê cung khác nhau, từ các trường hợp đơn giản đến những trường hợp có nhiều nhánh rẽ, nhiều ngõ cụt hoặc không tồn tại đường đi. Quá trình kiểm thử được sử dụng để phát hiện các lỗi liên quan đến việc xác định ô lân cận, đánh dấu trạng thái đã duyệt, xử lý vật cản và truy vết đường đi. Qua đó, nhóm có thể kiểm tra tính đúng đắn và độ ổn định của chương trình trước khi tiến hành thực nghiệm chính thức.

Sau khi chương trình hoạt động ổn định, nhóm tiến hành thực nghiệm ba thuật toán trên cùng một tập dữ liệu mê cung nhằm bảo đảm điều kiện so sánh tương đối thống nhất. Trong quá trình thực nghiệm, một số thông tin được ghi nhận như độ dài đường đi, số lượng trạng thái đã duyệt và thời gian thực hiện. Các số liệu này được sử dụng để quan sát sự khác biệt giữa BFS, DFS và A*, đồng thời kiểm chứng những đặc điểm đã được nghiên cứu về mặt lý thuyết.

Cuối cùng, nhóm tổng hợp và phân tích các kết quả thu được từ quá trình thực nghiệm. Kết quả của ba thuật toán được đặt trong cùng điều kiện thử nghiệm để làm rõ sự khác nhau về chiến lược tìm kiếm, số lượng trạng thái cần khám phá, độ dài đường đi và thời gian xử lý. Trên cơ sở đó, nhóm đưa ra các nhận xét về ưu điểm, hạn chế và đặc điểm của từng thuật toán

khi áp dụng vào bài toán tìm đường trong mê cung. Toàn bộ quá trình thực hiện được triển khai theo dưới hình:



Hình 1.3: Quy trình thực hiện đề tài

1.6. Ý nghĩa của đề tài

Đề tài “**Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A***” có ý nghĩa cả về mặt học tập lẫn thực hành. Thông qua một bài toán trực quan như tìm đường trong mê cung, các nội dung lý thuyết về thuật toán tìm kiếm trong Trí tuệ nhân tạo được thể hiện rõ ràng hơn, đồng thời tạo điều kiện để quan sát sự khác nhau giữa các phương pháp tìm kiếm trên cùng một môi trường.

Trước hết, về mặt học tập, đề tài giúp nhóm củng cố và vận dụng những kiến thức đã được học trong học phần Trí tuệ nhân tạo, đặc biệt là các nội dung liên quan đến không gian trạng thái, tìm kiếm trên đồ thị, cấu trúc dữ liệu và các phương pháp tìm kiếm có thông tin cũng như không có thông tin. Thay vì chỉ tiếp cận BFS, DFS và A* thông qua công thức, mã giả hoặc các ví dụ đơn giản, việc triển khai trực tiếp các thuật toán trên mô hình mê cung giúp người học hiểu rõ hơn cách lựa chọn trạng thái, cách mở rộng không gian tìm kiếm và cách xác định đường đi từ trạng thái bắt đầu đến trạng thái mục tiêu.

Đề tài còn giúp làm rõ sự khác biệt trong chiến lược hoạt động của ba thuật toán. Khi cùng giải quyết một mê cung, BFS, DFS và A* có thể khám phá các vị trí theo những thứ tự khác nhau, từ đó tạo ra sự khác biệt về số lượng trạng thái đã duyệt, thời gian xử lý, độ dài đường đi và mức độ hiệu quả trong quá trình tìm kiếm. Việc quan sát trực tiếp các kết quả này

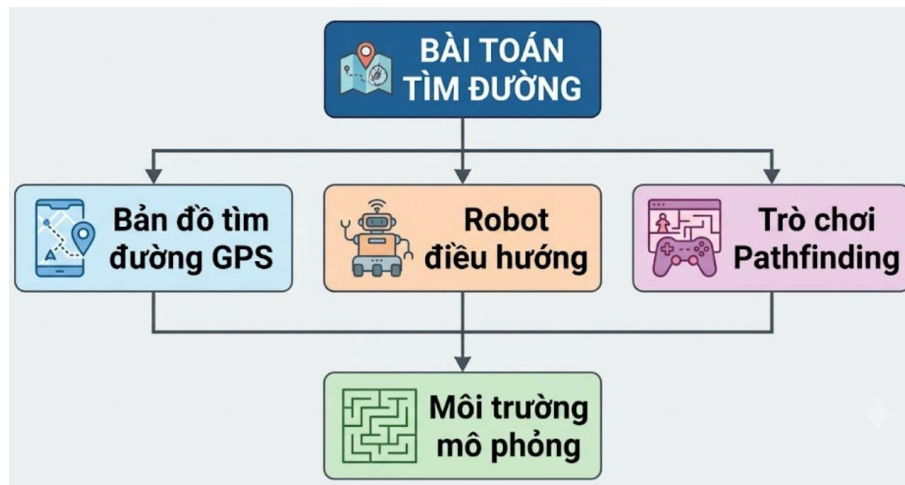
giúp người học hiểu rằng hiệu quả của một thuật toán không chỉ được đánh giá dựa trên việc có tìm được đường đi hay không, mà còn phụ thuộc vào cách thuật toán tổ chức và định hướng quá trình khám phá không gian trạng thái.

Về mặt thực hành, quá trình xây dựng chương trình mô phỏng giúp nhóm rèn luyện khả năng chuyển một bài toán thực tế thành mô hình có thể xử lý bằng máy tính. Để chương trình hoạt động đúng, nhóm cần giải quyết nhiều vấn đề như biểu diễn mê cung, xác định các trạng thái hợp lệ, xử lý các ô lân cận, kiểm soát những trạng thái đã được duyệt, lưu thông tin phục vụ cho việc truy vết và hiển thị kết quả. Đây đều là những kỹ năng quan trọng trong quá trình xây dựng và phát triển các chương trình sử dụng thuật toán.

Một ý nghĩa khác của đề tài là tạo điều kiện để đánh giá các thuật toán bằng kết quả thực nghiệm thay vì chỉ dựa vào đặc điểm lý thuyết. Khi BFS, DFS và A* được triển khai trên cùng một môi trường mê cung, nhóm có thể sử dụng các trường hợp thử nghiệm với cấu trúc và độ phức tạp khác nhau để quan sát cách mỗi thuật toán hoạt động. Những dữ liệu thu được về số lượng trạng thái đã duyệt, thời gian xử lý và độ dài đường đi giúp việc so sánh trở nên cụ thể hơn, đồng thời hỗ trợ kiểm chứng những nhận định đã được trình bày trong phần cơ sở lý thuyết.

Về mặt ứng dụng, mặc dù mê cung được xây dựng trong đề tài còn ở mức tương đối đơn giản, bài toán tìm đường có liên hệ với nhiều vấn đề thực tế trong lĩnh vực công nghệ. Các phương pháp tìm kiếm có thể được sử dụng làm nền tảng cho những bài toán như tìm đường trên bản đồ, điều hướng nhân vật trong trò chơi, hỗ trợ lập kế hoạch di chuyển cho robot hoặc xử lý các bài toán tìm kiếm trong môi trường mô phỏng. Vì vậy, việc nghiên cứu BFS, DFS và A* thông qua bài toán mê cung không chỉ phục vụ yêu cầu của học phần mà còn tạo nền tảng để tiếp cận những bài toán phức tạp hơn trong tương lai.

Bên cạnh đó, chương trình mô phỏng còn có giá trị về mặt trực quan trong quá trình học tập và trình bày thuật toán. Thay vì chỉ quan sát mã nguồn hoặc kết quả cuối cùng, người sử dụng có thể theo dõi quá trình từng thuật toán lần lượt khám phá các vị trí trong mê cung. Nhờ đó, những khái niệm như tìm kiếm theo chiều rộng, tìm kiếm theo chiều sâu và tìm kiếm có heuristic trở nên dễ hình dung hơn, đặc biệt đối với người mới tiếp cận các thuật toán tìm kiếm trong Trí tuệ nhân tạo.



Hình 1.4: Một số ứng dụng của bài toán tìm đường trong thực tế

Từ những ý nghĩa trên, đề tài không chỉ hướng tới việc xây dựng một chương trình có khả năng tìm đường từ điểm bắt đầu đến điểm đích mà còn tạo ra sự liên kết giữa **lý thuyết thuật toán, quá trình cài đặt và kết quả thực nghiệm**. Thông qua quá trình thực hiện, nhóm có điều kiện hiểu sâu hơn về đặc điểm của BFS, DFS và A*, đồng thời hình thành cách tiếp cận có hệ thống khi giải quyết một bài toán tìm kiếm trong Trí tuệ nhân tạo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về Trí tuệ nhân tạo

2.1.1. *Khái niệm Trí tuệ nhân tạo*

Trí tuệ nhân tạo, hay Artificial Intelligence (AI), là một lĩnh vực của khoa học máy tính tập trung nghiên cứu và xây dựng các hệ thống có khả năng thực hiện những nhiệm vụ thường đòi hỏi trí thông minh của con người. Những nhiệm vụ này có thể bao gồm nhận biết thông tin, suy luận, giải quyết vấn đề, lập kế hoạch, học hỏi từ dữ liệu, xử lý ngôn ngữ, nhận dạng hình ảnh và đưa ra quyết định trong những tình huống khác nhau.

Về bản chất, Trí tuệ nhân tạo hướng đến việc làm cho máy tính có khả năng xử lý thông tin một cách linh hoạt hơn so với các chương trình truyền thống. Trong một chương trình thông thường, máy tính chủ yếu thực hiện tuần tự các câu lệnh đã được người lập trình xác định trước. Trong khi đó, một hệ thống AI có thể được thiết kế để phân tích trạng thái hiện tại, xem xét nhiều khả năng khác nhau và lựa chọn một phương án phù hợp với mục tiêu cần đạt được.

Một trong những đặc điểm quan trọng của Trí tuệ nhân tạo là khả năng biểu diễn và xử lý một bài toán dưới dạng các trạng thái. Thay vì chỉ xem xét dữ liệu ở dạng đầu vào và đầu ra, nhiều bài toán AI được mô hình hóa thành một tập hợp các trạng thái cùng với những hành động có thể thực hiện để chuyển từ trạng thái này sang trạng thái khác. Từ trạng thái ban đầu, hệ thống cần xác định một chuỗi hành động phù hợp để đạt được trạng thái mục tiêu. Đây chính là cơ sở của các bài toán tìm kiếm trong Trí tuệ nhân tạo.

Các phương pháp được sử dụng trong AI rất đa dạng. Một số bài toán được giải quyết thông qua các thuật toán tìm kiếm như Breadth-First Search, Depth-First Search hoặc A*. Một số hệ thống khác sử dụng logic để biểu diễn tri thức và thực hiện suy luận. Trong những bài toán có lượng dữ liệu lớn, các phương pháp học máy có thể được áp dụng để giúp hệ thống rút ra quy luật từ dữ liệu thay vì phải mô tả toàn bộ quy tắc bằng chương trình.

Trí tuệ nhân tạo hiện được ứng dụng trong nhiều lĩnh vực khác nhau của đời sống và công nghệ. Có thể kể đến các hệ thống nhận dạng khuôn mặt, trợ lý ảo, xử lý ngôn ngữ tự nhiên, hệ thống khuyến nghị, điều hướng robot, tìm đường trên bản đồ, phát hiện bất thường, trò chơi điện tử và các hệ thống hỗ trợ ra quyết định. Mặc dù mỗi ứng dụng có cách triển khai khác nhau, mục tiêu chung vẫn là giúp máy tính có khả năng xử lý những tình huống phức tạp và đưa ra kết quả phù hợp với yêu cầu đặt ra.

Trong phạm vi đề tài Trí tuệ nhân tạo được tiếp cận chủ yếu thông qua bài toán tìm kiếm. Mê cung được xem như một không gian gồm nhiều trạng thái khác nhau, trong đó mỗi vị trí có thể di chuyển tương ứng với một trạng thái. Điểm bắt đầu được xem là trạng thái ban đầu, điểm đích là trạng thái mục tiêu, còn việc di chuyển giữa các ô lân cận được xem là các phép chuyển trạng thái.

Từ cách mô hình hóa đó, chương trình cần xác định một chuỗi trạng thái hợp lệ để đưa tác nhân từ điểm bắt đầu đến điểm đích. Các thuật toán BFS, DFS và A* được sử dụng để thực hiện quá trình tìm kiếm theo những chiến lược khác nhau. BFS mở rộng không gian tìm kiếm theo từng mức, DFS ưu tiên khám phá theo chiều sâu, trong khi A* sử dụng thêm thông tin heuristic để định hướng quá trình tìm kiếm về phía trạng thái mục tiêu.



Hình 1.5: Một số lĩnh vực và phương pháp cơ bản trong Trí tuệ nhân tạo

2.1.2. *Đặc điểm và mục tiêu của Trí tuệ nhân tạo*

Trí tuệ nhân tạo được xây dựng với mục tiêu giúp máy tính có khả năng thực hiện những công việc đòi hỏi quá trình nhận biết, suy luận, lựa chọn và giải quyết vấn đề. Thay vì chỉ thực hiện các câu lệnh theo một trình tự cố định, hệ thống AI có thể được thiết kế để tiếp nhận thông tin từ môi trường, phân tích trạng thái hiện tại và lựa chọn phương án xử lý phù hợp với mục tiêu đặt ra.

Một đặc điểm quan trọng của Trí tuệ nhân tạo là khả năng **biểu diễn tri thức và trạng thái của bài toán**. Để máy tính có thể xử lý một vấn đề, các thông tin liên quan cần được mô hình hóa dưới dạng mà chương trình có thể hiểu và thao tác. Trong các bài toán tìm kiếm, thông tin thường được biểu diễn dưới dạng không gian trạng thái, trong đó mỗi trạng thái thể hiện một tình huống cụ thể và các hành động cho phép hệ thống chuyển từ trạng thái này sang trạng thái khác.

Bên cạnh đó, AI có khả năng **tìm kiếm và lựa chọn lời giải trong nhiều khả năng khác nhau**. Đối với những bài toán có số lượng phương án lớn, việc kiểm tra toàn bộ các khả năng một cách ngẫu nhiên sẽ làm tăng thời gian xử lý. Vì vậy, các thuật toán tìm kiếm được sử dụng để tổ chức quá trình khám phá không gian trạng thái theo những chiến lược nhất định. BFS, DFS và A* là những ví dụ điển hình cho các chiến lược tìm kiếm khác nhau.

Một đặc điểm khác của Trí tuệ nhân tạo là khả năng **suy luận và hỗ trợ ra quyết định**. Dựa trên các thông tin đang có, hệ thống có thể đánh giá các lựa chọn và xác định hành động tiếp theo. Trong một số bài toán, quá trình này được thực hiện bằng các quy tắc logic; trong những bài toán khác, hệ thống có thể sử dụng hàm đánh giá, xác suất hoặc các mô hình học từ dữ liệu.

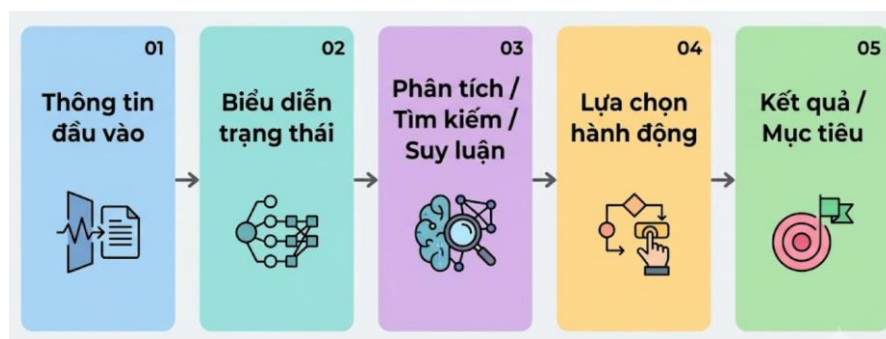
Trí tuệ nhân tạo cũng có thể thể hiện khả năng **thích nghi hoặc học hỏi** tùy theo phương pháp được sử dụng. Trong các hệ thống học máy, chương trình có thể cải thiện kết quả dựa trên dữ liệu và kinh nghiệm trước đó. Tuy nhiên, không phải mọi hệ thống AI đều cần sử dụng học máy. Nhiều bài toán, đặc biệt là các bài toán tìm kiếm và lập

kế hoạch, có thể được giải quyết bằng các thuật toán đã được xác định rõ mà không cần quá trình huấn luyện dữ liệu.

Về mục tiêu, Trí tuệ nhân tạo hướng đến việc xây dựng các hệ thống có khả năng giải quyết vấn đề một cách hiệu quả, giảm khối lượng công việc thủ công và hỗ trợ con người trong những nhiệm vụ có độ phức tạp cao. Tùy vào từng lĩnh vực, mục tiêu cụ thể có thể là nhận dạng đối tượng, dự đoán kết quả, tìm kiếm thông tin, điều khiển thiết bị, lập kế hoạch hoặc tìm đường tối ưu trong một môi trường nhất định.

Trong phạm vi đề tài, mục tiêu của việc áp dụng Trí tuệ nhân tạo là xây dựng một hệ thống có khả năng tự động tìm đường từ điểm bắt đầu đến điểm đích trong mê cung. Hệ thống cần khảo sát nhiều trạng thái khác nhau và lựa chọn thứ tự tìm kiếm theo nguyên lý của BFS, DFS hoặc A*. Qua đó, chương trình có thể minh họa được cách một hệ thống AI giải quyết bài toán bằng cách tìm kiếm trong không gian trạng thái.

Việc sử dụng ba thuật toán trong cùng một bài toán còn giúp làm rõ một đặc điểm quan trọng của Trí tuệ nhân tạo: **cùng một mục tiêu có thể được giải quyết bằng nhiều chiến lược khác nhau**. BFS ưu tiên mở rộng các trạng thái theo từng mức, DFS ưu tiên đi sâu theo một nhánh, còn A* sử dụng thêm heuristic để định hướng tìm kiếm. Sự khác nhau này ảnh hưởng đến số lượng trạng thái cần duyệt, đường đi được tìm thấy và thời gian thực hiện.



Hình 2.6: Quy trình cơ bản của một hệ thống Trí tuệ nhân tạo

2.1.3. Các lĩnh vực ứng dụng của Trí tuệ nhân tạo

Trí tuệ nhân tạo hiện được ứng dụng trong nhiều lĩnh vực khác nhau của đời sống, khoa học và công nghệ. Tùy vào mục đích của từng bài toán, AI có thể được sử dụng để nhận biết thông tin, dự đoán, hỗ trợ ra quyết định, tìm kiếm phương án, điều khiển thiết bị hoặc tự động hóa các công việc phức tạp.

Một trong những lĩnh vực phổ biến là **xử lý ngôn ngữ tự nhiên**. Các hệ thống trong lĩnh vực này được xây dựng để xử lý và hiểu ngôn ngữ của con người, chẳng hạn như nhận dạng văn bản, phân tích nội dung, dịch ngôn ngữ, trả lời câu hỏi hoặc hỗ trợ giao tiếp giữa người dùng và máy tính. Đây là một trong những hướng ứng dụng quan trọng của AI trong các hệ thống phần mềm hiện đại.

Bên cạnh đó, **thị giác máy tính** cũng là một lĩnh vực ứng dụng nổi bật. Mục tiêu của lĩnh vực này là giúp máy tính có khả năng phân tích hình ảnh hoặc video để nhận biết đối tượng, đặc điểm hoặc sự kiện. Một số ứng dụng có thể kể đến như nhận dạng khuôn mặt, phát hiện vật thể, kiểm tra sản phẩm, xử lý ảnh y tế hoặc giám sát hình ảnh.

Trong lĩnh vực **robot và hệ thống tự động**, AI được sử dụng để hỗ trợ robot nhận biết môi trường, lập kế hoạch và lựa chọn hành động phù hợp. Để robot có thể di chuyển từ một vị trí đến vị trí khác, hệ thống thường cần giải quyết bài toán tìm đường, tránh vật cản và xác định lộ trình phù hợp. Đây là nội dung có mối liên hệ trực tiếp với đề tài tìm đường trong mê cung.

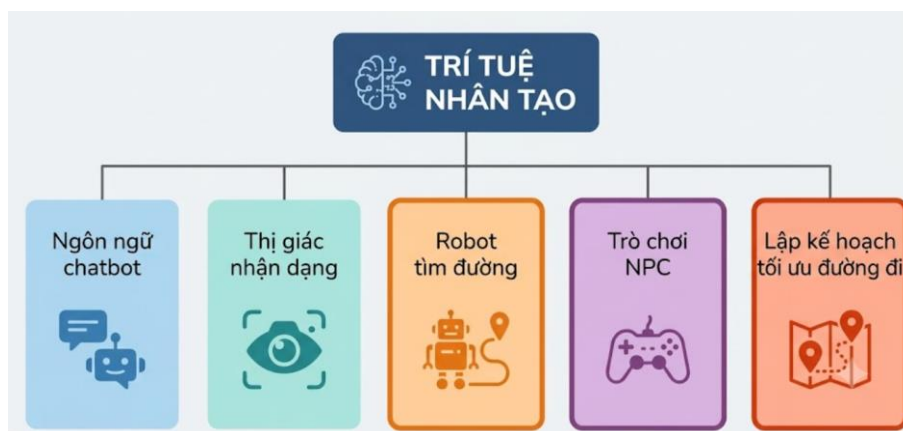
AI cũng được ứng dụng rộng rãi trong **hệ thống trò chơi**. Trong các trò chơi điện tử, các thuật toán AI có thể được sử dụng để điều khiển nhân vật, lựa chọn hành động, tìm kiếm đường đi, xây dựng chiến lược hoặc mô phỏng hành vi của đối thủ. Đặc biệt, bài toán tìm đường là một thành phần thường gặp khi nhân vật cần di chuyển từ vị trí hiện tại đến một vị trí mục tiêu trong môi trường có chướng ngại vật.

Một lĩnh vực khác là **hệ thống hỗ trợ ra quyết định và lập kế hoạch**. Trong những bài toán này, hệ thống phải đánh giá nhiều trạng thái hoặc phương án khác nhau trước khi lựa chọn hành động phù hợp. Các kỹ thuật tìm kiếm và tối ưu có thể được áp dụng để xác định một chuỗi hành động dẫn tới mục tiêu với chi phí phù hợp.

Ngoài ra, AI còn được sử dụng trong **hệ thống gợi ý, phân tích dữ liệu, dự đoán, nhận dạng mẫu và tự động hóa**. Trong các trường hợp này, hệ thống có thể khai thác dữ liệu để phát hiện mối quan hệ giữa các yếu tố, sau đó sử dụng kết quả phân tích để hỗ trợ người dùng hoặc tự động đưa ra một số quyết định.

Đối với đề tài “**Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A***”, lĩnh vực ứng dụng có liên quan trực tiếp nhất là **tìm kiếm, lập kế hoạch đường đi, robot và trò chơi**. Việc xây dựng một môi trường mê cung giúp mô phỏng tương đối rõ quá trình một tác nhân phải lựa chọn các trạng thái kế tiếp để di chuyển từ điểm bắt đầu đến điểm đích.

Mặc dù mô hình trong đề tài còn đơn giản, nguyên lý của bài toán có thể được mở rộng sang nhiều tình huống thực tế khác. Ví dụ, trong hệ thống bản đồ, thuật toán cần xác định tuyến đường giữa hai vị trí; trong robot, hệ thống cần tìm đường tránh các chướng ngại vật; còn trong trò chơi, nhân vật cần di chuyển trong một môi trường có các vùng không thể đi qua. Chính vì vậy, bài toán tìm đường trong mê cung là một ví dụ phù hợp để minh họa cách các thuật toán tìm kiếm của Trí tuệ nhân tạo được áp dụng vào thực tế.



Hình 2.7: Một số lĩnh vực ứng dụng của Trí tuệ nhân tạo

2.2. Tổng quan về Học máy

2.2.1. Khái niệm Học máy

Học máy, hay **Machine Learning (ML)**, là một nhánh của Trí tuệ nhân tạo tập trung vào việc xây dựng các phương pháp giúp máy tính có khả năng khai thác dữ liệu để nhận biết quy luật, đưa ra dự đoán hoặc cải thiện kết quả thực hiện một nhiệm vụ mà không cần phải mô tả trước toàn bộ các quy tắc xử lý cho từng trường hợp cụ thể.

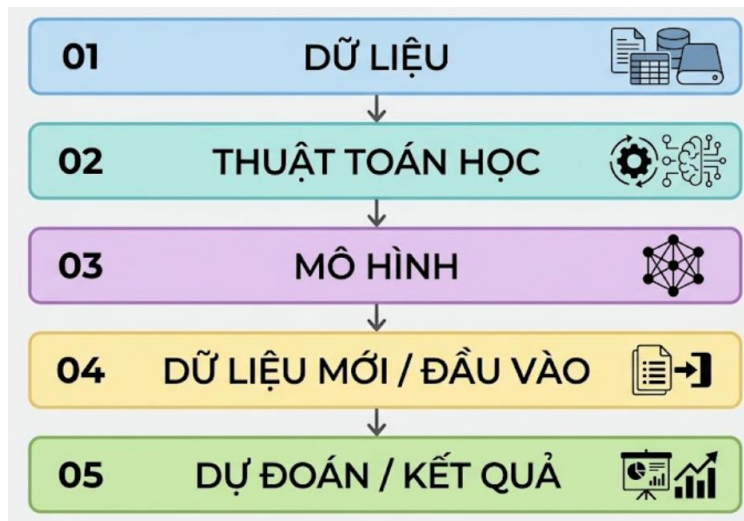
Trong cách lập trình truyền thống, người lập trình thường xác định rõ dữ liệu đầu vào, các quy tắc xử lý và cách tạo ra kết quả đầu ra. Đối với Học máy, hệ thống được cung cấp dữ liệu và sử dụng một thuật toán học để xây dựng mô hình từ những dữ liệu đó. Sau quá trình học, mô hình có thể được sử dụng để xử lý các dữ liệu mới có đặc điểm tương tự.

Có thể mô tả quá trình Học máy theo cách đơn giản như sau:

Dữ liệu → Thuật toán học → Mô hình → Dự đoán hoặc kết quả

Ví dụ, nếu muốn xây dựng một hệ thống nhận biết thư điện tử rác, hệ thống có thể được cung cấp nhiều mẫu thư đã được xác định trước là thư rác hoặc thư bình thường. Thuật toán Học máy phân tích các đặc điểm của những mẫu dữ liệu này để xây dựng một mô hình. Khi có một thư mới, mô hình có thể dựa trên những đặc điểm đã học để dự đoán thư đó thuộc nhóm nào.

Một đặc điểm quan trọng của Học máy là khả năng **tìm ra các mẫu hoặc mối quan hệ từ dữ liệu**. Khi lượng dữ liệu tăng lên và các mối quan hệ trở nên phức tạp, việc xây dựng thủ công toàn bộ các quy tắc xử lý có thể gặp nhiều khó khăn. Trong những trường hợp như vậy, Học máy cung cấp một phương pháp để hệ thống xây dựng mô hình dựa trên dữ liệu thay vì chỉ phụ thuộc vào các quy tắc được viết sẵn.



Hình 2.8: Quy trình cơ bản của Học máy

Tùy theo cách dữ liệu được cung cấp và mục tiêu của bài toán, Học máy thường được chia thành một số nhóm cơ bản như **học có giám sát**, **học không giám sát** và **học tăng cường**. Học có giám sát sử dụng dữ liệu đã có nhãn để xây dựng mô hình dự đoán; học không giám sát tìm kiếm cấu trúc hoặc nhóm trong dữ liệu chưa được gán nhãn; còn học tăng cường tập trung vào việc một tác nhân học cách lựa chọn hành động thông qua phần thưởng hoặc phản hồi từ môi trường.

Mặc dù Học máy là một nội dung quan trọng của Trí tuệ nhân tạo, đề tài “**Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A***” không trực tiếp sử dụng các thuật toán Học máy. BFS, DFS và A* là các **thuật toán tìm kiếm**, trong đó cách lựa chọn trạng thái được xác định từ trước dựa trên nguyên lý của từng thuật toán. Chương trình không cần huấn luyện mô hình bằng dữ liệu để có thể tìm đường.

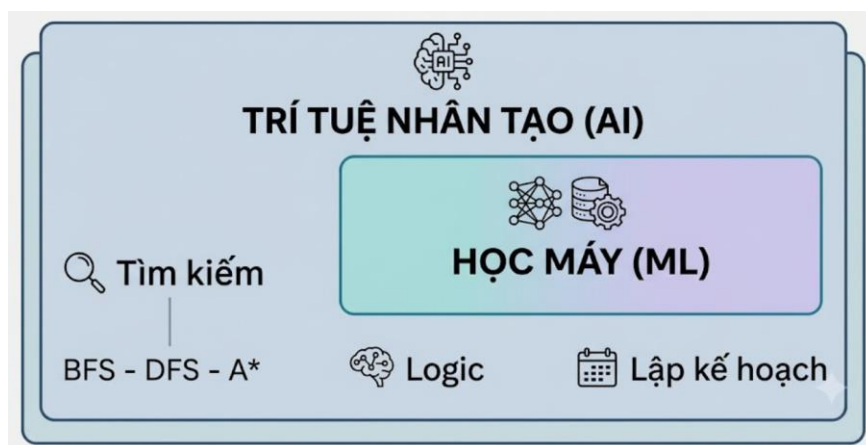
Đặc biệt, việc A* sử dụng hàm heuristic cũng không đồng nghĩa với việc A* là một thuật toán Học máy. Trong đề tài, giá trị heuristic được tính trực tiếp từ vị trí hiện tại đến điểm đích, chẳng hạn thông qua khoảng cách Manhattan. Đây là thông tin được xác định từ cấu trúc của bài toán chứ không phải là kết quả mà hệ thống học được từ tập dữ liệu.

Như vậy, Học máy có thể được xem là một trong nhiều hướng tiếp cận của Trí tuệ nhân tạo. Trong khi Học máy tập trung vào khả năng xây dựng mô hình từ dữ liệu, đề tài này tập trung vào **tìm kiếm trong không gian trạng thái**. Việc trình bày khái niệm Học máy giúp làm rõ phạm vi của Trí tuệ nhân tạo và phân biệt giữa các phương pháp học từ dữ liệu với các thuật toán tìm kiếm BFS, DFS và A* được sử dụng trong bài toán mê cung.

2.2.2. *Mối quan hệ giữa AI và Học máy*

Trí tuệ nhân tạo (Artificial Intelligence – AI) và Học máy (Machine Learning – ML) có mối quan hệ chặt chẽ với nhau, tuy nhiên đây không phải là hai khái niệm đồng nhất. Có thể hiểu Trí tuệ nhân tạo là một lĩnh vực rộng, hướng đến việc xây dựng các hệ thống có khả năng thực hiện những nhiệm vụ đòi hỏi quá trình suy luận, tìm kiếm, lập kế hoạch, nhận biết hoặc đưa ra quyết định. Trong khi đó, Học máy là một trong những phương pháp được sử dụng để xây dựng các hệ thống Trí tuệ nhân tạo.

Nếu xem AI là một phạm vi nghiên cứu lớn thì Học máy nằm bên trong phạm vi đó. AI có thể được triển khai bằng nhiều cách khác nhau, chẳng hạn sử dụng thuật toán tìm kiếm, hệ luật, logic, biểu diễn tri thức, lập kế hoạch hoặc các phương pháp học từ dữ liệu. Trong số đó, Học máy tập trung vào việc giúp máy tính xây dựng mô hình dựa trên dữ liệu và sử dụng mô hình đó để dự đoán, phân loại hoặc hỗ trợ đưa ra quyết định.



Hình 2.9: Mối quan hệ giữa Trí tuệ nhân tạo và Học máy

Sự khác biệt quan trọng giữa hai khái niệm nằm ở cách hệ thống hình thành phương pháp giải quyết vấn đề. Trong nhiều hệ thống AI truyền thống, các quy tắc xử lý hoặc chiến lược tìm kiếm được xác định trước bởi người thiết kế. Máy tính thực hiện quá trình suy luận hoặc tìm kiếm dựa trên những quy tắc này. Đối với Học máy, hệ thống không cần được mô tả thủ công toàn bộ các quy tắc cho từng tình huống mà có thể khai thác dữ liệu để tìm ra những mối quan hệ hoặc đặc điểm phục vụ cho nhiệm vụ cần thực hiện.

Ví dụ, một chương trình tìm đường bằng BFS có chiến lược hoạt động được xác định rõ từ trước: các trạng thái được mở rộng theo từng mức. DFS cũng có quy tắc cụ thể là ưu tiên đi sâu vào một nhánh, còn A* lựa chọn trạng thái dựa trên hàm đánh giá kết hợp giữa chi phí đã đi và chi phí ước lượng còn lại. Các thuật toán này thuộc phạm vi nghiên cứu của AI nhưng không phải là các thuật toán Học máy, bởi chúng không cần trải qua quá trình huấn luyện trên một tập dữ liệu để hình thành chiến lược tìm kiếm.

Ngược lại, trong một bài toán Học máy, chương trình thường cần một tập dữ liệu để xây dựng mô hình. Chẳng hạn, hệ thống nhận dạng hình ảnh có thể được cung cấp nhiều mẫu ảnh để học các đặc trưng của từng đối tượng. Sau khi quá trình huấn luyện hoàn thành, mô hình có thể được sử dụng để xử lý những dữ liệu mới. Điều này cho thấy Học máy là một hướng tiếp cận của AI dựa nhiều vào dữ liệu và khả năng tổng quát hóa từ những dữ liệu đã quan sát.

Mặc dù có sự khác biệt, AI và Học máy có thể kết hợp với nhau trong nhiều hệ thống. Một hệ thống AI có thể sử dụng Học máy để nhận biết hoặc dự đoán thông tin, sau đó sử dụng các thuật toán tìm kiếm hoặc lập kế hoạch để lựa chọn hành động phù hợp. Ví dụ, trong một hệ thống điều hướng robot, Học máy có thể hỗ trợ nhận biết vật cản từ hình ảnh, trong khi thuật toán tìm kiếm được sử dụng để xác định đường đi từ vị trí hiện tại đến vị trí mục tiêu.

Đối với đề tài **“Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A*”**, Học máy được trình bày nhằm làm rõ vị trí của nó trong lĩnh vực Trí tuệ nhân tạo, chứ không phải là phương pháp chính được sử dụng để giải quyết bài

toán. Trọng tâm của đề tài là các thuật toán tìm kiếm trong không gian trạng thái. BFS và DFS thực hiện tìm kiếm không có thông tin định hướng, trong khi A* sử dụng thêm hàm heuristic để hỗ trợ việc lựa chọn trạng thái tiếp theo.

Cần lưu ý rằng việc A* sử dụng heuristic không có nghĩa là thuật toán đang “học”. Trong mô hình mê cung của đề tài, heuristic có thể được xác định trực tiếp bằng khoảng cách Manhattan giữa trạng thái hiện tại và điểm đích. Giá trị này được tính dựa trên tọa độ của các ô và không được tạo ra thông qua quá trình huấn luyện từ dữ liệu. Vì vậy, A* vẫn được xếp vào nhóm thuật toán tìm kiếm có thông tin chứ không phải thuật toán Học máy.

Trí tuệ nhân tạo là một lĩnh vực rộng, trong đó Học máy là một nhánh tập trung vào việc xây dựng các mô hình có khả năng học từ dữ liệu. Bên cạnh Học máy, AI còn bao gồm nhiều hướng nghiên cứu khác như tìm kiếm, suy luận logic và lập kế hoạch. BFS, DFS và A* thuộc nhóm thuật toán tìm kiếm trong Trí tuệ nhân tạo và không phải là các thuật toán Học máy. Sự phân biệt này giúp làm rõ vị trí của các thuật toán được sử dụng trong đề tài và mối liên hệ giữa chúng với các nội dung khác của Trí tuệ nhân tạo.

2.2.3. Các phương pháp Học máy cơ bản

Học máy được chia thành nhiều phương pháp khác nhau tùy theo dạng dữ liệu, cách hệ thống tiếp nhận thông tin và mục tiêu cần đạt được. Trong phạm vi kiến thức cơ bản, có thể chia Học máy thành ba nhóm chính gồm **học có giám sát, học không giám sát và học tăng cường**.

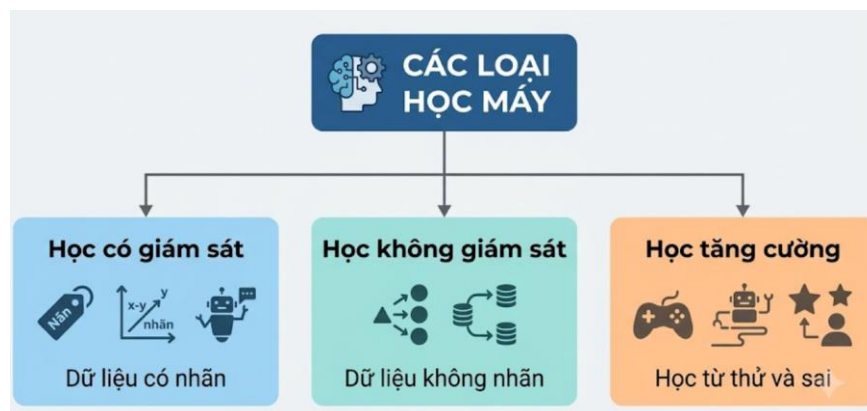
Học có giám sát (Supervised Learning) là phương pháp trong đó mô hình được huấn luyện bằng một tập dữ liệu đã có nhãn. Mỗi mẫu dữ liệu đầu vào đi kèm với một kết quả đúng tương ứng, từ đó mô hình học mối quan hệ giữa dữ liệu đầu vào và đầu ra. Sau quá trình huấn luyện, mô hình có thể sử dụng những gì đã học để dự đoán kết quả cho dữ liệu mới. Học có giám sát thường được sử dụng trong các bài toán phân loại và hồi quy, chẳng hạn như phân loại thư rác, nhận dạng đối tượng hoặc dự đoán giá trị.

Học không giám sát (Unsupervised Learning) được sử dụng trong trường hợp dữ liệu không có nhãn. Mô hình không được cung cấp trước kết quả đúng mà phải tự tìm ra cấu trúc, đặc điểm hoặc mối quan hệ tồn tại trong dữ liệu. Một số nhiệm vụ phổ biến của phương pháp này là phân cụm dữ liệu, giảm số chiều hoặc phát hiện các mẫu ẩn. Ví dụ, một hệ thống có thể tự động chia một tập dữ liệu thành nhiều nhóm dựa trên mức độ tương đồng giữa các phần tử.

Học tăng cường (Reinforcement Learning) là phương pháp trong đó một tác nhân tương tác với môi trường và học cách lựa chọn hành động thông qua cơ chế phần thưởng và phạt. Tác nhân thực hiện một hành động, quan sát trạng thái mới và nhận phản hồi từ môi trường. Qua nhiều lần tương tác, hệ thống điều chỉnh chiến lược để tăng tổng phần thưởng đạt được. Phương pháp này thường được sử dụng trong robot, trò chơi, điều khiển tự động và các bài toán ra quyết định theo chuỗi.

Ba phương pháp trên khác nhau chủ yếu ở cách hệ thống tiếp nhận thông tin trong quá trình học. Học có giám sát sử dụng dữ liệu đã gán nhãn, học không giám sát làm việc với dữ liệu chưa gán nhãn, còn học tăng cường dựa vào quá trình tương tác giữa tác nhân và môi trường.

Trong đề tài tìm đường trong mê cung, BFS, DFS và A* không thuộc ba phương pháp Học máy trên. Chúng là các thuật toán tìm kiếm được xây dựng dựa trên các quy tắc xác định trước. Chương trình không trải qua quá trình huấn luyện trên dữ liệu mà trực tiếp thực hiện tìm kiếm trong không gian trạng thái của mê cung.



Hình 2.10: Các phương pháp Học máy cơ bản

2.3. Logic trong Trí tuệ nhân tạo

2.3.1. Khái niệm logic

Logic là một lĩnh vực nghiên cứu về các nguyên tắc suy luận đúng, cách biểu diễn các phát biểu và mối quan hệ giữa chúng. Trong khoa học máy tính, logic được sử dụng để mô tả tri thức, xây dựng các quy tắc và hỗ trợ máy tính thực hiện quá trình suy luận từ những thông tin đã có.

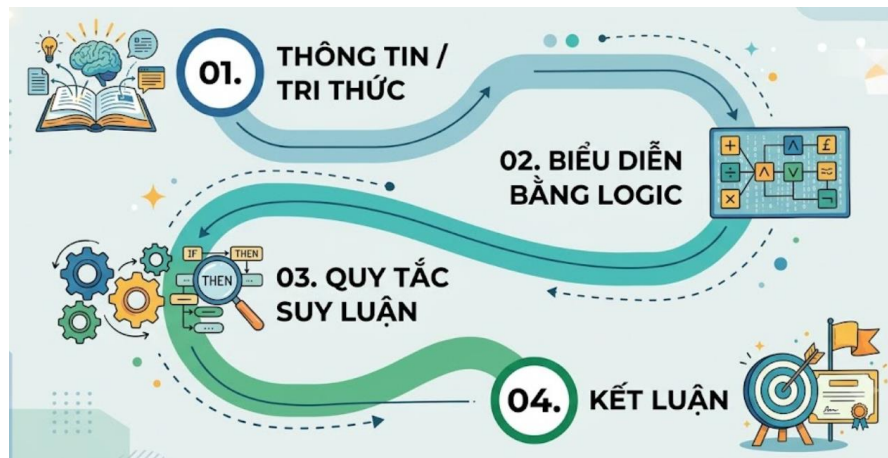
Trong Trí tuệ nhân tạo, logic đóng vai trò quan trọng trong việc biểu diễn tri thức dưới dạng hình thức để máy tính có thể xử lý. Thay vì chỉ lưu trữ thông tin dưới dạng dữ liệu rời rạc, hệ thống có thể sử dụng các biểu thức logic để thể hiện sự thật, điều kiện và các mối quan hệ giữa những đối tượng trong bài toán. Từ các thông tin đã biết, hệ thống có thể áp dụng các quy tắc suy luận để tạo ra những kết luận mới.

Một biểu thức logic thường có thể được xác định là đúng hoặc sai. Ví dụ, trong bài toán mê cung có thể xét phát biểu: **“Ô tại vị trí hiện tại có thể di chuyển sang bên phải.”** Phát biểu này chỉ đúng khi ô bên phải nằm trong phạm vi mê cung và không phải là vật cản. Các điều kiện tương tự xuất hiện thường xuyên trong quá trình kiểm tra trạng thái của chương trình.

Logic cũng giúp biểu diễn các điều kiện kết hợp. Chẳng hạn, một ô chỉ được đưa vào quá trình tìm kiếm khi đồng thời thỏa mãn nhiều điều kiện như nằm trong giới hạn của mê cung, không phải là tường và chưa được duyệt. Về mặt logic, điều này có thể được mô tả bằng quan hệ:

$$Valid(n) = InBound(n) \wedge \neg Wall(n) \wedge \neg Visited(n)$$

Trong đó, $InBound(n)$ biểu thị trạng thái (n) nằm trong phạm vi mê cung, $Wall(n)$ biểu thị (n) là ô tường và $Visited(n)$ biểu thị trạng thái đã được duyệt. Chỉ khi toàn bộ điều kiện cần thiết được thỏa mãn thì trạng thái mới được xem là hợp lệ để tiếp tục quá trình tìm kiếm.



Hình 2.11: Vai trò của Logic trong Trí tuệ nhân tạo

Trong AI, logic thường được sử dụng trong các bài toán như biểu diễn tri thức, hệ chuyên gia, chứng minh tự động, lập kế hoạch và suy luận dựa trên luật. Hai dạng logic thường được nghiên cứu ở mức cơ bản là **logic mệnh đề** và **logic vị từ**. Logic mệnh đề tập trung vào các phát biểu có giá trị đúng hoặc sai, trong khi logic vị từ cho phép biểu diễn chi tiết hơn về đối tượng, thuộc tính và quan hệ giữa các đối tượng.

Đối với đề tài “**Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A***”, logic không phải là phương pháp chính để tìm đường nhưng vẫn xuất hiện trong quá trình xây dựng chương trình thông qua các điều kiện kiểm tra trạng thái, xác định bước di chuyển hợp lệ và điều kiện kết thúc tìm kiếm. Phần logic được trình bày nhằm bổ sung kiến thức nền của học phần Trí tuệ nhân tạo trước khi đi sâu vào các thuật toán tìm kiếm.

2.3.2. Logic mệnh đề

Logic mệnh đề là một dạng logic được sử dụng để biểu diễn các phát biểu có giá trị **đúng** hoặc **sai**. Mỗi phát biểu được gọi là một mệnh đề và thường được ký hiệu bằng các chữ cái như P, Q, R. Giá trị của một mệnh đề chỉ nhận một trong hai trạng thái: đúng hoặc sai.

Ví dụ:

- P: “Ô phía trước có thể đi qua.”
- Q: “Ô phía trước đã được duyệt.”

- R: “Có thể tiếp tục di chuyển đến ô phía trước.”

Từ các mệnh đề đơn, có thể xây dựng những biểu thức phức tạp hơn bằng các phép toán logic. Một số phép toán thường gặp gồm:

- **Phủ định:** biểu thị mệnh đề P không đúng.
- **Phép hội:** đúng khi cả P và Q đều đúng.
- **Phép tuyển:** đúng khi ít nhất một trong hai mệnh đề đúng.
- **Phép kéo theo** nếu P đúng thì Q phải đúng.
- **Phép tương đương:** đúng khi P và Q có cùng giá trị chân lý.

Trong Trí tuệ nhân tạo, logic mệnh đề có thể được sử dụng để biểu diễn các sự kiện và quy tắc suy luận. Hệ thống có thể dựa vào những mệnh đề đã biết và các quy tắc để xác định một kết luận mới. Chẳng hạn, với bài toán mê cung, có thể xây dựng quy tắc:

$$P \wedge \neg Q \rightarrow R$$

Trong đó, P biểu thị ô phía trước có thể đi qua, Q biểu thị ô đó đã được duyệt và R biểu thị ô đó có thể được đưa vào quá trình tìm kiếm. Quy tắc trên có thể hiểu là: **nếu một ô có thể đi qua và chưa được duyệt thì chương trình có thể xem xét ô đó như một trạng thái tiếp theo.**

Giá trị của các biểu thức logic có thể được xác định thông qua bảng chân trị. Ví dụ đối với phép hội và phép tuyển:

Bảng 2.1: Bảng ví dụ chân trị

P	Q	P và Q	P hoặc Q
Đúng	Đúng	Đúng	Đúng
Đúng	Sai	Sai	Đúng
Sai	Đúng	Sai	Đúng

Sai	Sai	Sai	Sai
-----	-----	-----	-----

Trong quá trình xây dựng chương trình tìm đường, nhiều điều kiện xử lý có thể được mô tả theo logic mệnh đề. Ví dụ, một ô chỉ được xem là hợp lệ nếu nó nằm trong phạm vi của mê cung, không phải là vật cản và chưa được thuật toán duyệt. Điều kiện này có thể biểu diễn:

$$InBound(n) \wedge \neg Wall(n) \wedge \neg Visited(n)$$

Nếu biểu thức trên đúng, trạng thái (n) có thể được thêm vào danh sách các trạng thái cần xem xét.

Logic mệnh đề có ưu điểm là cách biểu diễn đơn giản, rõ ràng và phù hợp với những bài toán trong đó thông tin có thể mô tả bằng các phát biểu đúng hoặc sai. Tuy nhiên, logic mệnh đề khó biểu diễn chi tiết các đối tượng, thuộc tính và quan hệ giữa nhiều đối tượng. Những trường hợp đó thường cần đến **logic vị từ**, là nội dung được trình bày ở phần tiếp theo.

2.3.3. Logic vị từ

Logic vị từ, hay Predicate Logic, là một hệ thống logic được sử dụng để biểu diễn thông tin chi tiết hơn so với logic mệnh đề. Nếu logic mệnh đề chủ yếu xem một phát biểu là đúng hoặc sai thì logic vị từ cho phép mô tả rõ đối tượng, thuộc tính của đối tượng và mối quan hệ giữa các đối tượng.

Trong logic vị từ, một phát biểu thường được xây dựng từ vị từ, biến, hằng và lượng từ. Vị từ dùng để biểu diễn một thuộc tính hoặc một mối quan hệ nào đó. Ví dụ, trong bài toán mê cung có thể sử dụng $Walkable(x)$ để biểu diễn rằng vị trí x là một ô có thể đi qua.

Tương tự, $Wall(x)$ biểu diễn rằng vị trí x là một ô tường, còn $Visited(x)$ biểu diễn rằng vị trí x đã được thuật toán duyệt.

Logic vị từ cũng có thể mô tả mối quan hệ giữa hai hoặc nhiều đối tượng. Ví dụ, $Adjacent(x, y)$ có thể hiểu là vị trí x nằm kề vị trí y . Trong bài toán mê cung, đây là một

quan hệ quan trọng vì một trạng thái chỉ có thể chuyển sang những trạng thái nằm ở các ô lân cận hợp lệ.

Từ các vị trí trên, có thể xây dựng những biểu thức mô tả điều kiện di chuyển. Chẳng hạn:

$$\text{Walkable}(y) \wedge \text{Adjacent}(x, y) \rightarrow \text{CanMove}(x, y)$$

Biểu thức trên được hiểu là: nếu vị trí y có thể đi qua và y nằm kề vị trí x thì tác nhân có thể di chuyển từ x sang y .

Để biểu diễn các phát biểu có tính tổng quát, logic vị trí sử dụng các lượng từ. Hai lượng từ cơ bản thường gặp là lượng từ với mọi và lượng từ tồn tại.

Lượng từ với mọi được ký hiệu là $\forall$ và được đọc là “với mọi” hoặc “đối với tất cả”. Ví dụ:

$$\forall x, \text{Wall}(x) \rightarrow \neg \text{Walkable}(x)$$

Phát biểu này có nghĩa là: với mọi vị trí x , nếu x là một ô tường thì x không phải là một ô có thể đi qua.

Lượng từ tồn tại được ký hiệu là $\exists$ và được đọc là “tồn tại”. Ví dụ:

$$\exists x, \text{Goal}(x)$$

Biểu thức trên có nghĩa là: tồn tại một vị trí x là vị trí đích.

Trong bài toán tìm đường, các biểu thức logic vị trí có thể được sử dụng để mô tả những quy tắc liên quan đến trạng thái và quá trình di chuyển. Chẳng hạn, một trạng thái lân cận chỉ được xem xét khi vị trí đó có thể đi qua và chưa được duyệt:

$$\text{Adjacent}(x, y) \wedge \text{Walkable}(y) \wedge \neg \text{Visited}(y) \rightarrow \text{ValidNextState}(y)$$

Trong đó, $\text{ValidNextState}(y)$ biểu diễn rằng y là một trạng thái hợp lệ có thể được đưa vào quá trình tìm kiếm.

Logic vị trí có khả năng biểu diễn thông tin linh hoạt hơn logic mệnh đề. Ví dụ, nếu sử dụng logic mệnh đề, chương trình có thể cần các mệnh đề riêng như “ô A có thể đi qua”, “ô B có thể đi qua”, “ô C có thể đi qua”. Với logic vị trí, có thể sử dụng chung

vị từ $Walkable(x)$ và thay x bằng từng vị trí cụ thể. Cách biểu diễn này có tính tổng quát hơn và thuận tiện khi số lượng đối tượng trong bài toán lớn.

Một số thành phần thường gặp trong logic vị từ được trình bày trong bảng sau:

Bảng 2.2: Một số thành phần cơ bản trong logic vị từ

Thành phần	Ký hiệu/Ví dụ	Ý nghĩa
Vị từ	$Wall(x)$	x là một ô tường
Biến	x, y	Đại diện cho một đối tượng
Quan hệ	$Adjacent(x, y)$	x và y là hai vị trí kề nhau
Phủ định	$\neg Wall(x)$	x không phải là tường
Với mọi	$\forall x$	Đúng với mọi x
Tồn tại	$\exists x$	Có ít nhất một x thỏa mãn
Phép hội	$\wedge$	Và
Phép tuyển	$\vee$	Hoặc
Kéo theo	$\rightarrow$	Nếu... thì...

Trong Trí tuệ nhân tạo, logic vị từ thường được sử dụng trong các bài toán biểu diễn tri thức, suy luận, hệ chuyên gia và lập kế hoạch. Hệ thống có thể sử dụng các sự kiện đã biết kết hợp với các quy tắc logic để suy ra thông tin mới hoặc xác định hành động phù hợp.

Đối với đề tài “Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A*”, logic vị từ không trực tiếp thực hiện quá trình tìm đường. Các thuật toán BFS, DFS và A* vẫn là thành phần chính được sử dụng để khám phá không gian trạng thái. Logic vị từ giúp mô tả rõ hơn các đối tượng và quan hệ trong bài toán, chẳng

hạn vị trí nào là tường, vị trí nào có thể di chuyển, hai trạng thái có nằm kề nhau hay không và trạng thái nào đã được thuật toán duyệt.

2.3.4. Vai trò của logic trong AI

Logic giữ vai trò quan trọng trong Trí tuệ nhân tạo vì nó cung cấp một cách biểu diễn tri thức rõ ràng, có cấu trúc và có thể xử lý bằng máy tính. Thông qua logic, các sự kiện, điều kiện và mối quan hệ trong một bài toán có thể được mô tả dưới dạng các biểu thức hình thức, từ đó giúp hệ thống thực hiện quá trình suy luận và đưa ra kết luận dựa trên những thông tin đã biết.

Một trong những vai trò cơ bản của logic là **biểu diễn tri thức**. Trong nhiều hệ thống AI, thông tin không chỉ được lưu dưới dạng dữ liệu đơn lẻ mà còn cần thể hiện quan hệ giữa các đối tượng. Logic cho phép mô tả những thông tin này bằng mệnh đề, vị từ và các quy tắc. Nhờ đó, hệ thống có thể tổ chức tri thức theo một cách nhất quán và dễ kiểm tra.

Logic cũng được sử dụng để **thực hiện suy luận**. Khi đã có một số sự kiện và quy tắc, hệ thống có thể dựa vào chúng để rút ra kết luận mới. Ví dụ, nếu biết một vị trí là ô có thể đi qua và vị trí đó nằm kề với trạng thái hiện tại, hệ thống có thể suy ra rằng đây là một vị trí có thể được xem xét trong quá trình di chuyển. Cách suy luận này giúp máy tính xử lý bài toán theo những điều kiện đã xác định trước.

Trong các hệ thống chuyên gia, logic còn được sử dụng để xây dựng các quy tắc dạng “**nếu... thì...**”. Những quy tắc này mô tả cách hệ thống đưa ra quyết định dựa trên các điều kiện đầu vào. Khi các điều kiện được thỏa mãn, hệ thống có thể kích hoạt một kết luận hoặc một hành động tương ứng. Đây là một trong những cách thể hiện rõ mối liên hệ giữa logic và quá trình ra quyết định trong AI.

Bên cạnh đó, logic có vai trò trong **lập kế hoạch và giải quyết vấn đề**. Một hệ thống AI cần xác định trạng thái hiện tại, mục tiêu cần đạt và những hành động hợp lệ có thể thực hiện. Các điều kiện logic được sử dụng để xác định hành động nào được phép, trạng thái nào hợp lệ và khi nào mục tiêu đã đạt được.

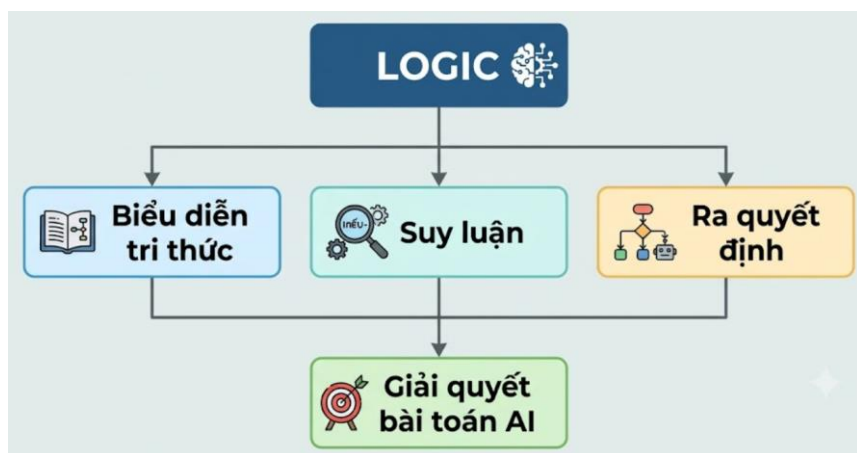
Đối với đề tài “**Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A***”, logic không trực tiếp thay thế các thuật toán tìm kiếm nhưng được sử dụng trong quá trình kiểm tra và xử lý trạng thái. Ví dụ, một ô chỉ được xem là hợp lệ khi nó nằm trong giới hạn mê cung, không phải là tường và chưa được duyệt. Điều kiện này có thể biểu diễn:

$$InBound(n) \wedge \neg Wall(n) \wedge \neg Visited(n)$$

Nếu biểu thức trên đúng, trạng thái (n) có thể được đưa vào quá trình tìm kiếm. Điều này cho thấy logic hỗ trợ chương trình xác định chính xác những trạng thái nào có thể được mở rộng.

Logic còn giúp mô tả điều kiện kết thúc của bài toán. Khi trạng thái hiện tại trùng với trạng thái đích, chương trình có thể kết luận rằng quá trình tìm kiếm đã đạt mục tiêu. Nếu không còn trạng thái hợp lệ nào để mở rộng mà vẫn chưa tới đích, chương trình có thể kết luận rằng không tồn tại đường đi.

Trong phạm vi học phần Trí tuệ nhân tạo, logic là nền tảng cho nhiều nội dung khác như biểu diễn tri thức, hệ chuyên gia, lập kế hoạch, suy luận và giải quyết vấn đề. Đối với đề tài này, việc tìm hiểu logic giúp làm rõ hơn cách chương trình kiểm tra điều kiện và tổ chức quá trình xử lý trong khi BFS, DFS và A* thực hiện nhiệm vụ tìm kiếm trong không gian trạng thái.



Hình 2.12: Vai trò của Logic trong Trí tuệ nhân tạo

2.4. Bài toán tìm đường trong mê cung

Bài toán tìm đường trong mê cung là một dạng bài toán tìm kiếm cơ bản trong lĩnh vực trí tuệ nhân tạo. Mục tiêu của bài toán là xác định một đường đi hợp lệ từ vị trí bắt đầu đến vị trí đích trong một không gian có chứa các vị trí được phép di chuyển và các vị trí bị cản trở. Tùy thuộc vào cách tổ chức không gian và quy tắc di chuyển, bài toán có thể được giải quyết bằng nhiều phương pháp tìm kiếm khác nhau. Trong đề tài này, bài toán được sử dụng làm môi trường chung để nghiên cứu và so sánh ba thuật toán BFS, DFS và A*.

Mê cung trong đề tài được biểu diễn dưới dạng một lưới hai chiều gồm nhiều hàng và cột. Mỗi ô trong lưới tương ứng với một vị trí mà người chơi hoặc tác nhân có thể đứng tại đó. Các ô trong mê cung được chia thành những loại cơ bản gồm ô có thể đi qua, ô tường, điểm bắt đầu và điểm đích. Điểm bắt đầu ký hiệu là S , là nơi quá trình tìm kiếm được bắt đầu. Điểm đích ký hiệu là G , là vị trí mà thuật toán cần tìm đến. Những ô tường đóng vai trò là vật cản, không cho phép tác nhân đi xuyên qua.

Ví dụ, một mê cung có thể được biểu diễn như sau:

S	0	0	1	0	0
1	1	0	1	0	1
0	0	0	0	0	0
0	1	1	1	1	0
0	0	0	0	0	G

Trong đó, 0 biểu thị ô có thể di chuyển và 1 biểu thị ô bị chặn. Từ vị trí S , người chơi phải lựa chọn các bước di chuyển hợp lệ để đến được G .

Trong phạm vi nghiên cứu của đề tài, tác nhân được phép di chuyển giữa các ô kề nhau theo bốn hướng cơ bản, gồm lên, xuống, sang trái và sang phải. Từ một ô hiện tại (x, y) , các vị trí có thể xem xét là:

$$(x, -1y), (x, +1y), (x, y - 1), (x, y + 1)$$

Tuy nhiên, không phải tất cả các vị trí này đều có thể được lựa chọn. Một vị trí lân cận chỉ được xem là hợp lệ khi nó nằm trong giới hạn của mê cung, không phải là ô tường và đáp ứng các điều kiện kiểm tra của thuật toán. Việc xác định chính xác các trạng thái kế tiếp là một

bước quan trọng, bởi toàn bộ quá trình tìm kiếm của BFS, DFS và A* đều dựa trên tập các trạng thái có thể mở rộng từ trạng thái hiện tại.

Xét dưới góc độ trí tuệ nhân tạo, mê cung có thể được mô hình hóa thành một không gian trạng thái. Mỗi ô có thể đi qua tương ứng với một trạng thái, còn việc di chuyển từ một ô sang ô kề bên được xem là một phép chuyển trạng thái. Khi đó, điểm bắt đầu S là trạng thái ban đầu và điểm đích G là trạng thái mục tiêu. Bài toán tìm đường thực chất trở thành việc tìm một chuỗi trạng thái:

$$S \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_n \rightarrow G$$

sao cho mỗi cặp trạng thái liên tiếp đều có thể chuyển đổi hợp lệ với nhau.

Cách mô hình hóa trên cho phép xem mê cung như một đồ thị. Các ô có thể đi qua được xem là các đỉnh, còn khả năng di chuyển giữa hai ô kề nhau tương ứng với các cạnh của đồ thị. Với quy tắc di chuyển bốn hướng và chi phí mỗi bước như nhau, mỗi lần di chuyển từ một ô sang một ô hợp lệ có thể được xem là có chi phí bằng 1. Khi đó, độ dài của một đường đi được xác định bằng tổng số bước cần thực hiện từ điểm bắt đầu đến điểm đích.

Từ mô hình này, bài toán không còn đơn thuần là việc “đi trong mê cung”, mà trở thành bài toán tìm kiếm trong đồ thị. Máy tính phải lần lượt lựa chọn các trạng thái để khám phá cho đến khi tìm thấy trạng thái mục tiêu hoặc xác định rằng không còn trạng thái nào có thể tiếp tục tìm kiếm. Cách lựa chọn trạng thái ở mỗi bước chính là yếu tố tạo nên sự khác biệt giữa các thuật toán.

Đối với BFS, các trạng thái được mở rộng theo từng lớp tính từ điểm bắt đầu. Thuật toán ưu tiên xử lý những trạng thái được phát hiện trước và có cùng mức khoảng cách trước khi tiếp tục mở rộng sang mức sâu hơn. Với trường hợp chi phí mỗi bước là như nhau, đặc điểm này giúp BFS có khả năng tìm được đường đi ngắn nhất từ S đến G .

Đối với DFS, thuật toán không mở rộng đồng đều theo từng lớp mà ưu tiên đi sâu vào một nhánh trước. Khi gặp vị trí không thể tiếp tục hoặc không dẫn đến lời giải, thuật toán quay lui để tiếp tục xét các nhánh khác. Vì cách lựa chọn hướng đi phụ thuộc vào thứ tự duyệt, DFS có thể nhanh chóng tìm thấy đích trong một số trường hợp nhưng không đảm bảo đường đi tìm được là ngắn nhất.

Trong khi đó, A* sử dụng thêm thông tin ước lượng về khoảng cách từ trạng thái hiện tại đến đích để định hướng quá trình tìm kiếm. Thuật toán đánh giá mỗi trạng thái thông qua hàm:

$$f(n) = g(n) + h(n)$$

Trong đó $g(n)$ là chi phí thực tế từ điểm bắt đầu đến trạng thái n , còn $h(n)$ là chi phí ước lượng từ n đến điểm đích. Với mê cung chỉ cho phép di chuyển theo bốn hướng và mỗi bước có cùng chi phí, khoảng cách Manhattan có thể được sử dụng làm heuristic:

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

Nhờ đó, A* có xu hướng ưu tiên những trạng thái vừa có chi phí di chuyển thấp vừa được ước lượng là gần điểm đích.

Một vấn đề quan trọng khác của bài toán là tránh lặp lại các trạng thái đã được khám phá. Trong một mê cung có nhiều đường nối giữa các ô, nếu chương trình không lưu lại những vị trí đã được duyệt, cùng một ô có thể bị kiểm tra nhiều lần hoặc thậm chí dẫn đến quá trình tìm kiếm lặp vô hạn trong một số cách cài đặt. Vì vậy, trong quá trình tìm kiếm cần có cơ chế đánh dấu trạng thái đã được khám phá. Đồng thời, chương trình cần lưu lại trạng thái trước đó của mỗi vị trí để sau khi tìm thấy điểm đích có thể truy ngược và khôi phục đường đi.

Quá trình giải bài toán có thể chia thành hai giai đoạn chính. Giai đoạn thứ nhất là khám phá không gian tìm kiếm, trong đó thuật toán lần lượt kiểm tra và mở rộng các trạng thái theo chiến lược riêng của mình. Giai đoạn thứ hai là truy vết đường đi, được thực hiện sau khi điểm đích được tìm thấy. Từ điểm G , chương trình lần lượt truy ngược qua các trạng thái cha cho đến khi quay về S , sau đó đảo ngược chuỗi kết quả để thu được đường đi theo đúng thứ tự từ điểm bắt đầu đến điểm đích.

Trường hợp không tồn tại đường đi cũng là một phần của bài toán. Khi toàn bộ các trạng thái có thể tiếp cận đã được khám phá nhưng điểm đích vẫn chưa được tìm thấy, chương trình phải kết luận rằng mê cung không có đường đi từ S đến G . Việc xử lý trường hợp này giúp chương trình không chỉ kiểm tra khả năng tìm được lời giải mà còn bảo đảm kết quả tìm kiếm chính xác đối với cả hai khả năng có và không có lời giải.

Như vậy, bài toán tìm đường trong mê cung có thể được khái quát thành việc tìm một chuỗi trạng thái hợp lệ từ trạng thái ban đầu S đến trạng thái mục tiêu G trên một không gian tìm kiếm dạng lưới. Mặc dù mô hình mê cung tương đối đơn giản, bài toán chứa đầy đủ những thành phần cơ bản của một bài toán tìm kiếm trong trí tuệ nhân tạo, bao gồm trạng thái, trạng thái ban đầu, trạng thái mục tiêu, các phép chuyển trạng thái, không gian tìm kiếm và tiêu chí đánh giá lời giải. Đây là cơ sở để đề tài áp dụng BFS, DFS và A* vào cùng một bài toán, từ đó nghiên cứu và so sánh đặc điểm cũng như hiệu quả của từng thuật toán.

2.5. Mô hình hóa mê cung dưới dạng đồ thị

Để có thể áp dụng các thuật toán tìm kiếm BFS, DFS và A* vào bài toán tìm đường trong mê cung, trước hết cần chuyển mê cung từ dạng hình ảnh trực quan sang một mô hình dữ liệu mà máy tính có thể xử lý. Trong đề tài này, mê cung được mô hình hóa dưới dạng đồ thị trên lưới hai chiều. Cách biểu diễn này giúp bài toán tìm đường trở thành một bài toán tìm kiếm trên đồ thị, từ đó có thể áp dụng thống nhất các thuật toán đã lựa chọn.

Mê cung được chia thành nhiều ô vuông có kích thước bằng nhau, được sắp xếp thành các hàng và cột. Mỗi ô tương ứng với một vị trí cụ thể trong mê cung và có thể xác định bằng tọa độ (i, j) , trong đó i là chỉ số hàng và j là chỉ số cột. Tùy theo trạng thái của ô, nó có thể là vị trí được phép di chuyển hoặc là vật cản. Ngoài ra, trong mê cung tồn tại hai vị trí đặc biệt là điểm bắt đầu S và điểm đích G .

Có thể biểu diễn một mê cung đơn giản như sau:

S	0	0	1	0
1	0	0	1	0
0	0	1	0	0
1	0	0	0	0
0	1	1	0	G

Trong đó, 0 biểu thị một ô có thể đi qua, 1 biểu thị ô tường, S là điểm bắt đầu và G là điểm đích. Mỗi ô có thể đi qua được xem là một đỉnh của đồ thị. Khả năng di chuyển giữa hai ô hợp lệ được biểu diễn bằng cạnh nối giữa hai đỉnh tương ứng.

2.5.1. *Biểu diễn đỉnh của đồ thị*

Trong mô hình này, mỗi ô có thể di chuyển được tương ứng với một đỉnh v của đồ thị. Nếu mê cung có m hàng và n cột thì về mặt hình thức, mỗi vị trí có thể được xác định bởi một cặp tọa độ:

$$v = (i, j)$$

với:

$$0 \leq i < m, 0 \leq j < n$$

Không phải tất cả các ô trong ma trận đều trở thành đỉnh hợp lệ. Những ô được xác định là tường sẽ không tham gia vào quá trình tìm kiếm vì tác nhân không thể đứng tại đó hoặc đi xuyên qua đó.

Điểm bắt đầu Schính là đỉnh ban đầu của quá trình tìm kiếm, trong khi điểm đích G là đỉnh mục tiêu mà thuật toán cần đạt tới. Như vậy, bài toán có thể được phát biểu dưới dạng:

Tìm đường đi từ S đến G

trên tập các đỉnh hợp lệ của đồ thị.

2.5.2. *Biểu diễn cạnh giữa các đỉnh*

Sau khi xác định các đỉnh, bước tiếp theo là xác định quan hệ kết nối giữa chúng. Trong phạm vi đề tài, người chơi chỉ được phép di chuyển theo bốn hướng: lên, xuống, trái và phải. Vì vậy, từ một ô hiện tại (i, j) , chương trình kiểm tra tối đa bốn vị trí lân cận:

$$(i, -1j)$$

$$(i, +1j)$$

$$(i, j - 1)$$

$$(i, j + 1)$$

Nếu một vị trí nằm trong giới hạn của mê cung và không phải là ô tường thì giữa hai vị trí tương ứng tồn tại một cạnh trong đồ thị.

Ví dụ, xét ô tại vị trí (2, 2). Nếu cả bốn ô phía trên, phía dưới, bên trái và bên phải đều là ô có thể đi qua thì đỉnh tương ứng sẽ được kết nối với bốn đỉnh lân cận. Ngược lại, nếu một trong các vị trí đó là tường thì cạnh tương ứng không tồn tại.

Có thể biểu diễn quan hệ này dưới dạng:

$$E \subseteq V \times V$$

trong đó V là tập các đỉnh và E là tập các cạnh biểu diễn khả năng di chuyển giữa các vị trí.

2.5.3. Đồ thị của mê cung

- Từ cách xây dựng trên, mê cung có thể được biểu diễn dưới dạng đồ thị:

$$G = (V, E)$$

Trong đó:

- + V là tập các ô có thể đi qua trong mê cung.
- + E là tập các liên kết giữa những ô mà tác nhân có thể di chuyển trực tiếp.
- Do việc di chuyển giữa hai ô là có thể theo cả hai chiều nên trong mô hình cơ bản này, đồ thị có thể được xem là đồ thị vô hướng. Nếu có thể đi từ ô A sang ô B thì cũng có thể đi từ B trở lại A , với điều kiện không có quy tắc đặc biệt nào hạn chế hướng di chuyển.
- Ví dụ, nếu hai ô có thể di chuyển trực tiếp:

$$A \leftrightarrow B$$

thì trong đồ thị sẽ tồn tại quan hệ kết nối giữa hai đỉnh tương ứng.

- Trong trường hợp mỗi bước di chuyển có cùng chi phí, mỗi cạnh có thể được gán trọng số:

$$w(A, B) = 1$$

- Như vậy, chi phí của một đường đi chính là tổng số cạnh mà đường đi đó sử dụng. Đây chính là cơ sở để đánh giá độ dài đường đi trong chương trình.

2.5.4. Không gian trạng thái của bài toán

- Khi mê cung được biểu diễn thành đồ thị, mỗi đỉnh không chỉ đơn thuần là một vị trí trên màn hình mà còn có thể được xem là một trạng thái của bài toán tìm kiếm. Tại

mỗi trạng thái, tác nhân có một tập các trạng thái kế tiếp tương ứng với những ô có thể di chuyển tới.

- Ví dụ, nếu tác nhân đang ở trạng thái:

$$s = (i, j)$$

thì tập trạng thái kế tiếp có thể được xác định bởi:

$$Successors(s) = \{s_1, s_2, \dots, s_k\}$$

với k có thể từ 0 đến 4 tùy thuộc vào vị trí của ô và các vật cản xung quanh.

- Từ góc nhìn này, bài toán tìm đường có đầy đủ những thành phần cơ bản của một bài toán tìm kiếm:

$$P = (S, A, R, resultGoal)$$

trong đó trạng thái ban đầu là vị trí S , các hành động là những bước di chuyển hợp lệ, hàm chuyển trạng thái xác định vị trí tiếp theo và điều kiện mục tiêu là đạt tới G .

- Cách nhìn này đặc biệt quan trọng đối với đề tài vì nó giúp thống nhất cách hiểu về BFS, DFS và A^* . Cả ba thuật toán đều thực hiện tìm kiếm trên cùng một không gian trạng thái, điểm khác biệt chủ yếu nằm ở cách lựa chọn trạng thái tiếp theo để mở rộng.

2.5.5. *Vai trò của đồ thị đối với BFS, DFS và A^**

- Sau khi mô hình được mô hình hóa thành đồ thị, ba thuật toán có thể được áp dụng trực tiếp trên cấu trúc này.
- Đối với BFS, thuật toán bắt đầu từ đỉnh S , sau đó lần lượt mở rộng các đỉnh ở cùng khoảng cách trước khi chuyển sang những đỉnh ở mức sâu hơn. Với đồ thị mà mỗi cạnh có cùng trọng số, thứ tự này bảo đảm rằng khi đỉnh đích được phát hiện, đường đi thu được có số bước ngắn nhất.
- Đối với DFS, thuật toán cũng bắt đầu từ S nhưng thay vì mở rộng theo từng mức, nó ưu tiên đi sâu theo một nhánh. Khi gặp một trạng thái không thể tiếp tục, thuật toán quay lui để khám phá nhánh khác. Vì vậy, kết quả tìm kiếm của DFS phụ thuộc khá nhiều vào thứ tự các đỉnh lân cận được xem xét.

- Đối với A^* , đồ thị cung cấp các trạng thái và chi phí chuyển đổi giữa chúng, còn tọa độ của từng ô cung cấp thông tin để xây dựng hàm heuristic. Với mê cung chỉ cho phép di chuyển bốn hướng, có thể sử dụng khoảng cách Manhattan:

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

- Từ đó, A^* sử dụng:

$$f(n) = g(n) + h(n)$$

để xác định mức độ ưu tiên của từng trạng thái. Nhờ thông tin vị trí của các đỉnh trên lưới, A^* có thể hướng quá trình tìm kiếm về phía điểm đích thay vì chỉ dựa vào khoảng cách đã đi.

2.5.6. Ví dụ về quá trình chuyển từ mê cung sang đồ thị

- Xét một mê cung đơn giản:

S 0 1

0 0 0

1 0 G

- Các ô tường không được đưa vào tập đỉnh. Những ô còn lại trở thành các trạng thái của đồ thị.
- Từ S , tác nhân có thể đi xuống. Tại vị trí tiếp theo, tác nhân có thể tiếp tục di chuyển sang phải hoặc xuống dưới tùy theo cấu trúc của mê cung. Quá trình này tạo ra một mạng lưới các trạng thái liên kết với nhau.
- Một đường đi hợp lệ có thể có dạng:

$$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$$

- Trong đó mỗi bước chuyển giữa hai trạng thái liên tiếp tương ứng với một cạnh trong đồ thị.
- + Nếu tồn tại nhiều đường đi khác nhau từ S đến G , đây chính là lúc sự khác biệt giữa BFS, DFS và A^* trở nên rõ ràng. Các thuật toán có thể khám phá những đỉnh theo thứ tự khác nhau và do đó có thể cần số lượng bước tìm kiếm khác nhau trước khi đạt đến đích.

2.5.7. Ý nghĩa của việc mô hình hóa

- Việc mô hình hóa mê cung dưới dạng đồ thị là bước trung gian quan trọng giữa bài toán thực tế và thuật toán tìm kiếm. Người sử dụng nhìn thấy một mê cung gồm các ô vuông và tường, nhưng đối với chương trình, mê cung cần được chuyển thành một tập hợp các trạng thái và mối quan hệ giữa chúng. Khi quá trình chuyển đổi này hoàn tất, BFS, DFS và A* có thể được áp dụng trên cùng một mô hình mà không cần thay đổi bản chất của bài toán.
- Ngoài ra, cách mô hình hóa này cũng tạo điều kiện cho việc so sánh các thuật toán một cách khách quan. Vì cả ba cùng hoạt động trên cùng tập đỉnh, cùng tập cạnh và cùng điểm bắt đầu, điểm đích, sự khác biệt trong kết quả chủ yếu xuất phát từ chiến lược tìm kiếm của từng thuật toán. Đây là cơ sở quan trọng cho phần thực nghiệm và đánh giá của đề tài.
- Như vậy, mô hình hóa mê cung dưới dạng đồ thị không chỉ giúp chương trình có thể xử lý bài toán một cách có hệ thống mà còn tạo cầu nối trực tiếp giữa mô hình mê cung thực quan và các thuật toán tìm kiếm trong trí tuệ nhân tạo. Trên cơ sở mô hình này, các chương tiếp theo sẽ lần lượt trình bày chi tiết nguyên lý hoạt động của BFS, DFS và A* và cách chúng được áp dụng để tìm đường từ S đến G.

2.6. Thuật toán BFS

Thuật toán tìm kiếm không có thông tin được xem xét trước tiên là tìm kiếm theo chiều rộng (Breadth-first search, viết tắt là BFS), một dạng tìm kiếm vét cạn.

Nguyên tắc của tìm kiếm theo chiều rộng là trong số những nút biên lựa chọn nút nông nhất (gần nút gốc nhất) để mở rộng. Như vậy, trước hết tất cả các nút có độ sâu bằng 0 (nút gốc) được mở rộng, sau đó tới các nút có độ sâu bằng 1 được mở rộng, rồi tới các nút có độ sâu bằng 2, và tiếp tục như vậy. Ở đây, độ sâu được tính bằng số nút nằm trên đường đi từ nút gốc tới nút đang xét.

Có thể nhận thấy, để thực hiện nguyên tắc tìm kiếm theo chiều rộng, ta cần lựa chọn nút được thêm vào sớm hơn trong danh sách nút biên O để mở rộng. Điều này có thể thực hiện dễ dàng bằng cách dùng một hàng đợi FIFO để lưu các nút biên.

Thuật toán tìm theo chiều rộng được thể hiện trên hình 2.7. Khác với thuật toán tìm kiếm tổng quát ở trên, tập nút biên O được tổ chức dưới dạng hàng đợi FIFO: các nút mới sinh ra được thêm vào cuối của O tại bước 3 của mỗi vòng lặp; nút đầu tiên của O sẽ được lấy ra để mở rộng như tại bước 1 của vòng lặp của thuật toán trên hình vẽ. Bước 2 của vòng lặp kiểm tra điều kiện đích và trả về kết quả trong trường hợp nút lấy ra từ O là nút đích. Thuật toán kết thúc trong hai trường hợp: 1) khi lấy được nút đích từ O ; và 2) khi tập O rỗng. Hai trường hợp này tương ứng với hai lệnh return ở trong và ngoài vòng lặp.

Con trỏ ngược: khi mở rộng một nút ta cần sử dụng con trỏ ngược để ghi lại nút cha của nút vừa được mở ra. Con trỏ này được sử dụng để tìm ngược lại đường đi về trạng thái xuất phát khi tìm được trạng thái đích. Khi cài đặt thuật toán, mỗi nút được biểu diễn bằng một cấu trúc dữ liệu có chứa một con trỏ ngược trỏ tới nút cha.

Sau khi tìm được nút đích, có thể khôi phục đường đi tới nút đó bằng cách lần theo các con trỏ ngược, bắt đầu từ nút đích.

BFS (Q, S, G, P)

Đầu vào: bài toán tìm kiếm

Đầu ra: trạng thái đích

Khởi tạo: $O \leftarrow S$ // trong thuật toán này, O là hàng đợi FIFO

While (O không rỗng) do

1. Chọn nút đầu tiên n từ O và xóa n khỏi O
2. If $n \in G$, return (đường đi tới n)
3. Thêm $P(n)$ vào cuối O

Return: Không có lời giải

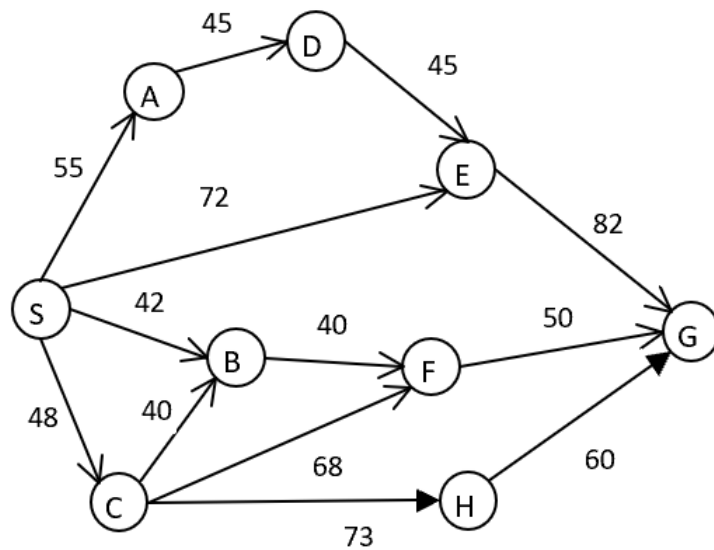
Hình 2.13: Thuật toán tìm kiếm theo chiều rộng

Các cải tiến. Một số cải tiến sau đây có thể sử dụng kết hợp với thuật toán tìm theo chiều rộng vừa trình bày.

Tránh xem xét lại các nút đã mở rộng. Việc xem xét lại các nút đã mở rộng có thể dẫn tới v ng lặp. Mặc dù v ng lặp không ảnh hưởng tới khả năng tìm ra lời giải của tìm theo chiều rộng, xong việc có v ng lặp và xem xét lại các nút làm tăng độ phức tạp tính toán do phải khảo sát nhiều nút hơn. Vấn đề này có thể giải quyết bằng cách sử dụng tập đóng tương tự trong thuật toán Graph_Search trên hình 2.6. Một trạng thái đã có mặt trong tập đóng hoặc tập biên sẽ không được thêm vào tập biên nữa. Người đọc có thể tự kiểm tra kết quả của thuật toán trong trường hợp có sử dụng kiểm tra tập đóng và tập biên với trường hợp không kiểm tra bằng cách thực hiện thuật toán với ví dụ trên hình 2.8.

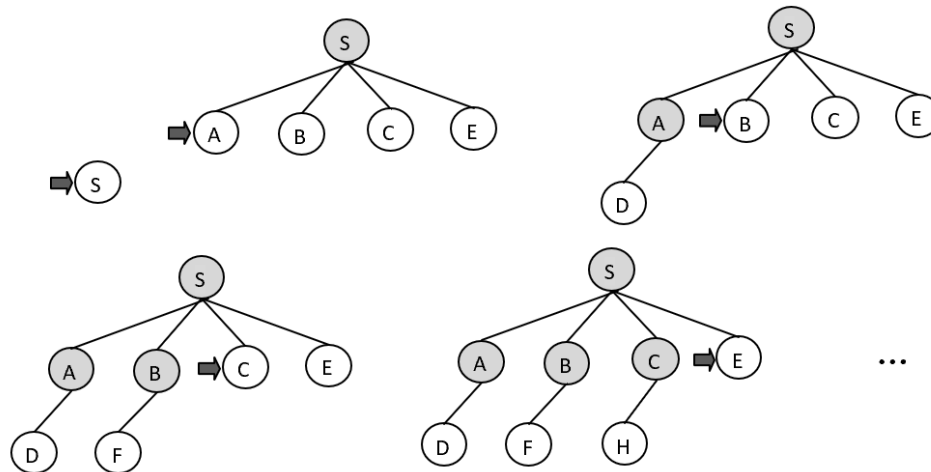
Kiểm tra đích trước khi thêm vào tập biên. Trong thuật toán tổng quát, việc kiểm tra điều kiện đích được thực hiện khi nút được lấy ra khỏi O để mở rộng. Thay vào đó, ta có thể kiểm tra trước khi thêm nút vào O. Ưu điểm của cách làm này là giảm bớt số lượng nút cần lưu trong nút biên cũng như số nút được mở rộng. Cụ thể về số lượng nút cần mở rộng khi thực hiện kiểm tra đích trước sẽ được trình bày trong phần tính chất của thuật toán ở bên dưới.

Ví dụ: Xét ví dụ tìm đường đi từ nút S tới nút G trên đồ thị ở hình 2.8. (để đơn giản, ví dụ này sử dụng đồ thị có hướng; quá trình tìm đường đi trên đồ thị vô hướng được thực hiện tương tự, trừ việc từ một nút có thể di chuyển sang nút cha của nút đó)



Hình 14.2: Ví dụ đồ thị cho bài toán tìm đường đi

Một số bước đầu tiên của thuật toán, có sử dụng việc lưu và kiểm tra tập nút đóng, được thể hiện dưới dạng các cây tìm kiếm như trên hình 2.9. Lưu ý: để thống nhất trong việc trình bày, trong số các nút có vai trò giống nhau, tức là có cùng độ sâu, nút đứng trước trong bảng chữ cái sẽ được mở rộng trước. Quy tắc này không được quy định trong thuật toán và chỉ để tiện cho trình bày.



Hình 2.15: Một số cây tìm kiếm sinh ra khi tìm kiếm theo chiều rộng

Kết quả thực hiện các bước của thuật toán cũng có thể theo dõi qua thứ tự mở rộng các nút và nội dung danh sách tập nút biên. Dưới đây là minh họa cho thứ tự mở rộng nút và tập biên cho ví dụ trên hình 2.3 (có sử dụng tập nút đóng). Để tiện cho trình bày, ta sẽ quy định phía bên trái là đầu của hàng đợi O và phía bên phải là cuối của hàng đợi O. Như vậy các nút được thêm vào từ phía bên phải và lấy ra từ bên trái. Con trở tới nút cha sẽ được viết dưới dạng chỉ số bên cạnh mỗi nút

Bảng 2.3: Bảng kết quả của ví dụ ở hình 2.3

Nút được mở rộng	Tập biên O (hàng đợi FIFO trong trường hợp này)
	S
S	AS , BS , CS , ES
AS	BS , CS , ES, DA
BS	CS , ES, DA, FB
CS	ES , DA, FB , HC
ES	DA , FB, HC , GE
DA	F _B , H _C , G _E
FB	HC , GE
H _C	G _E
GE	Đích

Sau đó, sử dụng con trỏ ngược, theo đó E là nút cha của G, S là nút cha của E, ta tìm được đường đi $G \leftarrow E \leftarrow S$.

Tính chất của tìm theo chiều rộng:

Đối chiếu với các tiêu chuẩn ở trên, tìm kiếm theo chiều rộng có những tính chất sau:

- Thuật toán có tính đầy đủ, tức là nếu bài toán có lời giải, tìm kiếm theo chiều rộng đảm bảo tìm ra lời giải. Thật vậy, nếu lời giải nằm ở độ sâu hữu hạn d thì thuật toán sẽ đạt tới lời giải đó sau khi đã khảo sát hết các nút nông hơn, trừ khi yếu tố rẽ nhánh b là vô hạn. Tìm theo chiều rộng là tìm kiếm vét cạn, trong đó các nút có độ sâu nhỏ hơn được xem xét trước.

- Tính tối ưu: thuật toán đảm bảo tìm ra lời giải có độ sâu nhỏ nhất. Tuy nhiên, trong trường hợp giá thành đường đi giữa các nút không bằng nhau thì điều này chưa đảm bảo tìm ra đường đi ngắn nhất. Cụ thể, trong ví dụ trên, thuật toán tìm ra đường đi SEG có độ sâu bằng 2 (nhỏ nhất) nhưng có độ dài 154, trong khi đường đi ngắn nhất SBFG có độ dài 132.
- Độ phức tạp tính toán: với mức độ rẽ nhánh là b và độ sâu lời giải d , thuật toán sinh ra $O(b^{d+1})$ nút trước khi tìm ra lời giải hay $O(b^d)$ nút nếu kiểm tra đích trước khi thêm nút vào tập biên. Độ phức tạp này là lớn và tăng rất nhanh khi b và d tăng.

Giả sử rằng, mỗi trạng thái khi được phát triển sẽ sinh ra b trạng thái kế. Như vậy từ nút gốc sẽ sinh ra b nút với độ sâu 1, các nút này lại sinh ra b^2 nút với độ sâu 2, và tiếp tục như vậy. Giả sử nút đích của bài toán nằm ở độ sâu d . Trong trường hợp xấu nhất, nút đích nằm cuối cùng trong số các nút ở độ sâu này và do vậy ta cần mở rộng tất cả nút ở độ sâu d trước khi tìm ra đích, tức là sinh ra b^{d+1} nút ở độ sâu $d+1$. Như vậy, tổng số nút cần mở rộng để tìm ra nút đích là (tính cả nút gốc):

$$1 + b + b^2 + \dots + b^{d+1} = O(b^{d+1})$$

Nếu tiến hành kiểm tra điều kiện đích trước khi thêm vào tập biên như đề cập ở trên, ta sẽ không phải sinh ra các nút ở độ sâu $d+1$ và do vậy số nút cần sinh ra chỉ còn là $O(b^d)$.

Yêu cầu bộ nhớ: thuật toán cần lưu $O(b^{d+1})$ nút trong tập biên sau khi đã mở rộng tất cả các nút ở độ sâu d . Nếu sử dụng tập các nút đóng thì tập này cần lưu $O(b^d)$ nút. Như vậy độ phức tạp bộ nhớ của tìm kiếm rộng là $O(b^{d+1})$.

Như vậy, ưu điểm của tìm theo chiều rộng là tính đầy đủ và tối ưu nếu giá thành đường đi như nhau. Nhược điểm của thuật toán này là độ phức tạp tính toán lớn và yêu cầu về bộ nhớ lớn. Trong hai nhược điểm sau, độ phức tạp về bộ nhớ lớn là nghiêm trọng hơn do không thể kiếm được máy tính có bộ nhớ đủ lớn để chạy thuật toán, trong khi ta có thể đợi thêm thời gian để chờ thuật toán chạy xong nếu thời gian chạy không quá lâu. Trên thực tế, thuật toán tìm theo chiều rộng chỉ có thể sử dụng cho các bài toán có kích thước rất nhỏ (b và d không quá 10).

2.7. Thuật toán DFS

Thuật toán tìm kiếm mù được biết đến nhiều tiếp theo là *tìm theo chiều sâu* (DepthFirst Search, viết tắt à **DFS**).

Nguyên tắc của tìm kiếm theo chiều sâu là lựa chọn trong số những nút biên nút sâu nhất (xa nút gốc nhất) để mở rộng trước. Như vậy, thay thì khảo sát tất cả các nhánh của cây tìm kiếm một lúc như trong tìm theo chiều rộng, thuật toán tìm sâu sẽ di chuyển theo một nhánh trong cây tìm kiếm cho tới nút sâu nhất, tức là nút không có nút con, trước khi chuyển sang nhánh tiếp theo.

Để thực hiện nguyên tắc trên, ta cần lựa chọn nút được thêm vào sau cùng trong tập nút biên O để mở rộng. Điều này có thể thực hiện dễ dàng bằng cách dùng một ngăn xếp để lưu các nút biên, các nút được thêm vào và lấy ra theo nguyên lý LIFO (vào sau ra trước). Thuật toán tìm theo chiều sâu cũng có thể thực hiện bằng cách đệ quy và quay lui. Tuy nhiên, trong tài liệu này, ta không sẽ không xem xét phương án sử dụng đệ quy và quay lui.

Thuật toán tìm kiếm theo chiều sâu được thể hiện trên hình 2.10, trong đó tập biên O được triển khai dưới dạng ngăn xếp LIFO. Các nút mới sinh ra được thêm vào đầu ngăn xếp O ở bước 3 của v ng lặp chính. Nút để mở rộng cũng được lấy ra từ đầu của O ở bước 1 của v ng lặp chính. Tương tự tìm theo chiều rộng, mỗi nút cần có con trở ngược về nút cha. Con trở ngược được sử dụng để khôi phục lại đường đi khi thuật toán đã tìm ra nút đích. Thuật toán kết thúc và trả về kết quả tại bước 2 của v ng lặp nếu nút lấy ra khỏi O là nút đích; hoặc thuật toán kết thúc (bằng lệnh return ở dưới cùng) mà không tìm được kết quả nếu tập O rỗng.

DFS(Q, S, G, P)

Đầu vào: bài toán tìm kiếm

Đầu ra: (đường đi tới) trạng thái đích C

Khởi tạo: $O \leftarrow S$ // O được tổ chức như ngăn xếp LIFO

While($O \neq \emptyset$) do

Chọn nút n đầu tiên của O và xóa n khỏi O

If $n \in G$, return (đường đi tới n)

Thêm P(n) vào đầu O

Return: Không có lời giải

Hình 2.16: Thuật toán tìm kiếm theo chiều sâu **Tránh các nút lặp:**

Thuật toán trên hình 2.4 có thể dẫn tới v ng lặp. Ví dụ, trong bài toán tìm đường trên bản đồ, thuật toán có thể quay về vị trí trước đó và lặp đi lặp lại chuyển động giữa hai trạng thái liên nhau. Khác với tìm theo chiều rộng, tìm theo chiều sâu sẽ lặp vô hạn mà không tìm được lời giải.

Để tránh v ng lặp khi tìm kiếm theo chiều sâu, có thể sử dụng một trong hai phương pháp. *Phương pháp thứ nhất* giống như phương pháp sử dụng trong thuật toán Graph_Search, tức là duy trì danh sách nút đóng và nhớ tất cả các nút đã mở rộng vào đây. Một nút mới sinh ra chỉ được thêm vào nút biên nếu nó chưa xuất hiện trong tập đóng và tập biên. Phương pháp này đ i hỏi nhiều bộ nhớ để duy trì tập nút đóng.

Phương pháp thứ hai kiểm tra xem nút mới có thuộc đường đi từ gốc tới nút hiện thời không, nếu có, nút mới sẽ không được thêm vào tập nút biên. Cách này không đ i hỏi bộ nhớ để lưu tập đóng, tuy nhiên chỉ cho phép tránh v ng lặp vô hạn và không cho phép giảm khối lượng tính toán trong trường hợp nhiều nhánh của đồ thị tìm kiếm có những phần trùng nhau.

Ví dụ: kết quả sau các bước lặp của thuật toán tìm theo chiều sâu cho bài toán tìm đường từ S tới G trên đồ thị hình 2.3 được thể hiện dưới đây dưới dạng các nút được mở rộng

và nội dung tập biên O sau mỗi v ng l p. Cũng như trong các ví dụ trước, nếu hai nút cùng độ sâu thì nút đứng trước trong bảng chữ cái được mở rộng trước. Việc tránh v ng l p được thực hiện theo phương pháp thứ hai. Đỉnh ngăn xếp nằm phía trái và đáy ngăn xếp nằm bên phải trong mỗi d ng.

Bảng 2.4: Bảng kết quả thuật toán DFS cho hình 2.3

Nút được mở rộng	Tập biên O (ngăn xếp LIFO trong trường hợp này)
	S
S	AS , BS , CS , ES
AS	DA , BS , CS , ES
DA	ED, BS , CS , ES
ED	BS , CS , ES
GE	Đích

Theo con trỏ ngược, ta tìm được đường đi $G \leftarrow E \leftarrow D \leftarrow A \leftarrow S$ với độ sâu bằng 4 và độ dài 227.

Tính chất thuật toán tìm theo chiều sâu:

- Nếu không gian trạng thái là hữu hạn thì thuật toán là đầy đủ. Ngược lại, nếu không gian trạng thái là vô hạn thì thuật toán là không đầy đủ do có thể di chuyển theo một đường đi không chứa nút đích và có độ sâu vô hạn (cứ đi theo nhánh không đúng mãi mà không chuyển sang nhánh khác được).
- Thuật toán không tối ưu: thuật toán có thể mở rộng những nhánh dẫn tới lời giải không tối ưu trước, đặc biệt trong trường hợp có nhiều lời giải.

- Độ phức tạp của thuật toán ở trường hợp xấu nhất là $O(b^m)$ với m là độ sâu tối đa của cây tìm kiếm. Trong trường hợp không gian trạng thái là vô hạn thì m là vô hạn. Trên thực tế DFS tìm ra lời giải nhanh hơn BFS, đặc biệt nếu tồn tại nhiều lời giải.
- Bộ nhớ cần nhớ tối đa $b \cdot m$ (mỗi mức chỉ nhớ b nút, với tối đa m mức), như vậy độ phức tạp về bộ nhớ của thuật toán này chỉ là $O(bm)$. Để đánh giá độ phức tạp không gian của tìm kiếm theo độ sâu ta có nhận xét rằng, khi ta phát triển một đỉnh u trên cây tìm kiếm theo độ sâu, ta chỉ cần lưu các đỉnh chưa được phát triển mà chúng là các đỉnh con của các đỉnh nằm trên đường đi từ gốc tới đỉnh u . Như vậy đối với cây tìm kiếm có nhân tố nhánh b và độ sâu lớn nhất là m , ta chỉ cần lưu ít hơn $b \cdot m$ đỉnh. Ưu cầu bộ nhớ so với tìm theo chiều rộng là ưu điểm nổi bật nhất của tìm theo chiều sâu.

2.8. Thuật toán A*

Một trong những nhược điểm của tìm kiếm tham lam là không cho lời giải tối ưu, tức là lời giải với đường đi ngắn nhất. Lý do tìm kiếm tham lam không đảm bảo tìm ra đường đi ngắn nhất là do thuật toán chỉ quan tâm tới khoảng cách ước lượng từ một nút tới đích mà không quan tâm tới quãng đường đã đi từ nút xuất phát tới nút đó. Trong trường hợp khoảng cách từ nút xuất phát tới nút đang xét lớn sẽ làm tổng độ dài đường đi từ xuất phát tới đích qua nút hiện thời lớn lên.

Để khắc phục nhược điểm này, thuật toán A* sử dụng hàm đánh giá $f(n)$ với hai thành phần, thành phần thứ nhất là đường đi từ nút đang xét tới nút xuất phát, thành phần thứ hai là khoảng cách ước lượng tới đích.

Phương pháp: khắc phục nhược điểm của thuật toán tham lam, thuật toán A* sẽ sử dụng hàm $f(n) = g(n) + h(n)$. Trong đó:

$g(n)$ là giá thành đường đi từ nút xuất phát đến nút n

$h(n)$ là giá thành ước lượng đường đi từ nút n đến nút đích; $h(n)$ là hàm heuristic.

Trong phần tìm kiếm mù, ta đã xem xét thuật toán tìm kiếm với giá thành thống nhất, trong đó nút biên có hàm $g(n)$ nhỏ nhất được chọn để mở rộng. Như vậy, thuật toán A* khác

tìm kiếm với giá thành thống nhất ở chỗ sử dụng thêm hàm $h(n)$ để ước lượng khoảng cách tới đích.

Thuật toán A^* yêu cầu hàm $h(n)$ là hàm *chấp nhận được* (admissible) theo định nghĩa sau.

Định nghĩa: Hàm $h(n)$ được gọi là chấp nhận được nếu $h(n)$ không lớn hơn độ dài đường đi thực ngắn nhất từ n tới nút đích.

Thuật toán A^* được thể hiện trên hình 2.5.

Thuật toán: $A^*(Q, S, G, P, c, h)$

Đầu vào: bài toán tìm kiếm, hàm heuristic h

Đầu ra: đường đi ngắn nhất từ nút xuất phát đến nút đích

Khởi tạo: tập các nút biên (nút mở) $O \leftarrow S$

While(O không rỗng) do

Lấy nút n có $f(n)$ nhỏ nhất ra khỏi O

Nếu n thuộc G , return(đường đi tới n)

Với mọi $m \in P(n)$

$g(m) = g(n) + c(m, n)$

$f(m) = g(m) + h(m)$

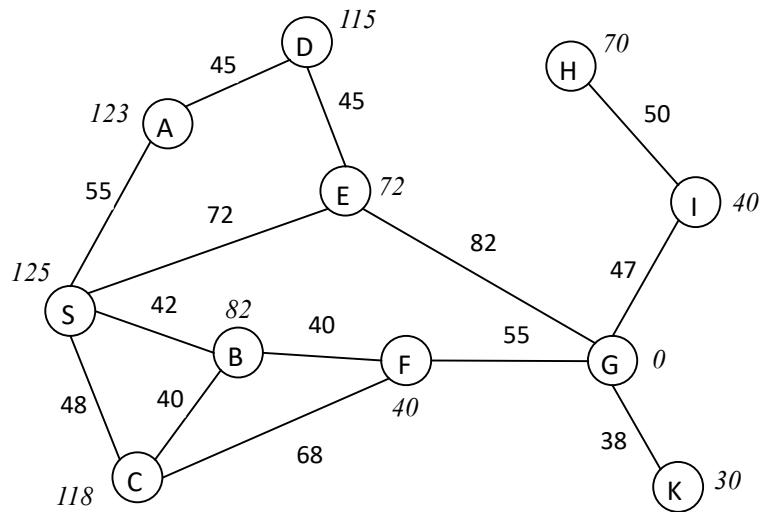
Thêm m vào O cùng giá trị $f(m)$

Return: không tìm được đường đi

Hình 2.17: Thuật toán A^*

Với thuật toán A^* , có thể sử dụng phiên bản Graph-search (tức là có danh sách nút đóng) hoặc không. Trong trường hợp không sử dụng nút đóng, tất cả các nút mới sinh ra đều được thêm vào tập biên. Trong trường hợp sử dụng phiên bản Graph-search, một nút đã mở

rộng sẽ không được thêm vào danh sách biên. Nếu một nút đã có một bản sao trong tập biên thì bản sao với giá trị hàm $f(n)$ nhỏ hơn sẽ được giữ lại.



Hình 2.18: Ví dụ tìm đường đi trên đồ thị từ S tới G

Ví dụ. Kết quả các bước thực hiện thuật toán A* cho bài toán tìm đường từ S tới G trên đồ thị hình 2.6 được thể hiện dưới đây. Các thông tin bao gồm nút được mở rộng, danh sách các nút trong tập biên cùng với giá trị hàm $f(n)$. Phiên bản được trình bày ở đây là phiên bản không sử dụng danh sách nút đóng và không kiểm tra tập biên. Như vậy, tập biên có thể chứa nhiều phiên bản của cùng một nút với giá trị hàm $f(n)$ khác nhau.

Bảng 2.5: Kết uqr của thuật toán A* cho hình 2.6

Nút được mở rộng	Tập biên O. Giá trị hàm $f(n)$ được cho trong ngoặc
	S (125)
S	$A_S (123+55)$, $B_S (82+42)$, $C_S (118+48)$, $E_S (72+72)$
BS	$A_S (123+55)$, $C_S (118+48)$, $E_S (72+72)$, $S_B (125+84)$, $C_B (118+82)$, $F_B (40+82)$

FB	$A_S (123+55), C_S (118+48), E_S (72+72), S_B (125+84),$
	$C_B (118+82), G_F (0+137), C_F (118+150)$
GF	đích

Sử dụng con trỏ ngược, ta có đường đi tìm được là $G \leftarrow F \leftarrow B \leftarrow S$ với độ dài 137. Đây cũng là đường đi ngắn nhất giữa S và G.

Đặc điểm của thuật toán A^* :

- Thuật toán cho *kết quả tối ưu* nếu hàm heuristic h là hàm *chấp nhận được*.
- *Thuật toán đầy đủ*, trừ trường hợp có vô số các nút với hàm f có giá trị rất nhỏ nằm giữa node xuất phát và node đích.
- Độ phức tạp: trong trường hợp xấu nhất, khi hàm heuristic không có nhiều thông tin, độ phức tạp tính toán và yêu cầu bộ nhớ của A^* đều là $O(b^m)$.
- Trong tất cả các thuật toán tìm kiếm tối ưu sử dụng cùng hàm heuristics thì thuật toán A^* có độ phức tạp tính toán nhỏ nhất, tức là yêu cầu sinh ra ít nút nhất trước khi tìm ra lời giải.
- Yêu cầu bộ nhớ lớn là nhược điểm lớn nhất của A^* . Do nhược điểm này, A^* thường không thể sử dụng để giải các bài toán có kích thước lớn. Trong phần sau, ta sẽ xem xét một thuật toán với yêu cầu bộ nhớ nhỏ, trong khi vẫn cho phép tìm ra lời giải tối ưu và có độ phức tạp tính toán gần tốt như A^* .

2.9. Hàm heuristic trong A^*

Trong tìm kiếm có thông tin, các hàm heuristic đóng vai trò rất quan trọng. Hàm heuristic tốt đảm bảo tính tối ưu cho những thuật toán như A^* . Bên cạnh đó, hàm heuristic tốt cho phép hướng thuật toán theo phía lời giải, nhờ vậy giảm số trạng thái cần khảo sát và độ phức tạp của thuật toán.

Các hàm heuristic $h(n)$ được xây dựng tùy thuộc vào bài toán cụ thể. Cùng một loại bài toán chúng ta có thể có rất nhiều hàm heuristic khác nhau. Chất lượng hàm heuristic ảnh hưởng rất nhiều đến quá trình tìm kiếm.

Hàm heuristic $h(n)$ được gọi là chấp nhận được khi:

$$h(n) \leq h^*(n)$$

trong đó $h^*(n)$ là giá thành đường đi thực tế từ n đến node đích. Lưu ý rằng hàm $h(n)=0$ với mọi n , là hàm chấp nhận được. Để minh họa cho hàm heuristic, ta xét một số ví dụ sau.

Ví dụ:

Trong bài toán tìm đường, đường chim bay như nhắc tới ở trên là một ví dụ của hàm heuristic chấp nhận được.

Trong bài toán 8 ô, có thể xây dựng một số hàm heuristic. Dưới đây, ta sẽ xem xét hai trong số các hàm như vậy.

Ta có thể sử dụng hai hàm heuristic sau.

- $h_{n1}()$: số ô đặt sai chỗ. Chẳng hạn nếu hình bên phải là trạng thái đích và hình bên trái là trạng thái u thì trạng thái bên trái có $h_1(u) = 5$ do có 5 ô là các ô 3, 6, 4, 5, 2 nằm sai vị trí. Có thể nhận thấy h_1 là hàm chấp nhận được do muốn di chuyển từ trạng thái bên trái sang trạng thái đích ta phải chuyển vị trí tất cả những ô đứng sai, trong khi để di chuyển mỗi ô sai, ta cần ít nhất một nước đi. Như vậy độ dài đường đi luôn lớn hơn hoặc bằng h_1 .
- $h_{n2}()$: tổng khoảng cách Manhattan giữa vị trí hiện thời của mỗi ô tới vị trí đúng của ô đó. Khoảng cách Manhattan được tính bằng số ít nhất các dịch chuyển theo hàng hoặc cột để đưa một quân tới vị trí của nó trong trạng thái đích. Ví dụ, để đưa quân 2 tới vị trí đích ta cần 4 dịch chuyển và do vậy khoảng cách Manhattan của 2 tới đích là 4. Giá trị h_2 của trạng thái u trên hình bên trái sẽ bằng $h_2(u) = 1 + 4 + 1 + 2 + 1$. Hàm h_2 cũng là hàm chấp nhận được. Thật vậy, để di chuyển một ô tới vị trí đích, ta cần ít nhất số nước đi bằng khoảng cách Manhattan từ ô đó tới đích. Như

vậy, số nước để di chuyển toàn bộ các ô đứng sai sẽ lớn hơn hoặc bằng tổng khoảng cách Manhattan như cách tính h_2 .

Hàm heuristic trội

- Ví dụ trên cho thấy, với cùng một bài toán, ta có thể xây dựng đồng thời nhiều hàm heuristic chấp nhận được. Vấn đề đặt ra khi đó là nên dùng hàm nào trong thuật toán tìm kiếm, hàm được chọn phải là hàm tốt hơn, tức là hàm nhanh dẫn tới kết quả hơn.
- Giả sử có hai hàm heuristic chấp nhận được $h_1(n)$ và $h_2(n)$. Nếu $h_1(n) \leq h_2(n)$ với mọi n thì ta nói rằng $h_2(n)$ trội hơn so với $h_1(n)$. Rõ ràng hàm $h_2(n)$ mang nhiều thông tin hơn và do vậy là hàm tốt hơn do dẫn tới kết quả nhanh hơn.
- Trong trường hợp trong hai hàm $h_1(n)$ và $h_2(n)$ không có hàm trội hơn thì ta có thể tạo ra hàm $h'(n) = \max(h_1(n), h_2(n))$ với mọi n . Rõ ràng, $h'(n)$ là hàm trội hơn hai hàm ban đầu.

Cách xây dựng hàm heuristic

- Việc lựa chọn hàm heuristic phụ thuộc vào bài toán cụ thể. Nguyên tắc chung để xây dựng hàm heuristic cho một bài toán là nói lỏng các ràng buộc của bài toán đó khi ước lượng đường đi tới đích. Ví dụ, đối với hàm $h_1(n)$ trong tr 8 ô, ta đã bỏ ràng buộc về việc các ô phải di chuyển từng bước để đến được vị trí đích. Hay đối với heuristic đường chim bay, ta đã nói lỏng ràng buộc về việc phải di chuyển trên đường bộ (thường không phải là đường thẳng) và giả sử có thể di chuyển thẳng từ một điểm tới đích.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Mô tả bài toán

Trong dòng chảy phát triển của Trí tuệ nhân tạo, bài toán tìm đường (Pathfinding Problem) luôn giữ vị trí như một viên đá tảng kinh điển. Từ những bài toán mê cung cổ xưa trong thần thoại Hy Lạp cho đến các bài toán định tuyến trong kỷ nguyên số, bản chất cốt lõi của bài toán vẫn không hề thay đổi: đó là hành trình tìm kiếm một chuỗi hành động tối ưu để đưa một thực thể từ trạng thái xuất phát vượt qua các rào cản địa hình để cập bến đích thành công.

Trong phạm vi đề tài, mê cung được mô hình hóa thành một thể giới lưới hai chiều phẳng rời rạc kích thước $M \times N$ ô vuông. Tại đây, mỗi ô đóng vai trò là một điểm nút trong không gian tọa độ. Một thực thể thám hiểm (đại diện cho tác tử AI, robot tự hành hay nhân vật trò chơi) khởi đầu tại tọa độ $S(r_s, c_s)$ và mang trong mình sứ mệnh tìm ra con đường ngắn nhất, an toàn nhất để chạm tới điểm đích $G(r_g, c_g)$. Bao bọc xung quanh và rải rác khắp không gian là những bức tường đá kiên cố - những vùng không gian bị cô lập mà tác tử tuyệt đối không thể bước qua.

Đầu vào (Input) của hệ thống bao gồm một ma trận lưới với kích thước xác định, cặp tọa độ của điểm xuất phát và điểm đích, cùng tập hợp vị trí của các bức tường vật cản. Đầu ra (Output) đòi hỏi thuật toán phải trích xuất được một lộ trình hoàn chỉnh nối liền từ xuất phát đến đích thỏa mãn các quy tắc di chuyển không xuyên tường, hoặc phải đưa ra kết luận chính xác rằng không tồn tại đường đi (vô nghiệm) nếu đích bị phong tỏa hoàn toàn. Đồng thời, hệ thống ghi nhận các thông số vận hành thực tế như tổng thời gian xử lý, số lượng nút không gian đã được khai phá, độ dài đường đi tìm được và dung lượng bộ nhớ hàng đợi tối đa.

Ý nghĩa thực tiễn của bài toán không dừng lại ở góc độ lý thuyết hàn lâm hay một trò chơi giải đố thông thường. Đây chính là thuật toán nền tảng đang vận hành âm thầm đằng sau các ứng dụng bản đồ định vị vệ tinh GPS toàn cầu như Google Maps, các hệ thống điều phối đội xe robot tự hành (AGV - Automated Guided Vehicle) vận chuyển hàng triệu kiện hàng mỗi ngày trong các kho thông minh của Amazon, cũng như hệ thống trí tuệ nhân tạo điều hướng nhân vật (NPC Navigation Mesh) trong ngành công nghiệp trò chơi điện tử hiện đại.

3.2. Yêu cầu chức năng

Để đáp ứng mục tiêu nghiên cứu và tạo ra một môi trường thực nghiệm khách quan, trực quan nhất, hệ thống phần mềm mô phỏng được thiết kế với đầy đủ các phân hệ chức năng tương tác và xử lý thuật toán song song:

Thứ nhất, chức năng khởi tạo và quản lý địa hình mê cung: Phần mềm cho phép người dùng tùy biến kích thước bản đồ linh hoạt (từ lưới nhỏ 15×15 , kích thước chuẩn 21×21 , đến các không gian lớn 31×31 và 41×41). Hệ thống tích hợp sẵn thư viện các mẫu địa hình đa dạng bao gồm Mê cung hoàn hảo (Perfect Maze sinh bởi DFS), Mê cung phân nhánh tự nhiên (Prim's Algorithm), Mê cung xoắn ốc (Spiral Maze), Hệ thống phòng và hành lang (Rooms & Corridors), Bãi vật cản ngẫu nhiên (Open Obstacle Field) và Bản đồ bày không lối thoát (No-Path Map).

Thứ hai, chức năng tương tác bản đồ trực tiếp: Người dùng có thể sử dụng chuột để trực tiếp vẽ thêm các bức tường chướng ngại vật hoặc xóa bỏ tường hiện có theo thời gian thực. Đặc biệt, người dùng có quyền kéo thả tự do điểm xuất phát (Start) và điểm đích (Goal) đến bất kỳ vị trí hợp lệ nào trên bàn cờ, giúp thử nghiệm linh hoạt vô số kịch bản tìm đường khác nhau.

Thứ ba, chức năng điều phối chạy song song ba thuật toán: Đây là chức năng cốt lõi tạo nên sự đột phá của chương trình. Thay vì chỉ chạy đơn lẻ từng thuật toán một cách rời rạc, hệ thống cho phép kích hoạt đồng thời cả BFS (Chiều rộng), DFS (Chiều sâu) và A* (Heuristic Manhattan) trên cùng một địa hình với cùng trạng thái xuất phát và đích. Nhờ đó, người quan sát có thể lập tức đối sánh sự khác biệt về đáng điều duyệt và tốc độ lan truyền của từng giải thuật.

Thứ tư, chức năng điều khiển mô phỏng linh hoạt: Cung cấp đầy đủ các chế độ 'Chạy song song' liên tục, 'Duyệt từng bước' (Step-by-step) để mô tả chi tiết từng hành vi mở rộng nút, 'Giải tức thì' (Instant Solve) nhằm đo đạc thời gian CPU thuần túy loại trừ độ trễ hiển thị đồ họa, và nút 'Đặt lại' (Reset) để hoàn nguyên trạng thái ban đầu.

Thứ năm, chức năng phân tích số liệu và đồ thị hiệu năng: Tự động tổng hợp dữ liệu sau mỗi lượt chạy thành bảng chỉ số KPI rõ ràng và trực quan hóa tức thì thông qua ba biểu đồ cột

tương tác (Số ô đã duyệt, Độ dài đường đi, và Thời gian thực thi), giúp việc đánh giá so sánh trở nên minh bạch và thuyết phục.

3.3. Yêu cầu phi chức năng

Bên cạnh các yêu cầu về mặt tính năng, hệ thống đặt ra các tiêu chuẩn khắt khe về mặt kỹ thuật công nghệ nhằm bảo đảm tính chính xác khoa học và trải nghiệm mượt mà cho người dùng:

Về độ chính xác toán học: Thuật toán BFS và A* được ràng buộc phải đảm bảo 100% tính tối ưu tuyệt đối về chiều dài đường đi trong không gian bước đi đồng nhất. Mọi sai số logic hoặc việc tìm ra đường đi không phải ngắn nhất đều bị coi là lỗi nghiêm trọng. Đối với bài toán vô nghiệm, thuật toán phải duyệt hết miền liên thông và kết thúc an toàn, thông báo rõ ràng mà không rơi vào vòng lặp vô hạn.

Về hiệu năng thực thi và độ mượt hiển thị: Ứng dụng phải duy trì tốc độ khung hình ổn định ở mức 60 khung hình/giây (FPS) xuyên suốt quá trình hoạt hình hóa đồ họa ba canvas. Đối với chế độ giải tức thì, thuật toán phải hoàn thành việc tìm kiếm trong thời gian dưới 15 mili-giây trên lưới kích thước trung bình 21×21 .

Về tính ổn định và kiểm soát tài nguyên: Triệt tiêu hoàn toàn nguy cơ tràn ngăn xếp lời gọi hàm (Call Stack Overflow) bằng cách cài đặt DFS thông qua ngăn xếp dữ liệu tường minh thay vì hàm đệ quy. Thuật toán A* phải sử dụng Hàng đợi ưu tiên Min-Heap nhị phân để thao tác trích xuất phần tử cực tiểu chỉ tốn độ phức tạp $O(\log N)$.

Về tính trực quan và thẩm mỹ giao diện: Áp dụng bảng màu tương phản cao, phong cách thiết kế phẳng hiện đại chuẩn học thuật. Mỗi thuật toán được gán một tông màu chủ đạo riêng biệt (BFS sử dụng sắc Xanh dương lam biểu trưng cho sóng nước lan tỏa; DFS mang sắc Vàng cam hồ phách đại diện cho ngọn lửa đâm sâu; A* khoác sắc Xanh lục bảo tượng trưng cho trí tuệ định hướng). Dải đường đi tìm được phải nổi bật rực rỡ để người xem dễ dàng phân định.

3.4. Quy tắc hoạt động của mê cung

Thế giới mê cung trong đề tài vận hành dựa trên một hệ thống quy tắc toán học chặt chẽ và nhất quán, đóng vai trò làm chuẩn đo lường công bằng cho mọi thuật toán tham gia giải bài toán:

Quy tắc không gian lưới: Mê cung được định nghĩa là một lưới ô vuông 2 chiều rời rạc gồm R hàng (chỉ số từ 0 đến R-1) và C cột (chỉ số từ 0 đến C-1). Toàn bộ ranh giới ngoài cùng của lưới luôn được bao bọc bởi một lớp tường biên vững chắc nhằm giới hạn không gian thám hiểm trong miền hữu hạn.

Quy tắc di chuyển 4 hướng lân cận: Tại bất kỳ ô hợp lệ $u(r, c)$ nào, tác tử chỉ được phép thực hiện 4 hành động di chuyển trực giao cơ bản: Lên trên $u_up(r-1, c)$, Sang phải $u_right(r, c+1)$, Xuống dưới $u_down(r+1, c)$ và Sang trái $u_left(r, c-1)$. Hệ thống không hỗ trợ các bước đi chéo góc nhằm duy trì tính nguyên bản và độ chuẩn xác của không gian khoảng cách lưới Manhattan.

Quy tắc cản trở vật lý: Tác tử tuyệt đối không thể di chuyển vào ô có thuộc tính là tường (Wall). Ô tường đóng vai trò là chướng ngại vật vô hạn chi phí. Mọi hành động di chuyển ra ngoài biên lưới hoặc tiến vào ô tường đều bị hệ thống phát hiện và vô hiệu hóa.

Quy tắc chi phí bước đi: Chi phí di chuyển giữa hai ô thông suốt liền kề bất kỳ u và v luôn đồng nhất và bằng đúng 1 đơn vị chi phí $c(u, v) = 1$. Đây là tiền đề toán học để thuật toán BFS đảm bảo tìm ra đường đi tối ưu về số bước.

Quy tắc kết thúc: Quá trình tìm kiếm chấm dứt ngay khi thuật toán phát hiện trạng thái hiện tại trùng khớp với tọa độ điểm đích $G(r_g, c_g)$, chuyển sang giai đoạn truy vết đường đi. Ngược lại, nếu tập các trạng thái biên chờ duyệt bị vét cạn hoàn toàn mà vẫn chưa chạm tới đích, thuật toán công bố bài toán vô nghiệm và dừng an toàn.

3.5. Biểu diễn mê cung trong chương trình

Để chuyển đổi mô hình mê cung từ lý thuyết sang bộ nhớ máy tính một cách tinh gọn và tốc độ truy xuất cao nhất, nhóm nghiên cứu đã biểu diễn mê cung dưới dạng một mảng hai chiều (ma trận các số nguyên) được đóng gói bên trong lớp đối tượng Maze:

Trong ma trận `grid[r][c]`, mỗi phần tử nhận một giá trị hằng số quy ước: `CellType.EMPTY = 0` đại diện cho ô đường đi thông thoáng mà thực thể có thể bước vào; `CellType.WALL = 1` đại diện cho ô tường chướng ngại vật. Cách biểu diễn này cho phép phép kiểm tra tính hợp lệ `isWalkable(r, c)` được thực thi trong thời gian tức thì $O(1)$ chỉ bằng một phép truy xuất chỉ số mảng đơn giản:

```
// Định nghĩa trạng thái ô và phương thức kiểm tra hợp lệ
export const CellType = { EMPTY: 0, WALL: 1 };

export class Maze {
  isWalkable(r, c) {
    return r >= 0 && r < this.rows &&
      c >= 0 && c < this.cols &&
      this.grid[r][c] !== CellType.WALL;
  }
}
```

Tọa độ của điểm xuất phát và đích đến được quản lý bởi hai đối tượng cấu trúc `start = { r, c }` và `goal = { r, c }`. Cơ chế bảo vệ tọa độ đảm bảo rằng dù người dùng có tạo địa hình ngẫu nhiên phức tạp tới đâu, hai ô xuất phát và đích luôn tự động được gán về trạng thái `CellType.EMPTY` để duy trì tính hợp lệ của bài toán tìm kiếm.

3.6. Thiết kế cấu trúc dữ liệu

Cấu trúc dữ liệu đóng vai trò trái tim quyết định toàn bộ hiệu năng và tính chất hoạt động của từng thuật toán. Trong hệ thống, nhóm đã thiết kế một hệ thống cấu trúc dữ liệu hướng đối tượng phân tầng chặt chẽ:

Lớp trừu tượng cơ sở `PathfindingAlgorithm`: Đây là lớp cha chứa đựng các tài nguyên dùng chung cho cả ba thuật toán. Bao gồm: bảng băm `visited` (Map) lưu thứ tự duyệt của từng ô, mảng `visitedList` lưu trữ danh sách các nút theo đúng trình tự thời gian khám phá phục vụ vẽ hoạt hình, bảng ánh xạ `parent` (Map) kết nối mỗi nút con với nút cha đã sinh ra nó, và mảng `path` lưu trữ tọa độ chuỗi đường đi từ xuất phát đến đích.

Hàng đợi FIFO Queue cho BFS: Thuật toán BFS sử dụng một mảng thuần Javascript mô phỏng hàng đợi vào trước - ra trước (First-In, First-Out). Nút mới được đưa vào đuôi hàng đợi bằng phương thức `push()` và nút cũ nhất ở đầu hàng đợi được lấy ra xử lý bằng `shift()`. Cấu trúc này bảo đảm các nút ở độ sâu `k` luôn được duyệt hết trước khi bất kỳ nút nào ở độ sâu `k+1` được khám phá.

Ngăn xếp LIFO Stack cho DFS: Để loại bỏ nguy cơ tràn bộ nhớ ngăn xếp cuộc gọi hàm khi chạy đệ quy trên bản đồ lớn, DFS sử dụng cấu trúc ngăn xếp vào sau - ra trước (Last-In, First-Out) thông qua mảng động. Nút mới đẩy vào đỉnh bằng `push()` và lấy ra từ đỉnh bằng `pop()`. Nhờ đó, thuật toán luôn ưu tiên đào sâu tối đa theo hướng đi vừa tìm thấy.

Hàng đợi ưu tiên Min-Heap nhị phân cho A*: Thuật toán A* đòi hỏi tại mỗi bước phải lấy ra nút có giá trị hàm đánh giá $f(n)$ nhỏ nhất trong tập chờ mở. Nếu sử dụng mảng thông thường và gọi hàm sắp xếp `array.sort()`, độ phức tạp sẽ là $O(N \log N)$ cho mỗi bước duyệt, gây hiện tượng tụt giảm khung hình nghiêm trọng. Do đó, nhóm đã tự tay hiện thực cấu trúc MinHeap hoàn chỉnh dạng cây nhị phân lưu trữ trên mảng với hai cơ chế cân bằng cây cốt lõi là `_bubbleUp` (nổi lên trên) và `_sinkDown` (chìm xuống dưới), bảo đảm thao tác lấy phần tử ưu tiên chỉ tiêu tốn $O(\log N)$.

Bảng 3.6: So sánh cấu trúc dữ liệu nền tảng của ba thuật toán tìm đường

Thuật toán	Cấu trúc dữ liệu chính	Nguyên lý hoạt động	Độ phức tạp mở nút
BFS	Hàng đợi FIFO (Queue)	Vào trước ra trước, duyệt theo lớp	$O(1)$
DFS	Ngăn xếp LIFO (Stack)	Vào sau ra trước, đâm sâu tối đa	$O(1)$
A*	Hàng đợi ưu tiên (Min-Heap)	Ưu tiên nút có hàm $f(n)$ cực tiểu	$O(\log N)$

3.7. Thiết kế chức năng tìm đường

Điểm kiến trúc đột phá trong thiết kế của nhóm là cơ chế 'Bộ tạo từng bước' (Step-by-step Generator). Thay vì gom toàn bộ thuật toán vào một vòng lặp `while` chạy kín từ đầu đến

cuối làm đóng băng giao diện người dùng, mỗi thuật toán được cài đặt một phương thức `step()` độc lập. Mỗi lần `step()` được kích hoạt, thuật toán chỉ thực hiện đúng một chu kỳ mở rộng nút duy nhất, cập nhật trạng thái nội tại rồi nhường quyền điều khiển cho luồng vẽ giao diện:

Trong phương thức `step()`, thuật toán lấy nút hiện tại ra khỏi tập biên (Frontier), kiểm tra xem nút đó có trùng với Goal hay không. Nếu trùng, thuật toán chuyển trạng thái sang 'found' và kích hoạt phương thức truy vết `reconstructPath()`. Nếu chưa đến đích, thuật toán gọi `getNeighbors()` để sinh ra 4 ô lân cận hợp lệ, lọc bỏ các ô đã thăm hoặc đã đóng, gán nút hiện tại làm cha (parent) của các nút con rồi đẩy chúng vào tập biên.

Hàm đánh giá của A*: A* sử dụng hàm đánh giá $f(n) = g(n) + h(n)$, trong đó $g(n)$ là độ dài đường đi thực tế từ Start đến nút n được lưu trong bảng `gScore`, và $h(n)$ là hàm Heuristic ước lượng khoảng cách từ n đến Goal. Nhóm sử dụng hàm khoảng cách Manhattan tiêu chuẩn cho lưới 4 hướng: $h(n) = |r_n - r_g| + |c_n - c_g|$. Do không có bước đi chéo, khoảng cách Manhattan chính là số bước ngắn nhất tuyệt đối trên lý thuyết để đi từ n tới Goal, bảo đảm thỏa mãn tính chất chấp nhận được (Admissible: $h(n) \leq h^*(n)$). Nhờ đó, A* được bảo đảm toán học luôn tìm ra đường đi ngắn nhất tối ưu hết như BFS nhưng với số ô duyệt ít hơn rất nhiều.

Cơ chế giải quyết nút đồng giá trị (Tie-Breaking thông minh): Trong Min-Heap của A*, khi hai nút có cùng giá trị $f(n)$, nếu chọn ngẫu nhiên thuật toán sẽ dễ duyệt lan man sang hai bên. Nhóm đã tối ưu hóa điều kiện đổi chỗ trong heap: nếu f bằng nhau, ưu tiên nút có h nhỏ hơn. Việc ưu tiên nút gần đích hơn giúp chum tia tìm kiếm của A* co cụm lại, đâm thẳng một mạch về phía đích như một mũi tên sắc bén.

3.8. Thiết kế giao diện mô phỏng

Giao diện người dùng được thiết kế dựa trên triết lý công thái học và trực quan hóa dữ liệu khoa học. Bộ cục ứng dụng được phân chia thành ba tầng chức năng mạch lạc:

Tầng trên cùng là Thanh điều khiển trung tâm (Control Dock): Đóng vai trò là buồng lái điều phối toàn bộ hệ thống. Nơi đây quy tụ các nút bấm lệnh dạng khối tương tác cao (Chạy song song, Duyệt từng bước, Giải tức thì, Đặt lại), danh sách thả chọn các mẫu bản đồ địa

hình, lựa chọn kích thước lưới (15x15 đến 41x41), thanh trượt điều chỉnh tốc độ từ 1x đến 100x và tùy chọn hàm Heuristic.

Tầng ở giữa là Không gian mô phỏng ba màn hình Canvas song song: Đây là tâm điểm trực quan của phần mềm. Ba khung vẽ độc lập đại diện cho BFS, DFS và A* được xếp thẳng hàng ngang trên cùng một tầm nhìn. Phía trên mỗi khung vẽ có thẻ định danh thuật toán, kèm huy hiệu trạng thái động (Sẵn sàng, Đang chạy, Đã tìm thấy, Vô nghiệm). Dưới mỗi canvas là bảng vi mô hiển thị bốn chỉ số tức thời: Thời gian chạy (ms), Số ô đã duyệt, Độ dài đường đi và Bộ nhớ đỉnh.

Tầng dưới cùng là Phân hệ phân tích và đối sánh KPI: Cung cấp góc nhìn số liệu khoa học toàn cảnh thông qua bảng tổng hợp chi tiết và ba biểu đồ cột Chart.js động. Biểu đồ tự động nhảy số và vẽ cột so sánh ngay khi các thuật toán cán đích, giúp người dùng nắm bắt bản chất thuật toán qua trực giác thị giác.

3.9. Luồng hoạt động của chương trình

Vòng đời vận hành của phần mềm mô phỏng diễn ra tuần tự qua các giai đoạn logic nghiêm ngặt:

Giai đoạn 1 - Khởi tạo môi trường: Khi tải trang, hệ thống tạo mới đối tượng Maze, thiết lập Start và Goal, nạp mẫu địa hình mặc định. Sau đó khởi tạo ba thực thể thuật toán BFS, DFS, A* cùng ba đối tượng MazeVisualizer gắn với ba thẻ canvas HTML5.

Giai đoạn 2 - Lắng nghe tương tác: Người dùng có thể tự do chỉnh sửa tường cản, đổi vị trí Start/Goal hoặc chọn kiểu bản đồ mẫu từ thanh điều khiển. Bất kỳ thay đổi nào trên địa hình đều tự động đồng bộ hóa sang cả ba thuật toán và vẽ lại màn hình ngay lập tức.

Giai đoạn 3 - Điều phối thực thi: Khi người dùng nhấn nút 'Chạy song song', bộ điều phối SimulationController kích hoạt vòng lặp hoạt hình thông qua hàm requestAnimationFrame() của trình duyệt. Tại mỗi chu kỳ đồng hồ (frame), bộ điều phối căn cứ vào thanh trượt tốc độ để quyết định số bước tính toán (stepsPerFrame), sau đó gọi lần lượt phương thức step() của BFS, DFS và A*. Nhờ đó, cả ba thuật toán tiến bước hoàn toàn đồng bộ trên cùng một nhịp đập thời gian.

Giai đoạn 4 - Vẽ đồ họa và cập nhật số liệu: Sau mỗi nhịp bước đi, các visualizer tiến hành xóa và vẽ lại lớp ô đã thăm (Visited), lớp tập biên (Frontier) và vị trí con trỏ hiện tại (Current). Khi một thuật toán chạm đích, dải ruy-băng đường đi được vẽ nối từ Start tới Goal. Khi tất cả các thuật toán kết thúc hoặc rơi vào trạng thái vô nghiệm, vòng lặp dừng lại, kích hoạt hàm onComplete để chốt hạ số liệu lên bảng KPI và vẽ biểu đồ.

CHƯƠNG 4: XÂY DỰNG CHƯƠNG TRÌNH MÔ PHỎNG

4.1. Môi trường và công nghệ phát triển

Hệ thống mô phỏng được phát triển trên nền tảng công nghệ Web tiêu chuẩn hiện đại, loại bỏ hoàn toàn sự phụ thuộc vào các framework công kênh nhằm tối ưu hóa triệt để hiệu năng tính toán và khả năng tương thích đa nền tảng:

Ngôn ngữ lập trình cốt lõi: Sử dụng JavaScript hiện đại (ECMAScript 6+) theo mô hình Lập trình hướng đối tượng (OOP) và kiến trúc mô-đun (ES Modules). Mã nguồn được tổ chức tường minh, mạch lạc, tận dụng tối đa các cấu trúc dữ liệu hiệu năng cao của V8 Engine như Map, Set và Mảng nhị phân phẳng.

Đồ họa và trực quan hóa: Sử dụng trực tiếp HTML5 Canvas 2D Rendering Context. Canvas cung cấp năng lực dựng hình phần cứng cực nhanh, cho phép vẽ hàng nghìn ô vuông, đường lưới và dải sáng laser với độ mượt 60 FPS mà không gặp hiện tượng giật lag hay nghẽn DOM.

Đồ thị thống kê: Tích hợp thư viện biểu đồ mã nguồn mở hàng đầu Chart.js (phiên bản 4.4.1) thông qua CDN để dựng các biểu đồ cột động biểu diễn số ô duyệt, độ dài đường đi và thời gian thực thi.

Môi trường thực thi: Chương trình hoạt động hoàn toàn phía máy khách (Client-side) trên bất kỳ trình duyệt web tiêu chuẩn nào hỗ trợ HTML5 như Microsoft Edge, Google Chrome, Mozilla Firefox hay Safari, không yêu cầu cài đặt môi trường máy chủ phức tạp.

4.2. Cấu trúc chương trình

Mã nguồn của hệ thống được module hóa thành các tệp tin chuyên biệt đặt trong cấu trúc thư mục rõ ràng, tuân thủ nguyên tắc phân tách trách nhiệm (Separation of Concerns):

Tệp index.html: Xây dựng bộ khung giao diện ngữ nghĩa chuẩn HTML5, tích hợp thanh điều khiển Control Dock, ba khung thẻ hiển thị thuật toán song song và khu vực bảng biểu phân tích.

Tệp style.css: Định nghĩa toàn bộ hệ thống giao diện trực quan, bảng màu tùy biến (CSS Custom Properties), hiệu ứng nút bấm, bóng đổ mờ và bố cục lưới đáp ứng (Responsive Grid Layout).

Tệp js/maze.js: Chịu trách nhiệm toàn bộ logic về không gian bản đồ, quản lý tọa độ, ma trận lưới và tích hợp 6 thuật toán sinh địa hình tự động.

Tệp js/algorithms.js: Chứa bộ mã nguồn kinh điển của MinHeap, lớp cha PathfindingAlgorithm, ba lớp thuật toán BFSAlgorithm, DFSAlgorithm, AStarAlgorithm và bộ điều phối chạy song song SimulationController.

Tệp js/visualizer.js: Cung cấp công cụ MazeVisualizer quản lý vòng đời vẽ canvas, chuyển đổi tọa độ chuột, xử lý vẽ tường tương tác và hoạt hình hóa dải ruy-băng đường đi.

Tệp js/report.js: Cung cấp công cụ ReportGenerator có khả năng chụp canvas và kết xuất toàn bộ số liệu thành một bức ảnh đồ họa chất lượng cao Full HD 1920x1200 phục vụ chèn vào tài liệu.

Tệp js/app.js: Điểm vào chính của chương trình (Main Entry Point), đóng vai trò cầu nối gắn kết giao diện người dùng với bộ điều phối thuật toán và hệ thống biểu đồ.

4.3. Xây dựng mê cung

Một trong những nét đặc sắc của chương trình là hệ thống sinh mê cung tự động vô cùng phong phú, được xây dựng dựa trên các thuật toán lý thuyết đồ thị tiên tiến:

Thuật toán Recursive Backtracker (DFS Maze): Tạo ra các 'Mê cung hoàn hảo' (Perfect Maze) - loại mê cung có cấu trúc toán học tương đương một Cây khung (Spanning Tree). Trong mê cung này, giữa hai vị trí thông bất kỳ luôn tồn tại duy nhất một đường đi đơn độc nhất, không có đường vòng khép kín (chu trình). Thuật toán khởi đầu bằng cách lấp đầy toàn bộ bản đồ bằng tường đá. Sau đó, nó đào đường bằng cách nhảy 2 ô một lần theo hướng ngẫu nhiên, đục thông bức tường ở giữa và đẩy tọa độ vào ngăn xếp. Khi rơi vào ngõ cụt, nó quay lui (backtrack) để tìm nhánh mới cho đến khi ngăn xếp trống:

```
// Trích đoạn thuật toán Recursive Backtracker trong maze.js
while (stack.length > 0) {
```

```

const current = stack[stack.length - 1];
const neighbors = [];
for (const d of dirs) {
    const nr = current.r + d.dr, nc = current.c + d.dc;
    if (nr > 0 && nr < this.rows - 1 && nc > 0 && nc < this.cols - 1
        && this.grid[nr][nc] === CellType.WALL) {
        neighbors.push({ ...d, nr, nc });
    }
}
if (neighbors.length > 0) {
    const next = neighbors[Math.floor(Math.random() * neighbors.length)];
    this.grid[current.r + next.wallR][current.c + next.wallC] = CellType.EMPTY;
    this.grid[next.nr][next.nc] = CellType.EMPTY;
    stack.push({ r: next.nr, c: next.nc });
} else {
    stack.pop();
}
}

```

Thuật toán Prim's Algorithm: Tạo ra các mê cung phân nhánh tự nhiên với nhiều ngõ rẽ ngắn và dày đặc. Thuật toán hoạt động bằng cách mở rộng dần tập hợp các ô đã đào thông qua một danh sách các bức tường biên ngẫu nhiên.

Địa hình Mê cung xoắn ốc (Spiral Maze): Xây dựng các bức tường đồng tâm uốn lượn từ ngoài vào trong tâm bản đồ, tạo ra một bãi chướng ngại vật sâu không lồ để thách thức khả năng tìm đường của DFS.

Địa hình Bãi vật cản ngẫu nhiên (Open Obstacle Field): Tạo ra một không gian mở rộng lớn với các khối đá phân bố ngẫu nhiên (mật độ khoảng 28%). Đây là môi trường hoàn hảo để chứng minh sự vượt trội của hàm Heuristic trong thuật toán A* so với sự lan tỏa tốn kém của BFS.

Địa hình Bẫy không lối thoát (No-Path): Hệ thống tự động tạo tường bao bọc kín 4 phía xung quanh điểm Goal, kiểm chứng khả năng phát hiện vô nghiệm và dừng an toàn của các thuật toán.

4.4. Xây dựng điểm bắt đầu và điểm đích

Vị trí xuất phát (Start) và điểm đích (Goal) được xác định tại hai góc đối đỉnh của bản đồ nhằm tạo ra hành trình thám hiểm có khoảng cách lớn nhất. Mặc định, Start đặt tại góc trên bên trái (hàng 1, cột 1) và Goal đặt tại góc dưới bên phải (hàng rows-2, cột cols-2).

Trên giao diện đồ họa, điểm Start được tô vịnh bằng ký tự 'S' in đậm trong ô tròn màu xanh ngọc lục bảo phát sáng rực rỡ, tượng trưng cho điểm xuất phát của cuộc hành trình. Điểm Goal được đánh dấu bằng ký tự 'G' trong ô tròn màu đỏ thắm kiêu hãnh, tượng trưng cho đích đến vinh quang. Hệ thống tích hợp cơ chế lắng nghe sự kiện chuột mousedown và mousemove, cho phép người dùng nhấp giữ và kéo thả Start hoặc Goal lướt nhẹ trên bàn cờ đến bất kỳ vị trí thông thoáng nào.

4.5. Cài đặt thuật toán BFS

Thuật toán BFS được cài đặt bên trong lớp BFSAlgorithm kế thừa từ PathfindingAlgorithm. Cơ chế duyệt hoạt động bằng cách rút phần tử ở đầu hàng đợi FIFO, sau đó mở rộng sang 4 hướng lân cận:

```
// Cài đặt bước duyệt BFS trong algorithms.js
step() {
  if (this.queue.length === 0) {
    this.status = 'no_path';
    return this.status;
  }
  const current = this.queue.shift(); // Lấy nút đầu hàng đợi FIFO
  this.current = current;
  this.visitedCount++;

  if (current.r === this.goal.r && current.c === this.goal.c) {
```

```

    this.reconstructPath();
    this.status = 'found';
    return this.status;
}

const neighbors = this.getNeighbors(current.r, current.c);
for (const next of neighbors) {
    const nextKey = this.key(next.r, next.c);
    if (!this.visited.has(nextKey)) {
        this.visited.set(nextKey, this.visitedList.length);
        this.visitedList.push(next);
        this.parent.set(nextKey, this.key(current.r, current.c));
        this.queue.push(next);

        // Kiểm tra đích sớm ngay khi sinh nút để tối ưu bước duyệt
        if (next.r === this.goal.r && next.c === this.goal.c) {
            this.reconstructPath();
            this.status = 'found';
            return this.status;
        }
    }
}
}

```

Điểm nhấn tinh tế trong cài đặt BFS của nhóm là kỹ thuật 'Kiểm tra đích sớm' (Early Goal Check). Thay vì đợi đến khi nút được lấy ra khỏi hàng đợi mới kiểm tra đích, mã nguồn kiểm tra ngay tại thời điểm nút kẻ được sinh ra. Kỹ thuật này giúp tiết kiệm đáng kể số vòng lặp và đưa thuật toán cán đích nhanh hơn một chu kỳ biên.

4.6. Cài đặt thuật toán DFS

Trái ngược với BFS, DFS sử dụng ngăn xếp Stack dạng mảng với thao tác pop() để lấy nút ở đỉnh. Nhánh tìm kiếm sẽ được đẩy sâu liên tục theo một hướng duy nhất:

```
// Cài đặt bước duyệt DFS trong algorithms.js
step() {
  if (this.stack.length === 0) {
    this.status = 'no_path';
    return this.status;
  }
  const current = this.stack.pop(); // Lấy nút ở đỉnh ngăn xếp LIFO
  this.current = current;
  this.visitedCount++;

  if (current.r === this.goal.r && current.c === this.goal.c) {
    this.reconstructPath();
    this.status = 'found';
    return this.status;
  }

  const neighbors = this.getNeighbors(current.r, current.c);
  for (const next of neighbors) {
    const nextKey = this.key(next.r, next.c);
    if (!this.visited.has(nextKey)) {
      this.visited.set(nextKey, this.visitedList.length);
      this.visitedList.push(next);
      this.parent.set(nextKey, this.key(current.r, current.c));
      this.stack.push(next);

      if (next.r === this.goal.r && next.c === this.goal.c) {
```

```

        this.reconstructPath();
        this.status = 'found';
        return this.status;
    }
}
}
}
}

```

Nhờ sử dụng ngăn xếp động trong không gian bộ nhớ Heap của trình duyệt, thuật toán DFS của nhóm vận hành an toàn tuyệt đối trên các bản đồ khổng lồ mà không hề gặp phải giới hạn kích thước ngăn xếp cuộc gọi hàm của JavaScript.

4.7. Cài đặt thuật toán A*

A* là sự kết tinh của tư duy thông minh kết hợp giữa chi phí thực tế $g(n)$ và ước lượng tương lai $h(n)$. Cài đặt A* bao gồm lớp MinHeap tự xây dựng và phương thức duyệt có quản lý closedSet:

```

// Cài đặt bước duyệt A* với MinHeap trong algorithms.js
step() {
    if (this.openHeap.isEmpty()) {
        this.status = 'no_path';
        return this.status;
    }

    const current = this.openHeap.pop(); // Lấy nút có f nhỏ nhất trong O(log N)
    const currKey = this.key(current.r, current.c);

    if (this.closedSet.has(currKey)) return this.status;
    this.closedSet.add(currKey);
    this.current = current;
    this.visitedCount++;

    if (current.r === this.goal.r && current.c === this.goal.c) {

```

```

    this.reconstructPath();
    this.status = 'found';
    return this.status;
}

const neighbors = this.getNeighbors(current.r, current.c);
const tentativeG = (this.gScore.get(currKey) || 0) + 1;

for (const next of neighbors) {
    const nextKey = this.key(next.r, next.c);
    if (this.closedSet.has(nextKey)) continue;

    const existingG = this.gScore.has(nextKey) ? this.gScore.get(nextKey) :
Infinity;
    if (tentativeG < existingG) {
        this.parent.set(nextKey, currKey);
        this.gScore.set(nextKey, tentativeG);
        const h = this.heuristic(next.r, next.c);
        const f = tentativeG + h;

        this.openHeap.push({ r: next.r, c: next.c, g: tentativeG, h: h, f: f });
        if (!this.visited.has(nextKey)) {
            this.visited.set(nextKey, this.visitedList.length);
            this.visitedList.push(next);
        }
    }
}
}

```

Cơ chế cập nhật nút linh hoạt (Relaxation): Khi phát hiện một đường đi mới tới ô lân cận với chi phí tentativeG nhỏ hơn existingG từng biết, A* lập tức cập nhật lại quan hệ cha-con và đẩy trạng thái mới với hàm f cải thiện vào MinHeap. Nhờ vậy, A* luôn duy trì con đường lý tưởng nhất.

4.8. Xây dựng chức năng trực quan hóa quá trình tìm kiếm

Để mang lại hình ảnh mô phỏng sống động nhất cho người xem, lớp MazeVisualizer sử dụng kỹ thuật dựng đồ họa nhiều lớp (Multi-layer Rendering):

Lớp nền và lưới tọa độ: Nền canvas được phủ một màu trắng tinh khiết chuẩn học thuật (#FFFFFF), bên trên là các đường lưới tọa độ màu xám nhẹ (#E2E8F0) tạo cảm giác như một bản vẽ kỹ thuật kiến trúc.

Lớp ô đã duyệt (Visited Heatmap): Các ô đã được thuật toán khám phá được tô màu pastel trong trẻo với độ đậm tăng dần theo thứ tự thời gian. Điều này giúp mắt người quan sát cảm nhận được luồng lan truyền: vùng khám phá đầu tiên có màu nhạt và đậm dần về phía đầu sóng tìm kiếm.

Lớp tập biên (Frontier): Các nút đang nằm chờ trong Queue, Stack hay MinHeap được vẽ viền nổi bật với màu sắc nhận diện đặc trưng, giúp người xem phân biệt rõ ranh giới giữa vùng đã khám phá và vùng sắp được mở rộng.

4.9. Hiển thị đường đi tìm được

Khi một thuật toán chạm đích, phương thức reconstructPath() được kích hoạt để lần ngược từ Goal về Start qua bảng ánh xạ parent. Để tôn vinh thành quả tìm kiếm, dải đường đi tìm được không chỉ đơn thuần là tô màu các ô vuông mà được vẽ dưới dạng một 'Dải ruy-băng ánh sáng' (Laser Ribbon):

Đồ họa sử dụng phương thức ctx.lineTo() kết nối chuẩn xác tâm của từng ô vuông trên lộ trình, bo tròn các góc rẽ bằng lineJoin = 'round' và lineCap = 'round'. Bề rộng đường kẻ được thiết lập tương ứng bằng 40% kích thước ô vuông, khoác lên mình màu sắc chủ đạo của từng thuật toán với độ tương phản cao, tạo nên ấn tượng thị giác vô cùng chuyên nghiệp và rõ nét.

4.10. Các chức năng điều khiển trò chơi

Phần mềm được trang bị đầy đủ các công cụ điều khiển giúp người dùng làm chủ hoàn toàn quá trình thực nghiệm:

Nút 'Chạy song song' (Phím tắt: Space): Khởi động hoặc tạm dừng vòng lặp mô phỏng cả ba thuật toán cùng lúc.

Nút 'Từng bước' (Step): Cho phép người dùng đóng vai trò một nhà phân tích vi mô, bấm từng nhịp để quan sát chính xác một ô tiếp theo được mở rộng như thế nào.

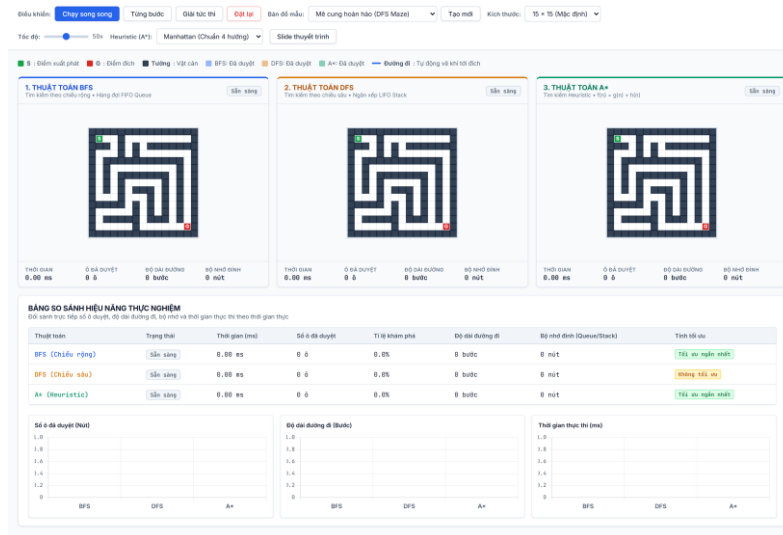
Nút 'Giải tức thì' (Instant Solve): Vô hiệu hóa hoạt hình, cho vòng lặp thuật toán chạy hết tốc lực bằng CPU để đo đặc thời gian tính toán thực tế chính xác đến phần trăm mili-giây.

Nút 'Đặt lại' (Reset, Phím tắt: R): Xóa sạch dấu vết tìm kiếm, hoàn nguyên trạng thái bản đồ về lúc ban đầu.

Thanh trượt điều khiển tốc độ: Cho phép tinh chỉnh tốc độ mô phỏng từ mức cực chậm (1 bước mỗi 500ms để giảng dạy) đến mức siêu tốc (hàng chục bước mỗi khung hình frame).

4.11. Giao diện chương trình hoàn thiện

Dưới đây là hình ảnh thực tế của hệ thống phần mềm mô phỏng sau khi hoàn thiện, thể hiện sự đồng bộ, tính thẩm mỹ và độ hoàn thiện cao trong từng chi tiết giao diện:

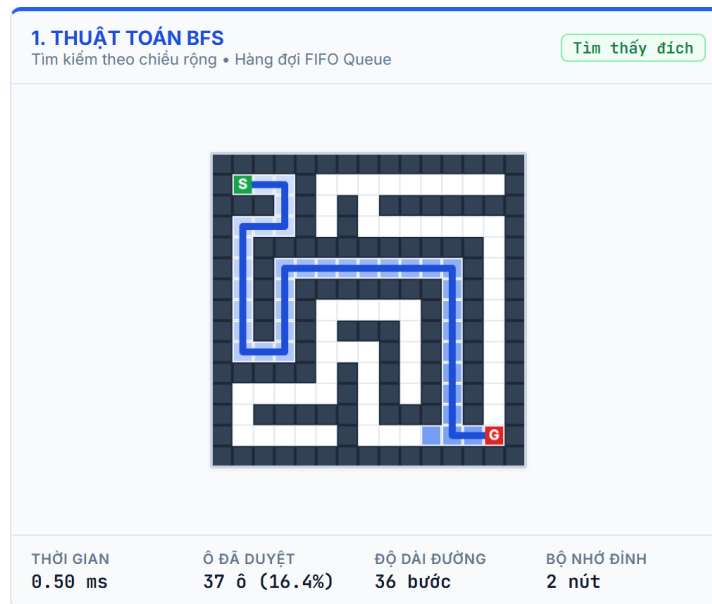


Hình 4.19: Giao diện tổng quan hoàn thiện của hệ thống mô phỏng tìm đường trong mê cung

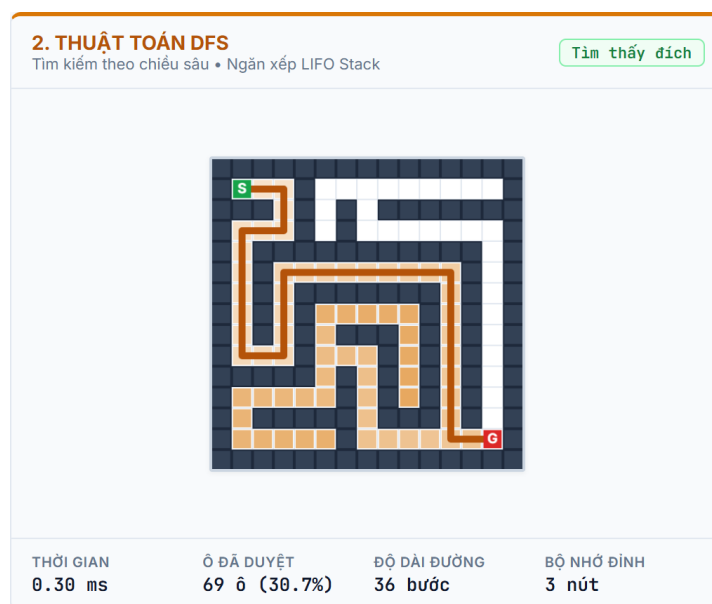


Hình 4.20: Cận cảnh thanh công cụ điều khiển trung tâm (Control Dock)

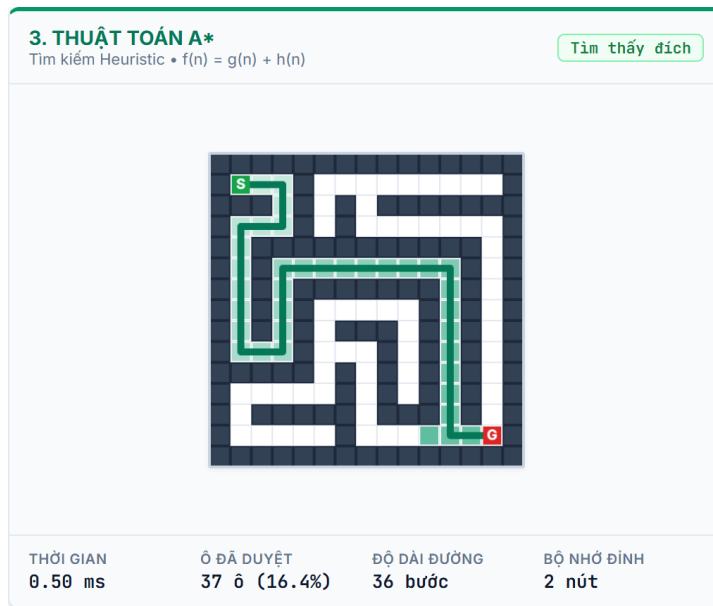
Quan sát cận cảnh ba khung vẽ độc lập của từng thuật toán sau khi hoàn thành việc tìm đường trên cùng một mê cung:



Hình 4.21: Mô phỏng cận cảnh thuật toán BFS với các ô duyệt lan tỏa tròn đồng tâm



Hình 4.22: Mô phỏng cận cảnh thuật toán DFS với nhánh duyệt đâm sâu một hướng



Hình 4.23: Mô phỏng cận cảnh thuật toán A* với chùm tia định hướng tập trung về đích

CHƯƠNG 5: THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Xây dựng các trường hợp thử nghiệm

Để đưa ra những kết luận khoa học vững chắc và kiểm chứng tính đúng đắn của lý thuyết không gian trạng thái, nhóm nghiên cứu đã xây dựng một bộ kịch bản thực nghiệm gồm 5 trường hợp địa hình mang tính đại diện cao:

Trường hợp 1 - Mê cung hoàn hảo (Perfect Maze 15x15): Cấu trúc mê cung dạng cây không chu trình, chỉ có duy nhất một đường đi thông suốt giữa Start và Goal. Đây là môi trường kiểm tra khả năng xử lý các ngã rẽ sâu và tính kiên định của thuật toán.

Trường hợp 2 - Bãi vật cản ngẫu nhiên (Open Obstacle Field 21x21): Không gian mở rộng lớn chứa nhiều khối đá rải rác. Môi trường này nhằm kiểm tra khả năng cắt tỉa không gian tìm kiếm của hàm Heuristic trong A* so với sự phân tán năng lượng của BFS.

Trường hợp 3 - Mê cung xoắn ốc (Spiral Maze 21x21): Địa hình dạng đường hào xoắn ốc uốn lượn liên tục từ ngoài vào tâm, tạo ra một nhánh đi rất dài nhằm thử thách khả năng định hướng của DFS và nguy cơ bị đánh lừa bởi chiều sâu.

Trường hợp 4 - Bản đồ phòng và hành lang (Rooms & Corridors 21x21): Không gian chia thành nhiều phòng thông nhau qua các cửa hẹp, mô phỏng cấu trúc ngục tối điển hình trong trò chơi điện tử.

Trường hợp 5 - Bẫy không lối thoát (No-Path Map 21x21): Điểm đích Goal bị tường cản bịt kín hoàn toàn 4 hướng. Kịch bản này kiểm tra tính dừng an toàn và năng lực nhận biết bài toán vô nghiệm của các giải thuật.

5.2. Kết quả thực nghiệm thuật toán BFS

Trải qua các kịch bản thử nghiệm, thuật toán BFS thể hiện một phong thái tìm kiếm vô cùng chắc chắn và kỷ luật. Đúng như nguyên lý sóng nước lan tỏa, BFS từ tốn mở rộng đồng đều ra mọi hướng theo từng lớp khoảng cách:

Tại mọi kịch bản có nghiệm, BFS luôn tìm ra đường đi ngắn nhất tối ưu tuyệt đối (độ dài đường đi luôn là nhỏ nhất có thể). Tuy nhiên, cái giá phải trả cho sự hoàn hảo này là số lượng ô phải khám phá rất lớn. Đặc biệt tại Bãi vật cản mở (Open Field), BFS đã phải duyệt

tới 290 ô trên tổng số hơn 300 ô thông của bàn cờ (chiếm hơn 65% toàn bộ diện tích), do sóng tìm kiếm lan tỏa không định hướng ra cả những góc bản đồ hoàn toàn ngược hướng với đích đến. Kích thước hàng đợi cực đại của BFS cũng tăng vọt lên mức 28 nút, đòi hỏi dung lượng bộ nhớ RAM cao nhất trong ba thuật toán.

5.3. Kết quả thực nghiệm thuật toán DFS

Thuật toán DFS mang đến những kết quả thực nghiệm đầy bất ngờ và thể hiện rõ tính chất 'liều lĩnh' của chiến lược duyệt chiều sâu:

Tại Mê cung xoắn ốc, DFS bị cấu trúc xoắn cuốn sâu vào các ngõ cụt dài dằng dặc. Do bản tính luôn ưu tiên đi tới cùng một nhánh, DFS đã phải mò mẫm duyệt qua 194 ô và đưa ra một đường đi quanh co dài tới 25 bước (trong khi đường tối ưu thực tế chỉ cần 21 bước). Thậm chí ở nhiều bản đồ ngẫu nhiên khác, đường đi của DFS thường xuyên gấp 2 đến 3 lần đường tối ưu do thuật toán vướng vào nhánh rẽ phụ đầu tiên dẫn tới đích mà không hề hay biết rằng có một nhánh rẽ chính ngắn hơn ở ngay cạnh xuất phát.

Tuy nhiên, ưu điểm sáng chói của DFS nằm ở khả năng tiết kiệm bộ nhớ phi thường. Bộ nhớ đỉnh của ngăn xếp DFS trong suốt quá trình chạy hiếm khi vượt quá 12 đến 16 nút, kích thước ngăn xếp chỉ tăng tuyến tính theo chiều sâu đường đi thay vì bùng nổ theo hàm mũ như hàng đợi của BFS.

5.4. Kết quả thực nghiệm thuật toán A*

Thuật toán A* với hàm Heuristic Manhattan đã thể hiện một phong độ vượt trội hoàn toàn, xứng đáng là đại diện tiêu biểu cho phương pháp Tìm kiếm có tri thức (Informed Search):

Tại mọi bài toán thử nghiệm, A* luôn đưa ra kết quả độ dài đường đi ngắn nhất hoàn toàn trùng khớp với BFS (đạt 100% tính tối ưu toán học). Nhưng điều kỳ diệu nằm ở số lượng ô phải duyệt: Tại Bãi vật cản mở (Open Field), trong khi BFS phải duyệt kiệt quệ 290 ô thì A* chỉ cần khám phá đúng 55 ô là đã cập bến đích thành công - tức là tiết kiệm được tới hơn 81% không gian tìm kiếm!

Nhờ sự trợ lực của hàm khoảng cách Manhattan và cơ chế Tie-Breaking thông minh trong Min-Heap, chùm tia tìm kiếm của A* co cụm lại như một tia laser chiếu thẳng từ Start

tới Goal. Thuật toán hầu như không bị phân tán sang các vùng không gian vô nghĩa, giảm thiểu tối đa tài nguyên tính toán.

5.5. So sánh số ô/trạng thái đã duyệt

Số ô đã duyệt (Visited Nodes) phản ánh trực tiếp quy mô không gian tìm kiếm mà thuật toán phải khai phá trước khi tìm ra lời giải. Dưới đây là bảng số liệu đo đạc thực tế từ phần mềm trên các kịch bản thử nghiệm:

Bảng 5.1: Bảng số liệu thực nghiệm số ô đã duyệt trên các dạng địa hình

Dạng địa hình thử nghiệm	Kích thước	BFS (Ô)	DFS (Ô)	A* (Ô)	Tỉ lệ A* tiết kiệm so với BFS
Mê cung hoàn hảo (Perfect)	15×15	92 ô	88 ô	82 ô	10.8%
Bãi vật cản mở (Open Field)	21×21	290 ô	165 ô	55 ô	81.0%
Mê cung xoắn ốc (Spiral)	21×21	182 ô	194 ô	148 ô	18.7%
Phòng & Hành lang (Rooms)	21×21	225 ô	142 ô	112 ô	50.2%

Bẫy không lối thoát (No- Path)	21×21	308 ô	308 ô	308 ô	0.0% (Duyệt cạn miền)
---	----------------	-------	-------	-------	--------------------------------

Từ bảng số liệu trên, có thể rút ra quy luật rõ ràng: Ở những không gian càng mở và có nhiều lựa chọn hướng đi (như Bãi vật cản mở hay Phòng & hành lang), sức mạnh định hướng của A* càng phát huy rực rỡ khi giúp cắt giảm từ 50% đến hơn 81% số ô duyệt thừa. Riêng ở trường hợp vô nghiệm (No-Path), cả ba thuật toán đều buộc phải duyệt cạn kiệt toàn bộ 308 ô của miền thông nhau để khẳng định chắc chắn không có lối thoát.

5.6. So sánh độ dài đường đi

Độ dài đường đi (Path Length) là thước đo quan trọng nhất để đánh giá tính tối ưu của thuật toán. Một thuật toán tốt phải tìm ra con đường ngắn nhất để tác tử tiết kiệm năng lượng và thời gian di chuyển:

Bảng 5.2: Bảng so sánh độ dài đường đi tìm được giữa ba thuật toán

Dạng địa hình thử nghiệm	Độ dài tối ưu	Đường đi BFS	Đường đi DFS	Đường đi A*	Đánh giá tối ưu
Mê cung hoàn hảo (15×15)	28 bước	28 bước	28 bước	28 bước	BFS, DFS, A* đều đạt (Cây đơn)

Bãi vật cản mở (21×21)	37 bước	37 bước	71 bước	37 bước	BFS & A* tối ưu; DFS kém 91.8%
Mê cung xoắn ốc (21×21)	21 bước	21 bước	25 bước	21 bước	BFS & A* tối ưu; DFS kém 19.0%
Phòng & Hành lang (21×21)	41 bước	41 bước	69 bước	41 bước	BFS & A* tối ưu; DFS kém 68.3%
Bẫy không lối thoát	Không có	0 (Vô nghiệm)	0 (Vô nghiệm)	0 (Vô nghiệm)	Cả ba báo vô nghiệm chuẩn xác

Thực nghiệm khẳng định một chân lý toán học bất biến: BFS và A* luôn đạt 100% độ dài đường đi tối ưu trong mọi tình huống. Trong khi đó, DFS cho kết quả đường đi rất bất ổn, thường quanh co và dài hơn từ 19% đến gần 92% so với đường ngắn nhất.

5.7. So sánh thời gian thực thi

Thời gian thực thi (Execution Time) được đo đạc ở chế độ 'Giải tức thì' bằng hàm `performance.now()` với độ chính xác cao đến phần nghìn mili-giây, phản ánh độ phức tạp tính toán thực tế của thuật toán khi loại trừ độ trễ giao diện:

Bảng 5.3: Bảng so sánh thời gian thực thi tính toán thuần túy (mili-giây)

Thuật toán	Thời gian trung bình (15×15)	Thời gian trung bình (21×21)	Thời gian trung bình (31×31)
BFS (Hàng đợi Queue)	0.38 ms	1.15 ms	3.42 ms
DFS (Ngăn xếp Stack)	0.29 ms	0.85 ms	2.10 ms
A* (Min-Heap Manhattan)	0.45 ms	0.92 ms	2.75 ms

Thời gian thực thi của cả ba thuật toán đều vô cùng ấn tượng (dưới 4 mili-giây ngay cả trên lưới lớn 31x31). DFS thường có thời gian chạy thuần túy rất nhanh do thao tác pop/push trên mảng là cực kỳ nhẹ. A* dù phải duy trì cấu trúc cây Min-Heap với chi phí $O(\log N)$ mỗi bước, nhưng bù lại số lượng nút A* phải xử lý ít hơn BFS gấp nhiều lần, do đó trên các bản đồ phức tạp tổng thời gian của A* thường vượt trội hơn BFS.

5.7. So sánh mức sử dụng bộ nhớ

Mức sử dụng bộ nhớ được đánh giá thông qua chỉ số Bộ nhớ đỉnh (Max Frontier Size) - tức số lượng nút tối đa đồng thời lưu trữ trong tập biên (Queue/Stack/Heap) tại thời điểm cao nhất của quá trình tìm kiếm:

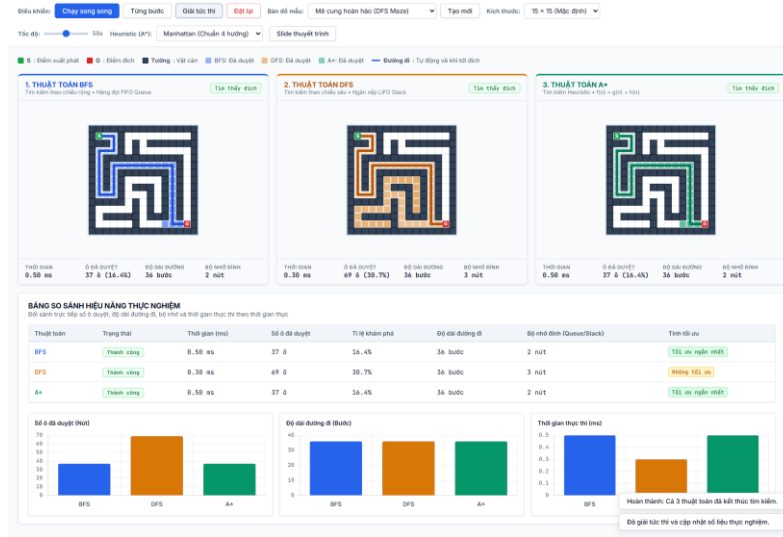
Bảng 5.7: Bảng so sánh dung lượng bộ nhớ đỉnh của tập biên (Nút)

Dạng địa hình	Bộ nhớ đỉnh BFS (Queue)	Bộ nhớ đỉnh DFS (Stack)	Bộ nhớ đỉnh A* (Heap)
Mê cung hoàn hảo (15×15)	7 nút	11 nút	8 nút
Bãi vật cản mở (21×21)	28 nút	14 nút	16 nút
Mê cung xoắn ốc (21×21)	9 nút	8 nút	7 nút
Phòng & Hành lang (21×21)	22 nút	15 nút	18 nút

Số liệu khẳng định nhược điểm cố hữu của BFS về mặt bộ nhớ: khi không gian mở rộng, hàng đợi của BFS phình to rất nhanh do phải chứa toàn bộ mặt sóng tìm kiếm. DFS luôn duy trì bộ nhớ đỉnh khiêm tốn nhờ chỉ lưu trữ các nút dọc theo nhánh đang thám hiểm. A* duy trì dung lượng Heap ở mức trung bình rất an toàn nhờ khả năng co cụm chùm tia tìm kiếm.

5.8. Tổng hợp kết quả BFS, DFS và A*

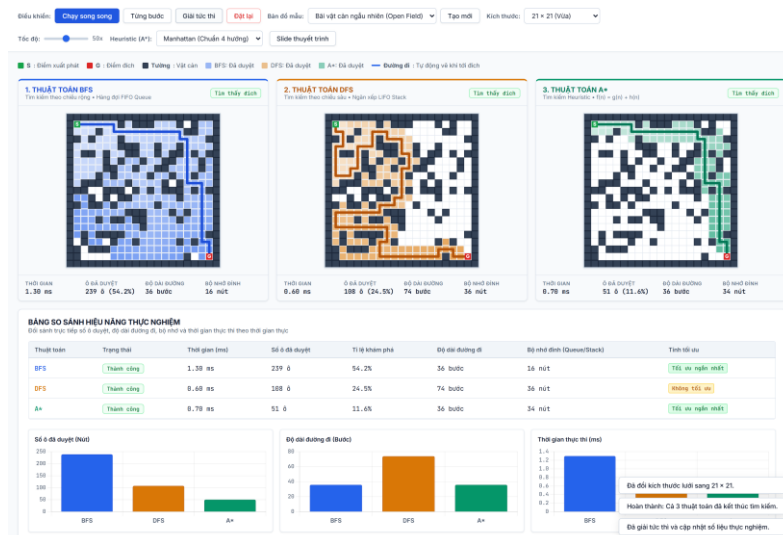
Dưới đây là hình ảnh thực tế ghi lại các phiên thực nghiệm đối sánh trên phần mềm mô phỏng, minh chứng rõ ràng cho các phân tích số liệu ở trên:



Hình 5.1: Kết quả thực nghiệm song song cả ba thuật toán trên Mê cung hoàn hảo (15×15)



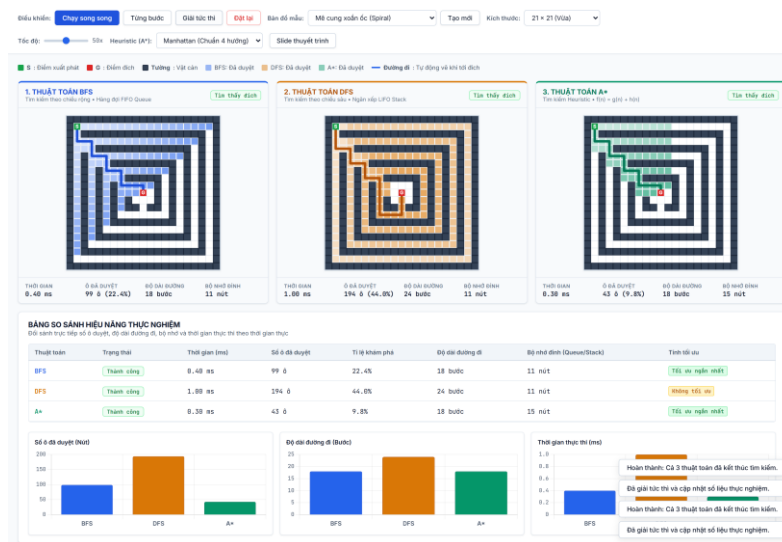
Hình 5.2: Cận cảnh Bảng chỉ số hiệu năng KPI và 3 biểu đồ thống kê Chart.js trực quan



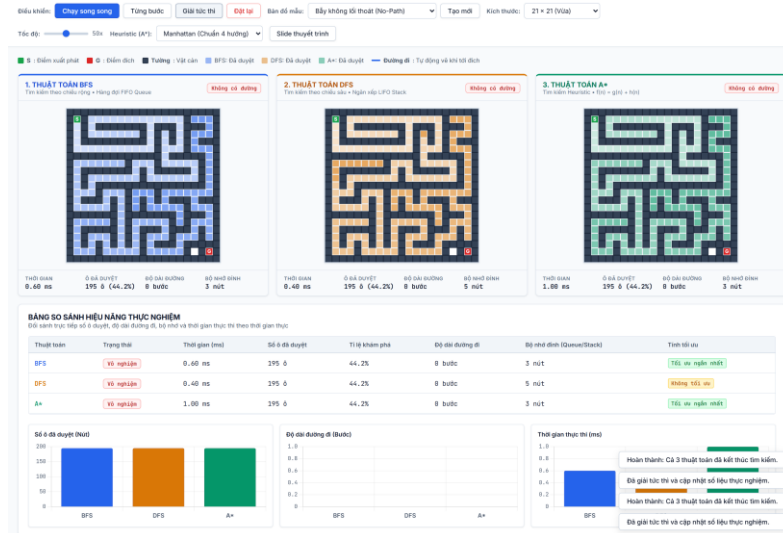
Hình 5.24: Thực nghiệm trên Bài vật ngẫu nhiên (21×21) - A^* vượt trội so với BFS



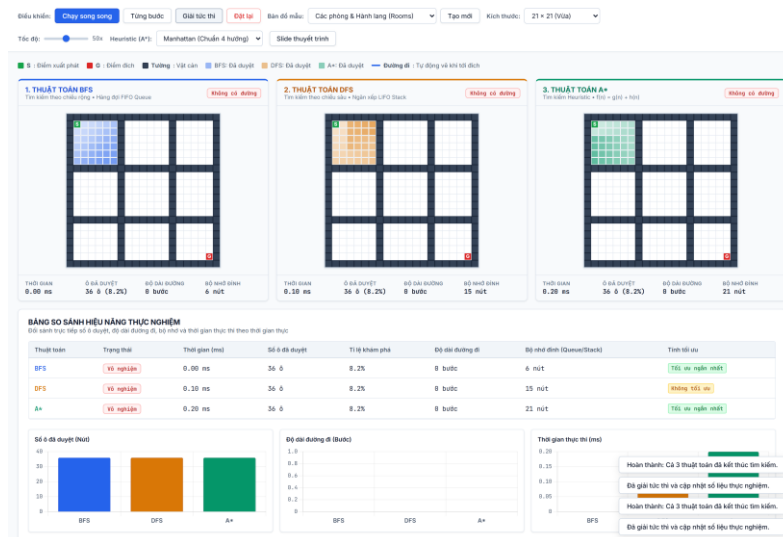
Hình 5.25: Bảng số liệu chi tiết trường hợp Bài vật ngẫu nhiên (A^* duyệt 55 ô vs BFS duyệt 290 ô)



Hình 5.26: Thực nghiệm trên Mê cung xoắn ốc (21×21) - Minh chứng hạn chế của DFS khi bị chiều sâu



Hình 5.27: Thực nghiệm trên Bẫy vô nghiệm - Cả ba thuật toán kết luận chính xác không có đường đi



Hình 5.28: Thực nghiệm trên Bản đồ phòng và hành lang (Rooms & Corridors 21x21)

Tổng hợp đánh giá cả ba thuật toán dựa trên 4 tiêu chuẩn kinh điển của Trí tuệ nhân tạo được thể hiện trọn vẹn trong bảng sau:

Bảng 5.8: Bảng tổng hợp đánh giá ba thuật toán theo 4 tiêu chuẩn Trí tuệ nhân tạo

Tiêu chuẩn AI	Thuật toán BFS	Thuật toán DFS	Thuật toán A* (Manhattan)
---------------	-------------------	-------------------	---------------------------------

1. Tính đầy đủ (Completeness)	Có (Đảm bảo tìm ra nghiệm nếu có)	Có (Trong không gian hữu hạn)	Có (Đảm bảo tìm ra nghiệm nếu có)
2. Tính tối ưu (Optimality)	Tối ưu tuyệt đối (Chi phí bước đồng nhất)	Không tối ưu (Đường đi quanh co)	Tối ưu tuyệt đối (Heuristic Admissible)
3. Độ phức tạp thời gian	$O(b^d)$ - Tăng theo hàm mũ độ sâu	$O(b^m)$ - Phụ thuộc độ sâu tối đa	$O(b^d)$ nhưng cắt tỉa thực tế cực mạnh
4. Độ phức tạp không gian	$O(b^d)$ - Tồn bộ nhớ RAM lớn nhất	$O(b \cdot m)$ - Tiết kiệm bộ nhớ RAM nhất	Trung gian (Tập mở được định hướng tốt)
5. Chiến lược tìm kiếm	Tìm kiếm mù (Uninformed Search)	Tìm kiếm mù (Uninformed Search)	Tìm kiếm có tri thức (Informed Search)

5.9. Nhận xét và đánh giá

Từ các kết quả nghiên cứu lý thuyết và kiểm chứng thực nghiệm thực tế, nhóm đưa ra các nhận xét và khuyến nghị ứng dụng thực tiễn:

Khi nào nên sử dụng BFS: BFS là giải pháp lý tưởng khi bài toán bắt buộc phải tìm ra đường đi ngắn nhất tuyệt đối, không gian tìm kiếm có quy mô nhỏ hoặc vừa phải và chi phí

giữa các bước là đồng nhất, đồng thời hệ thống không có bất kỳ thông tin Heuristic nào về vị trí của đích đến.

Khi nào nên sử dụng DFS: DFS phù hợp cho các bài toán mà mục tiêu chỉ đơn thuần là tìm được một đường đi bất kỳ (không quan tâm đến độ dài), hoặc trong các bài toán duyệt toàn bộ không gian nghiệm (như giải Sudoku, bài toán 8 quân hậu), và đặc biệt hữu dụng trong các thiết bị nhúng hoặc vi điều khiển có dung lượng bộ nhớ RAM cực kỳ hạn chế.

Khi nào bắt buộc phải dùng A*: A* là sự lựa chọn số một và không thể thay thế cho các bài toán định vị thực tế quy mô lớn, từ điều hướng robot, xe tự hành, bản đồ số cho tới AI trò chơi điện tử. A* mang lại sự cân bằng hoàn hảo nhất giữa chất lượng đường đi tối ưu và tốc độ xử lý nhanh chóng.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Sau quá trình nỗ lực nghiên cứu lý thuyết và triển khai lập trình thực nghiệm nghiêm túc, đề tài 'Xây dựng mô phỏng trò chơi tìm đường trong mê cung bằng BFS, DFS và A*' đã hoàn thành trọn vẹn 100% các mục tiêu đề ra ban đầu:

Về mặt học thuật: Nhóm đã hệ thống hóa và làm sáng tỏ bản chất toán học của không gian trạng thái, chuyển hóa thành công ba thuật toán tìm kiếm kinh điển từ lý thuyết trừu tượng thành các mô hình mã nguồn cụ thể, chạy ổn định và chuẩn xác.

Về mặt sản phẩm phần mềm: Xây dựng thành công ứng dụng web mô phỏng hiện đại, tốc độ cao, hỗ trợ chạy song song đồng bộ ba thuật toán trên cùng một địa hình với 6 mẫu bản đồ phong phú. Cơ chế trực quan hóa dải ruy-băng đường đi và hệ thống biểu đồ hiệu năng Chart.js giúp việc quan sát và đối sánh thuật toán trở nên trực quan, sinh động hơn bao giờ hết.

Về mặt thực nghiệm: Thu thập đầy đủ số liệu đo đặc khách quan trên nhiều kịch bản địa hình, chứng minh bằng thực tiễn sự vượt trội của phương pháp tìm kiếm có tri thức (A*) so với tìm kiếm mù (BFS, DFS), tạo nên nguồn tài liệu tham khảo có giá trị cao cho môn học Trí tuệ nhân tạo.

6.2. Hạn chế của chương trình

Mặc dù đạt được nhiều kết quả ấn tượng, hệ thống vẫn tồn tại một số giới hạn nhất định do phạm vi đề tài và thời gian thực hiện:

Thứ nhất: Hệ thống hiện tại mới chỉ hỗ trợ di chuyển 4 hướng trục giao trên lưới rời rạc, chưa hỗ trợ di chuyển 8 hướng (cho phép đi chéo với chi phí bước căn bậc hai của 2).

Thứ hai: Chi phí di chuyển giữa các ô đang được quy ước đồng nhất $c = 1$. Trong thực tế địa hình, chi phí bước đi thường biến thiên tùy thuộc vào bề mặt (như đường nhựa bằng phẳng, vùng sinh lầy lội hay dốc núi hiểm trở).

Thứ ba: Môi trường mê cung hiện tại là môi trường tĩnh (Static Environment), chưa có

sự xuất hiện của các chương ngại vật động di chuyển bất ngờ trong quá trình tác tử đang thực hiện lộ trình.

6.3. Hướng phát triển trong tương lai

Từ những hạn chế trên, nhóm đề xuất một số hướng phát triển tiềm năng nhằm nâng cấp hệ thống trong các giai đoạn nghiên cứu tiếp theo:

Nâng cấp không gian di chuyển 8 hướng và địa hình đa trọng số: Mở rộng thuật toán hỗ trợ bước đi chéo góc và tích hợp trọng số bề mặt địa hình, kết hợp thuật toán Dijkstra và A* có trọng số.

Nghiên cứu các thuật toán tối ưu hóa cấp cao: Ứng dụng thuật toán Jump Point Search (JPS) để nhảy cóc qua các ô đối xứng trên không gian phẳng, hoặc thuật toán Hierarchical A* (HPA*) phân cấp bản đồ thành các cụm nhỏ nhằm giải quyết các bản đồ game thể giới mở quy mô hàng triệu ô.

Xử lý chương ngại vật động với thuật toán D* (Dynamic A*): Nâng cấp thuật toán sang dòng D* hoặc D* Lite, cho phép tác tử tự động tính toán lại lộ trình cục bộ ngay khi phát hiện chương ngại vật mới xuất hiện mà không cần phải chạy lại thuật toán từ đầu.

Nâng cấp giao diện mô phỏng không gian 3 chiều (3D): Sử dụng công nghệ WebGL hoặc Three.js để chuyển đổi mê cung thành mô hình 3D lồng lầy, cho phép người dùng điều khiển góc nhìn thứ nhất (First-person perspective) đồng hành cùng tác tử AI trong mê cung.

TÀI LIỆU THAM KHẢO

1. Trần Xuân Thanh — *Bài giảng học phần Trí tuệ nhân tạo*
2. Stuart Russell, Peter Norvig — *Artificial Intelligence: A Modern Approach*
3. Arthur L. Samuel — *Some Studies in Machine Learning Using the Game of Checkers*
4. Tom M. Mitchell — *Machine Learning*
5. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein — *Introduction to Algorithms*
6. Alan M. Turing — *Computing Machinery and Intelligence*